

Full Length Article

A robust multi-view support vector machine with the RoBoSS loss function

Yash Arora^a, S. K. Gupta^a, M. Tanveer^{b,*}^a Department of Mathematics, Indian Institute of Technology Roorkee, Roorkee, 247667, India^b Department of Mathematics, Indian Institute of Technology Indore, Indore, Madhya Pradesh, 453552, India

ARTICLE INFO

Keywords:

Multi-view learning
 RoBoSS loss function
 Support vector machines
 Consensus and complementarity principles

ABSTRACT

Multi-view learning capitalizes the multiple representations of data and integrates their heterogeneous information to improve the learning performance. Several support vector machine (SVM)-based multi-view models have been proposed and shown excellent performance. However, they mainly rely on consensus-based strategies and neglect the complementary information. Moreover, these approaches lack robustness against noise, errors, and view-inconsistent patterns, common in multi-view datasets. To overcome these limitations, we propose a robust multi-view SVM framework that utilizes the RoBoSS loss function called RoBoSS-MvSVM. The proposed method explicitly integrates consensus and complementarity information across views, enriching the multi-view data representation and ensuring resilient learning. The RoBoSS loss function exhibits robustness, boundedness, sparsity, and smoothness, which makes it effective in handling noisy and inconsistent samples, while its classification-calibrated theoretical property ensures reliable generalization performance. We use the Nesterov accelerated gradient algorithm to solve the optimization problem of the proposed RoBoSS-MvSVM. Furthermore, the generalization capacity of the proposed RoBoSS-MvSVM technique is theoretically established through Rademacher complexity analysis. To further validate its robustness and effectiveness in multi-view learning, comprehensive experiments are conducted on 3 synthetic, 39 benchmark UCI and KEEL, and 45 Animal with Attribute datasets. The experimental results consistently demonstrate that RoBoSS-MvSVM outperforms existing baseline methods. Additionally, hyperparameter sensitivity analysis and statistical evaluation confirm the stability and significance of the proposed model's generalization performance.

1. Introduction

Multi-view learning (MvL) is an emerging machine learning paradigm in which the training data is characterized by distinct sets of features or multiple representations (Tang et al., 2024). Each view captures a different aspect of the same object and provides complementary information to the others. MvL is particularly useful in real-world applications where data is inherently multi-perspective in nature, such as image annotation (Kolenda et al., 2002), social network analysis (Yang et al., 2014), web page classification (Jing et al., 2015), and bioinformatics (Fu et al., 2022). Conventional approaches for handling multi-view data either amalgamate all views into a single representation to construct a single feature space, or process each view independently by learning separate functions and optimizing them jointly through redundant information across views. The first approach overlooks the statistical characteristics of each view and often leads to the curse of dimensionality, whereas the second approach may fail to fully exploit the complementary nature of different representations. Since multiple views characterize the same collection of objects through different fea-

ture spaces, they are inherently correlated. This has motivated substantial research efforts toward building MvL frameworks that are capable of integrating multiple views and exploiting their relationships to enhance the learning outcomes (Zheng et al., 2026, 2025).

MvL problems are commonly addressed through two key principles: consensus and complementarity (Zhao & Bu, 2022). Consensus-based approaches align the outputs of different views to achieve a common predictive objective. In contrast, complementarity-based methods focus on preserving the distinct characteristics of each view and combining them to form a more robust and comprehensive model. Building on the principles of consensus and complementarity, several MvL algorithms have been developed, which are broadly classified into co-training (Wang & Zhou, 2010; Wang et al., 2015) and co-regularization (Farquhar et al., 2005; Xie & Sun, 2015) approaches. Co-training methods iteratively train classifiers on each view, enhancing their mutual agreement while sharing complementary information. On the other hand, co-regularization methods incorporate the disagreement among views as an additional regularization term in the objective function, which is then minimized to achieve improved performance. Several

* Corresponding author.

E-mail addresses: yash_a@ma.iitr.ac.in (Y. Arora), s.gupta@ma.iitr.ac.in (S.K. Gupta), mtanveer@iiti.ac.in (M. Tanveer).<https://doi.org/10.1016/j.neunet.2026.108937>

Received 11 November 2025; Received in revised form 11 February 2026; Accepted 2 April 2026

Available online 4 April 2026

0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

SVM-based algorithms have been introduced in both categories, as SVM provides strong theoretical foundations and robustness in high-dimensional spaces, making it well-suited for MvL.

SVMs (Cortes & Vapnik, 1995) are developed as one of the most influential supervised learning models in machine learning due to their robustness, superior generalization ability, and interpretability. The essence of SVM lies in building an optimal separating hyperplane that maximizes the inter-class margin, thereby guaranteeing superior generalization performance on unseen data. To reduce the computational complexity of standard SVM, Jayadeva et al. (2007) introduced the twin support vector machine (TSVM), which finds two non-parallel hyperplanes by solving two smaller quadratic programming problems (QPPs). For a comprehensive overview of TSVM and its developments, the reader may refer to Tanveer et al. (2024). However, traditional SVMs and TSVM are intrinsically designed for single-view settings as they rely on a single feature representation of the data. This restriction limits their ability to exploit the diverse and complementary information inherent in multi-view settings. To address this shortcoming, researchers have extended the SVM framework to develop multi-view SVM models.

Among these approaches, the SVM-2K model proposed by Farquhar et al. (2005) is one of the earliest methods to generalize the large-margin principle of SVMs to MvL. It jointly trains classifiers on multiple views within a unified framework and incorporates additional constraints to minimize the disagreement between them. Another notable extension is the multi-view Laplacian SVM (Sun, 2011), which integrates the SVM with multi-view and manifold regularization, thus providing a bridge between supervised and semi-supervised learning. Xie and Sun (2015) proposed the multi-view twin SVM (MvTSVM) based on the co-regularization method by capitalizing on the effectiveness of twin SVM in multi-view settings. MvTSVM solves two QPPs of small size, yielding four non-parallel hyperplanes corresponding to the two views. Further, Tang et al. (2018b) proposed the multi-view privileged SVM (PSVM-2V) model, which integrates privileged information into the MvL framework. Unlike SVM-2K, which relies solely on the consensus principle across multiple views, PSVM-2V incorporates both consensus and complementarity principles. This extension leverages the complementary information across views and improves generalization performance. The development of multi-view SVM models has expanded significantly in recent years, with numerous advanced models such as multi-view robust double-sided TSVM (Ye et al., 2021), multi-view twin hypersphere SVM (Zhu et al., 2022), deep multi-view TSVM (Xie et al., 2023), and multi-view one-class SVM with privileged information (Xiao et al., 2024).

Further, to enhance the classification in MvL, Tang et al. (2018a) proposed the multi-view non-parallel SVM (MvNPSVM), which constructs two non-parallel hyperplanes corresponding to each view. By integrating the large-margin principle with the consensus mechanism, it utilizes both intra-class structure and inter-class information to achieve superior classification performance. Sun et al. (2018) proposed the multi-view generalized eigenvalue proximal SVM by adding a co-regularization term into the MvL framework to encourage the agreement across views. Subsequently, Houthuys et al. (2018) introduced the multi-view least squares SVM, which formulates the optimization problem in a primal-dual framework by minimizing error variables across views simultaneously. Quadir et al. (2025b) introduced the multi-view twin restricted kernel machine that integrates the strengths of kernel machines within a multi-view learning framework. Furthermore, Hu et al. (2025) introduced the multi-view large margin distribution machine (MvLDM), based on the margin distribution architecture. MvLDM learns the structural information of the training data by modeling the mean and variance of the margins across views, and simultaneously balances the consistency and complementarity principles. It flexibly assigns different kernel functions and regularization parameters to each view, with a coupling term that enforces joint consistency across views. In a recent study, Quadir and Tanveer (2024) introduced the multi-view twin parametric margin SVM (MvTPMSVM), where parametric margin hyperplanes are employed for each class. This design helps in reducing the influence of

heteroscedastic noise, thereby enhancing the stability of classification in multi-view scenarios.

Despite the proliferation of multi-view SVM models, a majority of the variants are intrinsically sensitive to the loss function, which plays a pivotal role in determining the robustness and generalization performance of the model. Traditional loss functions, such as the hinge loss (Farquhar et al., 2005), often struggle to capture the intricate interactions between the views and are susceptible to noise or irrelevant features. To address these shortcomings, Lou and Xie (2023) proposed the intuitionistic fuzzy multi-view SVMs with an insensitive pinball loss function to address multi-view classification problems. It effectively maximizes the quantization distance and better exploits complementary view information by integrating the pinball loss into MvL. Further, Tang et al. (2021) proposed a multi-view SVM model with the LINEX loss that takes advantage of asymmetry and the smooth nature of the loss function. It enhances the robustness against noisy, error-prone, and view-inconsistent samples, while effectively modeling cross-view dependencies to improve generalization performance. Quadir et al. (2025a) proposed a multi-view SVM that leverages the wave loss function to unify consensus and complementarity principles while enhancing robustness against noise and inconsistencies in multiview datasets. However, multi-view extensions based on pinball, wave, and LINEX loss functions improve robustness to some extent but still exhibit inherent limitations. The pinball loss demonstrates effectiveness in handling noisy samples but lacks boundedness and smoothness, which may compromise stability during the training process. Moreover, the LINEX loss remains sensitive to extreme outliers and does not promote sparsity in the decision models. These limitations highlight the necessity for adopting a more flexible and bounded loss function to enhance the robustness and reliability of MvL models. Further details on the advantages and limitations of existing multi-view variants are summarized in the Table 1.

In this paper, we propose a novel robust multi-view support vector machine with the RoBoSS loss function (RoBoSS-MvSVM). The RoBoSS loss function (Akhtar et al., 2025, 2026) has shown remarkable performance in single-view SVM and TSVM through its ability to resist outliers, constrain the effects of extreme error, and achieve sparsity by assigning zero penalty to correctly classified instances. Building upon these merits, we extend the RoBoSS loss into the multi-view framework to enhance generalization performance and resilience against noisy samples across multiple views. Furthermore, in the proposed RoBoSS-MvSVM, we incorporate both the consensus and complementarity principles. A co-regularization term is introduced into the objective function to align the predictors of different views and at the same time, complementarity information is exploited through an adaptive weighting mechanism that allows the model to prioritize the most informative and discriminative features from each view. The resulting optimization problem is efficiently addressed using the Nesterov accelerated gradient (NAG) algorithm. The main contributions of this study can be summarized as follows:

1. We propose a novel multi-view SVM model, RoBoSS-MvSVM, by incorporating the RoBoSS loss function, which ensures robustness, boundedness, sparsity, and smoothness in the learning process.
2. We exploit both the consensus and complementarity principles in the proposed framework, enabling the model to align shared information across views while capitalizing on heterogeneous information to enhance overall performance.
3. We conduct comprehensive empirical evaluations on synthetic, 39 UCI/KEEL, and Animals with Attributes (AwA) datasets, demonstrating that the proposed approach consistently outperforms state-of-the-art methods and exhibits strong versatility across diverse application scenarios.
4. We further perform sensitivity analyses of hyperparameters and conduct statistical tests to validate the robustness, stability, and significance of the results.

Table 1

Brief review of SVM and multi-view SVM variants.

Method	Setting	Algorithm	Advantages	Limitations
SVM-2K (Farquhar et al., 2005)	Two view	QP solver	• First multi-view SVM framework. • Enforces cross-view consensus.	• Not suitable for large-scale QPPs. • Limited robustness to noise.
Mv-Laplacian SVM (Sun, 2011)	Multi-view		• Incorporates manifold and multi-view regularization. • Bridges supervised and semi-supervised learning.	• Requires multiple regularization assumptions. • Sensitive to noise and hyperparameters.
MvTSVM (Xie & Sun, 2015)	Two view	QP solver	• Learns two non-parallel classifiers. • Enforces projection-based view agreement.	• Requires multiple matrix inversions. • Assumes uniform or predefined noise structure.
MvNPSVM (Tang et al., 2018a)	Multi-view	ADMM	• Finds non-parallel classifiers under the large margin principle. • Exploits both within-class and between-class information.	• Assumes both the views are equally important. • Computationally expensive.
PSVM-2V (Tang et al., 2018b)	Two view	QP solver	• Uses privileged information across views. • Imposes consensus via cross-view regularization.	• Not suitable for large-scale datasets. • Primarily designed for a two-view setting only.
LINEX-MvSVM (Tang et al., 2021)	Multi-view	Iterative + CVX	• Incorporates the asymmetric LINEX loss to penalize error-prone samples. • Exploits view complementarity information by adaptively learning the combination weights for each view.	• Lacks robustness to noise and outliers. • Adopt CVX-based alternating optimization strategy which is computationally expensive.
MVLDM (Hu et al., 2025)	Multi-view	ADMM	• Extends margin distribution to multi-view learning. • Exploits both complementarity and consistency principles.	• Computationally expensive. • High sensitivity to the tradeoff parameter.
Wave-MvSVM (Quadir et al., 2025a)	Multi-view	GD + ADMM	• Uses smooth, asymmetric, and bounded loss function. • Enhances robustness to noise and outliers	• Requires careful tuning of hyperparameters. • Computationally expensive for large-scale datasets.
RoBoSS-SVM (Akhtar et al., 2025)	Single view	NAG	• Integrates a robust, bounded, sparse, and smooth loss into a single-view SVM. • Significantly improves the performance of single-view SVM.	• Limited to single-view datasets. • Requires manual tuning of loss hyperparameters.
RoBoSS-MvSVM (Proposed)	Multi-view	NAG	• Improves robustness in MvL by using the RoBoSS loss and reduces outlier influence across views. • Enables fast gradient-based optimization due to smooth loss. • Jointly exploits consensus and view-specific information.	• Involves manual tuning of loss parameters.

The remainder of this article is structured as follows: Section 2 provides a comprehensive review of related work. Section 3 presents the proposed methodology, including the RoBoSS loss function, the mathematical formulations of the proposed RoBoSS-MvSVM model, and the optimization algorithm for solving the resulting problem. Section 4 establishes the theoretical foundations of the proposed method. Section 5 presents the outcomes of empirical results conducted on synthetic, UCI/KEEL, and AWA datasets using accuracy, sensitivity, specificity, F-measure, and G-mean. The convergence analysis, sensitivity analysis of hyperparameter, experiments with added Gaussian and label noise, and ablation studies are also presented in this section. Section 6 discusses the statistical analyses of the experimental outcomes. Finally, Section 7 concludes the article with the directions for future research.

2. Related work

This section briefly reviews related MvL techniques, including SVM-2K and MvTSVM. Let $\{(x_i^{[1]}, x_i^{[2]}, y_i), i = 1, 2, \dots, m\}$ denote the m training patterns, where $x_i^{[1]} \in \mathbb{R}^{d_1}$ and $x_i^{[2]} \in \mathbb{R}^{d_2}$ represent the data points from the first and second views, respectively, and $y_i \in \{+1, -1\}$ denotes the corresponding class labels. Here, d_1 and d_2 denote the number of features corresponding to the first and second view, respectively. Let ψ_1 and ψ_2 represent the feature mappings that map the input data into high-dimensional spaces, and let K_1 and K_2 be the corresponding kernel functions for the two views.

2.1. Two view learning: SVM-2K

Farquhar et al. (2005) introduces the SVM-2K technique to learn from two distinct views of the same training data. It extends the conventional SVM framework by simultaneously training two hyperplanes, one for each view, with an additional constraint to align their predictions. SVM-2K finds the two hyperplanes $w^{[1]T} \psi_1(x^{[1]}) + b^{[1]} = 0$ and $w^{[2]T} \psi_2(x^{[2]}) + b^{[2]} = 0$ corresponds to each view by solving the following optimization problems:

$$\begin{aligned}
 \min \quad & \frac{1}{2} \|w^{[1]}\|^2 + \frac{1}{2} \|w^{[2]}\|^2 + C_1 \sum_{i=1}^m \xi_i^{[1]} + C_2 \sum_{i=1}^m \xi_i^{[2]} + D \sum_{i=1}^m \eta_i \\
 \text{s.t.} \quad & \left| \langle w^{[2]}, \psi_2(x_i^{[2]}) \rangle + b^{[2]} - \langle w^{[1]}, \psi_1(x_i^{[1]}) \rangle - b^{[1]} \right| \leq \epsilon + \eta_i, \\
 & y_i (\langle w^{[2]}, \psi_2(x_i^{[2]}) \rangle + b^{[2]}) \geq 1 - \xi_i^{[2]}, \\
 & y_i (\langle w^{[1]}, \psi_1(x_i^{[1]}) \rangle + b^{[1]}) \geq 1 - \xi_i^{[1]}, \\
 & \xi_i^{[1]} \geq 0, \quad \xi_i^{[2]} \geq 0, \quad \eta_i \geq 0, \quad i = 1, \dots, m,
 \end{aligned} \tag{1}$$

where $C_1, C_2, D > 0$ are penalty parameters, $\epsilon > 0$ is an insensitive threshold, $\xi_i^{[1]}, \xi_i^{[2]}, \eta_i$ denote the slack variables. The dual problem corresponding to (1) can be written as:

$$\begin{aligned}
 \max \quad & -\frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \left(\alpha_i^{[1]} \alpha_j^{[1]} K_1(x_i, x_j) + \alpha_i^{[2]} \alpha_j^{[2]} K_2(x_i, x_j) \right) + \sum_{i=1}^m (\beta_i^{[1]} + \beta_i^{[2]}) \\
 \text{s.t.} \quad & \alpha_i^{[1]} = \beta_i^{[1]} y_i - \gamma_i^+ + \gamma_i^-, \\
 & \alpha_i^{[2]} = \beta_i^{[2]} y_i + \gamma_i^+ - \gamma_i^-,
 \end{aligned}$$

$$\begin{aligned} \sum_{i=1}^m \alpha_i^{[1]} &= \sum_{i=1}^m \alpha_i^{[2]} = 0, \\ 0 &\leq \gamma_i^+ + \gamma_i^-, \quad \gamma_i^+, \gamma_i^- \leq D, \\ 0 &\leq \rho_i^{[1]/[2]} \leq C_{1/2}, \end{aligned} \quad (2)$$

where $\rho_i^{[1]}, \rho_i^{[2]}, \gamma_i^+, \gamma_i^-$ are the Lagrangian multipliers, $K_1(x_i, x_j) = \langle \psi_1(x_i^{[1]}), \psi_1(x_j^{[1]}) \rangle$ and $K_2(x_i, x_j) = \langle \psi_2(x_i^{[2]}), \psi_2(x_j^{[2]}) \rangle$ are the kernel functions corresponding to first and second views, respectively. The prediction function for each view is defined as:

$$f_{[1]/[2]}(x) = \sum_{i=1}^m \alpha_i^{[1]/[2]} K_{1/2}(x_i, x) + b^{[1]/[2]}. \quad (3)$$

2.2. Multi-view twin SVM

Multi-view twin support vector machine (MvTSVM) (Xie & Sun, 2015) extends the TSVM framework to multi-view learning. Unlike traditional SVM, which determines a single separating hyperplane, TSVM constructs the two non-parallel hyperplanes, each positioned closer to one class while being farther from the other. For a dataset represented by two distinct views, MvTSVM aims to construct four non-parallel hyperplanes. Let $A^{[1]}$ and $A^{[2]}$ denote the positive class samples corresponding to views 1 and 2, respectively, and let $B^{[1]}$ and $B^{[2]}$ denote the negative class samples for views 1 and 2, respectively. For simplicity, define $A_1 = [A^{[1]}, e]$, $A_2 = [A^{[2]}, e]$, $B_1 = [B^{[1]}, e]$, $B_2 = [B^{[2]}, e]$, where e is the vector of ones of appropriate dimensions and $u_1 = [w_1^{[1]}, b_1^{[1]}]^T$, $u_2 = [w_1^{[2]}, b_1^{[2]}]^T$, $v_1 = [w_2^{[1]}, b_2^{[1]}]^T$, $v_2 = [w_2^{[2]}, b_2^{[2]}]^T$. The optimization problem for MvTSVM is then formulated as:

$$\begin{aligned} \min \quad & \frac{1}{2} \|A_1 u_1\|^2 + \frac{1}{2} \|A_2 u_2\|^2 + C_1 e_1^T \xi_1 + C_2 e_2^T \xi_2 + D_1 e_1^T \eta_1 \\ \text{s.t.} \quad & |A_1 u_1 - A_2 u_2| \leq \epsilon + \eta_1, \\ & -B_1 u_1 \geq e_2 - \xi_1, \\ & -B_2 u_2 \geq e_2 - \xi_2, \\ & \xi_1 \geq 0, \xi_2 \geq 0, \eta_1 \geq 0, \end{aligned} \quad (4)$$

and

$$\begin{aligned} \min \quad & \frac{1}{2} \|B_1 v_1\|^2 + \frac{1}{2} \|B_2 v_2\|^2 + C_3 e_1^T \xi_3 + C_4 e_1^T \xi_4 + D_2 e_2^T \eta_2 \\ \text{s.t.} \quad & |B_1 v_1 - B_2 v_2| \leq \epsilon + \eta_2, \\ & A_1 v_1 \geq e_1 - \xi_3, \\ & A_2 v_2 \geq e_1 - \xi_4, \\ & \xi_3 \geq 0, \xi_4 \geq 0, \eta_2 \geq 0, \end{aligned} \quad (5)$$

where $C_1, C_2, C_3, C_4, D_1, D_2 > 0$ are positive penalty parameters, and $\epsilon > 0$ is an insensitive parameter; $\xi_1, \xi_2, \xi_3, \xi_4, \eta_1, \eta_2$ are the slack variables, respectively.

To assign a new testing pattern $x = (x^{[1]}, x^{[2]})$ to +1 or -1 class, the following decision rule is applied:

$$f(x) = \frac{1}{2} \left((|x^{[1]T} w_1^{[1]} + b_1^{[1]}| + |x^{[2]T} w_1^{[2]} + b_1^{[2]}|) - (|x^{[1]T} w_2^{[1]} + b_2^{[1]}| + |x^{[2]T} w_2^{[2]} + b_2^{[2]}|) \right). \quad (6)$$

The pattern is classified as in +1 class when $f(x) < 0$ and otherwise in -1 class.

3. Proposed work

In this section, we integrate the RoBoSS loss function into a multi-view SVM framework to develop a novel approach called RoBoSS-MvSVM. As a robust, bounded, sparse, and smooth loss function, RoBoSS loss enhances model stability while effectively resisting outliers. The proposed method combines the RoBoSS loss function with the consensus and complementarity principle to facilitate efficient MvL. The consensus principle is concerned with adding a co-regularization term that

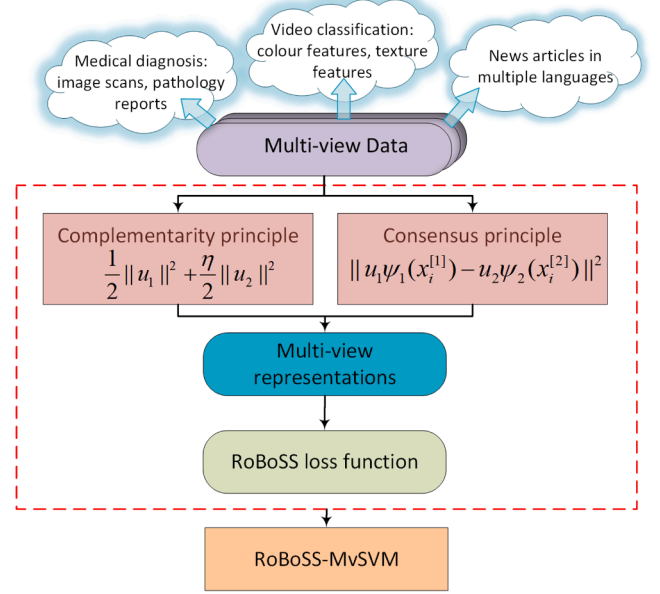


Fig. 1. Flowchart of the RoBoSS-MvSVM framework.

minimizes the discrepancies among multiple views, while complementary information is captured by optimizing the weighting coefficients of different views. The two-view learning framework of the proposed RoBoSS-MvSVM is depicted in Fig. 1.

3.1. RoBoSS loss function

Akhtar et al. (2025) proposed the RoBoSS loss function to achieve robustness, boundedness, sparsity, and smoothness in supervised machine learning. Its mathematical formulation is given by

$$\mathcal{L}_{\text{RoBoSS}}(t) = \begin{cases} \lambda \{1 - (at + 1) \exp(-at)\}, & t > 0, \\ 0, & t \leq 0, \end{cases} \quad (7)$$

where $a > 0$ and $\lambda > 0$ denote the shape and bound parameters, respectively. Fig. 2 shows the RoBoSS loss function for different values of a and λ . It exhibits several distinctive characteristics, including non-convexity, boundedness, sparsity-inducing behavior, smoothness, and robustness against outliers. In contrast to conventional loss functions like hinge loss, pinball loss, or LINEX loss, the RoBoSS loss function integrates all these characteristics in a unified framework to improve both adaptability and stability throughout the training process. Its bounded nature mitigates the adverse effects of extreme outliers, while the sparsity property ensures that only the most informative samples contribute to model training. Further, the RoBoSS loss function possesses the following key properties:

- The loss function is capped by parameter λ for samples with $t > 0$, which limits the influence of outliers, and assigns zero loss to correctly classified samples, thereby inducing sparsity.
- It involves two key parameters: the shape parameter a , which controls the steepness of penalization, and the bounding parameter λ , which sets the maximum loss value.
- When $\lambda = 1$ and $a \rightarrow +\infty$, the function converges pointwise to the “0–1” loss function.

To clearly highlight the benefits of the RoBoSS loss function, Table 2 provides a concise comparison of its key properties against several baseline loss functions.

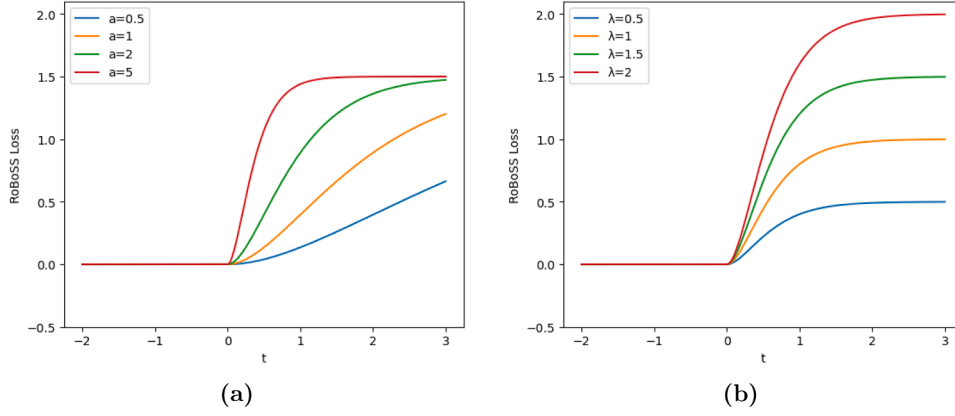


Fig. 2. RoBoSS loss function with (a) varying a for fixed $\lambda = 1.5$, and (b) varying λ for fixed $a = 3$.

Table 2

Comparison of different loss functions.

Characteristics → Loss function ↓	Smooth	Convex	Bounded	Sparse	Robust to outliers
Hinge loss (Farquhar et al., 2005)	×	✓	×	✓	×
Pinball loss (Quadri & Tanveer, 2025)	×	✓	×	×	×
LINEX loss (Tang et al., 2021)	✓	✓	×	×	×
Wave loss (Akhtar et al., 2024)	✓	×	✓	×	✓
RoBoSS loss (Akhtar et al., 2025)	✓	×	✓	✓	✓

Table 3

Details of multi-view features for UCI/KEEL and AWA datasets.

Dataset group	No. of datasets	View 1 construction	View 2 construction
UCI/KEEL	39	Original feature vector.	PCA projection of the same original features by retaining 95% cumulative variance.
AWA	45	2000-dim L1-normalized speeded-up robust features (SURF).	252-dimensional histogram of oriented gradients (HOG) descriptors.

Table 4

Comparison of the proposed RoBoSS-MvSVM with baseline models on synthetic datasets.

Dataset	SVM-2K (Farquhar et al., 2005) Accuracy (%)	MvTSVM (Xie & Sun, 2015) Accuracy (%)	PSVM-2V (Tang et al., 2018b) Accuracy (%)	MvNPSVM (Tang et al., 2018a) Accuracy (%)	MvLDM (Hu et al., 2025) Accuracy (%)	MvTPMSVM (Quadri & Tanveer, 2024) Accuracy (%)	RoBoSS-MvSVM (Proposed) Accuracy (%)
Synthetic 1	92.54 ± 1.45	88.47 ± 5.89	93.62 ± 2.31	85.24 ± 4.56	94.37 ± 0.03	94.42 ± 2.11	94.62 ± 1.62
Synthetic 2	95.71 ± 2.42	92.42 ± 4.12	96.75 ± 0.32	90.46 ± 3.51	96.50 ± 0.24	97.17 ± 1.55	97.66 ± 0.61
Synthetic 3	100.00 ± 0.00	100.00 ± 0.00	98.24 ± 1.05	96.75 ± 1.54	100.00 ± 0.00	100.00 ± 0.00	100.00 ± 0.00
Average	96.08	93.63	96.20	90.82	96.96	97.20	97.43

3.2. Multi-view learning with RoBoSS loss

This subsection introduces the mathematical formulation of the multi-view support vector machine with RoBoSS loss (RoBoSS-MvSVM), which integrates both consensus and complementarity principles. The fusion of RoBoSS with a multi-view SVM framework constitutes a substantial advancement toward more robust supervised learning approaches. Suppose each training pattern is represented by V distinct views. Let $w^{[\theta]T} \phi_1(x^{[\theta]}) + b^{[\theta]} = 0$ denote the optimal separating hyperplane, and let $u_\theta = [w^{[\theta]} \quad b^{[\theta]}]^T$ represent the weight vector corresponding to each view $\theta \in \{1, 2, \dots, V\}$. The corresponding optimization problem of RoBoSS-MvSVM is given by:

$$\begin{aligned}
 \min \quad & \sum_{\theta=1}^V \sum_{i=1}^m D_\theta \left((u_\theta \psi_\theta(x_i^{[\theta]})) - (u_\omega \psi_\omega(x_i^{[\omega]})) \right)^2 \\
 & + \sum_{\theta=1}^V \sum_{i=1}^m \frac{C_\theta}{m} \left(\lambda_\theta \left(1 - (a_\theta \{\xi_i^{[\theta]}\}_+ + 1) \exp(-a_\theta \{\xi_i^{[\theta]}\}_+) \right) \right) + \sum_{\theta=1}^V \frac{1}{2} \|u_{[\theta]}\|^2 \\
 \text{s.t.} \quad & y_i (u_1 \psi_1(x_i^{[1]})) = 1 - \xi_i^{[1]}, \\
 & \vdots
 \end{aligned}$$

$$y_i (u_\theta \psi_\theta(x_i^{[\theta]})) = 1 - \xi_i^{[\theta]},$$

$$\theta = 1, 2, \dots, V, \quad i = 1, 2, \dots, m,$$

(8)

where $\{\xi_i^{[\theta]}\}_+ = \xi_i^{[\theta]}$ if $\xi_i^{[\theta]} > 0$ and 0 otherwise, $C_\theta > 0$ denotes the penalty parameters, $D_\theta > 0$ denotes the regularization parameters, a_θ and λ_θ are the loss function parameters, and the function $\psi_\theta(\cdot)$ represents the feature mapping associated with the kernel function. The significance of each term in the RoBoSS-MvSVM optimization formulation is summarized below:

1. The first term $\sum_{\theta,\omega=1, \theta \neq \omega}^V \sum_{i=1}^m D_\theta \left((u_\theta \psi_\theta(x_i^{[\theta]})) - (u_\omega \psi_\omega(x_i^{[\omega]})) \right)^2$ represents the consensus regularization term, which penalizes discrepancies in the decision function outputs of the same sample across different views. It encourages agreement among views and thereby improves the overall classification performance of the model.
2. The second term, $\lambda_\theta (1 - (a_\theta \{\xi_i^{[\theta]}\}_+ + 1) \exp(-a_\theta \{\xi_i^{[\theta]}\}_+))$ applies the RoBoSS loss function across each view to handle the misclassifications in RoBoSS-MvSVM. It assigns zero loss to correctly predicted samples and ensures that only the most informative samples contribute to training, yielding more optimized models.
3. The third term u_θ serves as the regularization component for view θ , which controls the complexity of the decision hyperplane in that

Table 5

Comparison of the proposed RoBoSS-MvSVM with baseline models on UCI and KEEL datasets.

Dataset	SVM-2K (Farquhar et al., 2005) (C_1, σ)	MvTSVM (Xie & Sun, 2015) (C_1, σ)	PSVM-2V (Tang et al., 2018b) (C_1, σ)	MvNPSVM (Tang et al., 2018a) (C_1, D, σ)	MvLDM (Hu et al., 2025) (C_1, v, σ)	MvTPMSVM (Quadir & Tanveer, 2024) (C_1, C_2, σ)	RoBoSS-MvSVM (Proposed) ($a, \lambda, C_1, D, \sigma$)
	Accuracy (%) Time (s)	Accuracy (%) Time (s)	Accuracy (%) Time (s)	Accuracy (%) Time (s)	Accuracy (%) Time (s)	Accuracy (%) Time (s)	Accuracy (%) Time (s)
appendicitis	(1, 0.5) 80.22 $\pm$ 3.75 0.07	(0.1, 4) 85.89 $\pm$ 0.03 0.16	(1, 0.125) 83.03 $\pm$ 4.18 0.64	(1, 0.5, 1) 80.22 $\pm$ 3.75 1.27	(10, 1, 0.5) 83.98 $\pm$ 0.06 0.15	(100, 100, 0.0625) 84.03 $\pm$ 5.06 0.18	(1, 2, 0.3, 0.01, 1, 0.0625) 86.84 $\pm$ 6.02 0.16
auction	(0.01, 0.125) 84.58 $\pm$ 4.24 152.61	(0.01, 4) 86.10 $\pm$ 1.21 137.80	(1, 0.125) 87.52 $\pm$ 3.26 117.18	(1, 0.05, 0.125) 87.18 $\pm$ 3.58 158.15	(1, 0.1, 0.5) 85.00 $\pm$ 0.05 34.83	(0.1, 1, 0.03125) 85.46 $\pm$ 2.49 42.75	(2, 1, 0.01, 10, 0.0625) 88.06 $\pm$ 1.56 23.98
aus	(1, 0.125) 81.59 $\pm$ 2.64 1.78	(100, 16) 61.45 $\pm$ 0.02 8.66	(1, 0.25) 82.03 $\pm$ 2.87 14.06	(0.1, 0.5, 0.125) 85.51 $\pm$ 2.56 18.54	(1, 1, 1) 85.65 $\pm$ 0.03 14.64	(100, 100, 0.0625) 83.91 $\pm$ 3.13 13.86	(0.1, 0.1, 0.01, 10, 0.0625) 83.48 $\pm$ 1.80 6.12
bank	(1, 0.125) 86.60 $\pm$ 3.25 212.02	(0.01, 0.125) 86.04 $\pm$ 1.26 374.73	(10, 0.25) 84.05 $\pm$ 2.58 471.89	(1, 0.05, 0.125) 81.57 $\pm$ 2.31 555.16	(10, 1, 0.25) 86.64 $\pm$ 0.06 184.77	(1, 0.1, 0.03125) 84.72 $\pm$ 3.14 227.28	(1, 0.1, 1, 1, 1) 88.48 $\pm$ 2.15 125.28
bankruptcy	(1, 0.0625) 94.46 $\pm$ 1.02 357.32	(0.1, 8) 94.75 $\pm$ 1.39 673.04	(1, 0.125) 95.91 $\pm$ 1.48 1726.16	(100, 5, 1) 93.74 $\pm$ 1.98 2253.72	(100, 10, 1) 95.95 $\pm$ 0.01 453.77	(1, 0.1, 0.03125) 95.70 $\pm$ 1.47 1427.38	(1, 1, 0.01, 1, 4) 96.77 $\pm$ 1.24 242.45
breast cancer	(1, 0.125) 96.84 $\pm$ 1.92 7.59	(100, 2) 74.55 $\pm$ 0.03 6.79	(0.01, 1) 92.62 $\pm$ 2.87 7.71	(0.1, 5, 0.0625) 83.48 $\pm$ 9.81 12.84	(1, 0.1, 8) 96.14 $\pm$ 0.02 5.96	(0.01, 0.01, 0.0625) 97.02 $\pm$ 1.81 6.26	(0.1, 0.1, 0.01, 1, 0.125) 97.36 $\pm$ 1.68 5.40
bupa	(100, 0.0625) 56.23 $\pm$ 2.59 0.33	(0.01, 0.125) 57.68 $\pm$ 4.15 1.31	(10, 0.125) 58.26 $\pm$ 4.51 3.83	(1, 0.5, 0.0625) 59.13 $\pm$ 4.98 4.47	(1, 1, 0.5) 58.26 $\pm$ 0.05 2.48	(1, 100, 0.125) 60.29 $\pm$ 7.74 1.29	(0.1, 0.1, 10, 10, 0.125) 63.48 $\pm$ 6.03 1.18
card	(1, 1) 83.04 $\pm$ 2.79 7.82	(1, 8) 76.09 $\pm$ 2.64 8.49	(10, 0.5) 82.46 $\pm$ 2.01 11.99	(1, 1, 1) 82.75 $\pm$ 5.92 18.37	(100, 0.1, 2) 80.87 $\pm$ 0.06 10.83	(1, 1, 0.25) 83.77 $\pm$ 1.96 8.28	(0, 0.001, 1, 0.5) 86.23 $\pm$ 2.94 7.59
churn	(1, 0.125) 92.92 $\pm$ 1.17 148.30	(0.01, 0.125) 79.49 $\pm$ 3.25 178.39	(1, 0.125) 92.86 $\pm$ 1.48 371.58	(0.01, 0.05, 0.0625) 85.46 $\pm$ 2.58 390.26	(100, 10, 1) 93.08 $\pm$ 0.04 86.87	(1, 0.1, 0.03125) 93.67 $\pm$ 1.78 127.01	(1, 2, 0.1, 0.01, 1, 0.0625) 93.75 $\pm$ 1.57 70.32
crx	(0.1, 1) 84.68 $\pm$ 4.78 3.61	(0.1, 4) 75.81 $\pm$ 5.14 15.98	(1, 0.125) 82.39 $\pm$ 3.08 16.10	(1, 5, 1) 84.83 $\pm$ 6.96 17.39	(0.01, 1, 8) 82.85 $\pm$ 0.03 11.55	(1, 1, 0.25) 83.46 $\pm$ 4.46 13.67	(1, 1, 0.3, 0.01, 1, 0.25) 86.22 $\pm$ 1.78 7.39
divorce	(10, 2) 97.65 $\pm$ 2.46 0.09	(0.01, 0.5) 76.47 $\pm$ 7.50 0.30	(0.01, 0.25) 98.24 $\pm$ 2.63 1.99	(10, 3, 0.125) 97.06 $\pm$ 7.32 2.04	(10, 1, 0.5) 98.24 $\pm$ 0.03 0.28	(10, 0.1, 1) 97.65 $\pm$ 3.22 0.83	(1, 2, 0.3, 1, 1, 0.125) 98.82 $\pm$ 1.61 0.48
ecoli1	(1, 0.125) 89.89 $\pm$ 3.49 0.30	(0.1, 1) 84.83 $\pm$ 4.71 1.59	(10, 1) 87.80 $\pm$ 3.04 2.46	(0.1, 3, 0.0625) 84.21 $\pm$ 4.49 4.71	(10, 10, 0.5) 88.10 $\pm$ 0.02 1.77	(0.1, 0.1, 0.03125) 88.70 $\pm$ 4.10 1.12	(1, 3, 0.2, 0.1, 1, 0.125) 91.06 $\pm$ 2.04 1.21
german	(1, 0.5) 72.20 $\pm$ 2.89 4.59	(0.01, 4) 68.60 $\pm$ 3.13 11.11	(1, 0.5) 71.00 $\pm$ 2.40 24.06	(0.01, 0.5, 0.0625) 73.10 $\pm$ 3.64 35.84	(1, 1, 1) 75.80 $\pm$ 0.02 12.48	(0.1, 1, 0.25) 73.60 $\pm$ 3.10 14.14	(1, 1, 0.01, 1, 0.25) 72.00 $\pm$ 3.62 9.03
glass1	(0.01, 0.0625) 74.25 $\pm$ 7.62 2.94	(0.1, 4) 70.09 $\pm$ 8.60 1.59	(10, 0.125) 77.54 $\pm$ 4.45 1.81	(100, 0.5, 1) 65.43 $\pm$ 5.50 2.56	(10, 0.1, 0.0625) 75.22 $\pm$ 0.04 0.89	(0.1, 0.1, 0.03125) 79.42 $\pm$ 8.27 1.35	(1, 1, 1, 1, 0.0625) 79.86 $\pm$ 3.41 1.02
haberman	(100, 0.125) 70.60 $\pm$ 3.28 0.25	(10, 32) 71.24 $\pm$ 4.53 1.33	(0.1, 0.25) 71.58 $\pm$ 2.87 3.90	(1, 0.5, 0.125) 72.88 $\pm$ 3.23 4.13	(10, 1, 1) 72.23 $\pm$ 0.02 1.15	(0.01, 10, 0.03125) 72.57 $\pm$ 3.22 0.90	(1, 3, 0.6, 100, 10, 0.25) 73.85 $\pm$ 2.12 0.89
heart	(0.1, 0.125) 77.04 $\pm$ 4.83 1.55	(10, 8) 65.56 $\pm$ 14.40 1.13	(10, 0.25) 75.93 $\pm$ 5.71 2.04	(10, 1, 0.125) 77.78 $\pm$ 10.44 3.15	(10, 0.1, 0.25) 77.78 $\pm$ 0.09 1.00	(1, 10, 0.03125) 76.67 $\pm$ 5.77 1.66	(0, 0, 0.01, 1, 0.5) 82.96 $\pm$ 6.33 0.76
hepatitis	(1, 1) 80.00 $\pm$ 8.35 0.16	(10, 8) 80.00 $\pm$ 6.99 0.29	(1, 0.25) 80.00 $\pm$ 2.70 1.79	(0.01, 0.5, 0.0625) 79.35 $\pm$ 7.43 1.78	(100, 10, 1) 80.65 $\pm$ 0.06 0.21	(1, 10, 0.125) 81.29 $\pm$ 7.70 0.40	(1, 3, 0.4, 0.01, 1, 0.5) 82.58 $\pm$ 5.86 0.39
ionosphere	(0.1, 0.5) 80.91 $\pm$ 5.88 0.35	(0.01, 32) 64.10 $\pm$ 4.42 1.43	(0.1, 1) 89.46 $\pm$ 4.56 3.95	(100, 5, 0.0625) 78.11 $\pm$ 2.29 4.78	(100, 10, 16) 89.75 $\pm$ 0.04 1.05	(0.01, 10, 0.25) 90.31 $\pm$ 10.16 1.09	(0.9, 0.2, 100, 1, 0.125) 90.60 $\pm$ 3.83 1.14
mammographic	(0.1, 1) 78.07 $\pm$ 5.47 6.85	(1, 4) 60.60 $\pm$ 6.85 13.70	(0.01, 1) 78.80 $\pm$ 5.48 14.90	(0.01, 0.5, 1) 75.90 $\pm$ 4.69 25.31	(1, 10, 16) 79.28 $\pm$ 0.08 8.34	(1, 1, 0.25) 78.55 $\pm$ 8.54 9.26	(1, 2, 0.3, 10, 1, 0.0625) 80.60 $\pm$ 4.28 8.01
monk-2	(1, 0.625) 80.77 $\pm$ 8.28 0.58	(10, 8) 57.86 $\pm$ 4.16 2.48	(0.01, 0.25) 84.25 $\pm$ 7.98 5.20	(0.01, 0.5, 0.5) 74.08 $\pm$ 11.61 6.87	(1, 10, 0.25) 89.13 $\pm$ 0.08 3.65	(10, 10, 0.125) 89.14 $\pm$ 6.00 3.91	(0.9, 0.1, 0.01, 10, 0.25) 85.41 $\pm$ 0.95 1.72
musk	(0.01, 0.125) 95.91 $\pm$ 3.21 1311.64	(10, 0.125) 82.19 $\pm$ 1.25 1289.84	(1, 0.125) 96.12 $\pm$ 2.57 664.90	(0.01, 0.05, 0.0625) 84.59 $\pm$ 3.68 933.76	(10, 1, 0.125) 96.10 $\pm$ 0.08 413.53	(1, 0.1, 0.03125) 96.06 $\pm$ 4.12 583.91	(1, 2, 0.3, 0.01, 1, 0.25) 96.50 $\pm$ 2.45 239.80
newthyroid	(0.1, 0.125) 90.70 $\pm$ 2.85 0.13	(0.01, 0.125) 95.35 $\pm$ 1.04 0.52	(0.01, 0.125) 91.16 $\pm$ 1.95 1.29	(10, 0.5, 0.0625) 73.49 $\pm$ 3.29 2.22	(10, 0.01, 0.25) 93.49 $\pm$ 0.04 0.60	(1, 10, 0.03125) 95.81 $\pm$ 3.53 1.12	(1, 8, 0.8, 1, 1, 0.125) 96.74 $\pm$ 1.14 0.65
ozone	(1, 0.0625) 94.68 $\pm$ 1.00 217.06	(10, 4) 96.14 $\pm$ 10.59 150.15	(1, 0.125) 95.90 $\pm$ 0.81 193.43	(1, 0.05, 0.5) 96.93 $\pm$ 0.69 269.85	(1, 0.1, 0.5) 96.92 $\pm$ 0.01 38.18	(1, 1, 0.03125) 95.58 $\pm$ 0.72 40.55	(1, 2, 1, 0.01, 1, 0.5) 97.12 $\pm$ 0.69 39.92

Table 5
Continue.

Dataset	SVM-2K (Farquhar et al., 2005) (C_1, σ) Accuracy (%) Time (s)	MvTSVM (Xie & Sun, 2015) (C_1, σ) Accuracy (%) Time (s)	PSVM-2V (Tang et al., 2018b) (C_1, σ) Accuracy (%) Time (s)	MvNPSVM (Tang et al., 2018a) (C_1, D, σ) Accuracy (%) Time (s)	MvLDM (Hu et al., 2025) (C_1, v, σ) Accuracy (%) Time (s)	MvTPMSVM (Quadir & Tanveer, 2024) (C_1, C_2, σ) Accuracy (%) Time (s)	RoBoSS-MvSVM (Proposed) ($a, \lambda, C_1, D, \sigma$) Accuracy (%) Time (s)
parkinson	(0.1, 0.25) 87.18 $\pm$ 11.18 2.43	(0.01, 0.125) 78.46 $\pm$ 3.63 2.38	(10, 0.5) 93.85 $\pm$ 3.89 1.93	(10, 0.5, 1) 80.51 $\pm$ 9.17 2.26	(100, 1, 0.25) 94.87 $\pm$ 0.02 0.46	(1, 0.01, 0.03125) 94.87 $\pm$ 7.12 0.84	(0, 0, 0.01, 1, 0.0625) 96.41 $\pm$ 4.29 0.50
phoneme	(0.01, 0.125) 80.00 $\pm$ 4.58 264.24	(10, 4) 70.50 $\pm$ 5.18 620.96	(10, 0.25) 85.29 $\pm$ 4.51 325.84	(0.01, 0.05, 0.0625) 67.89 $\pm$ 5.33 516.31	(10, 1, 0.25) 81.14 $\pm$ 0.01 205.86	(0.1, 1, 0.03125) 84.27 $\pm$ 5.61 261.18	(0.1, 1, 0.1, 1, 0.125) 75.54 $\pm$ 3.16 164.40
pima	(1, 0.25) 72.79 $\pm$ 5.41 2.51	(0.01, 0.5) 62.23 $\pm$ 15.13 6.68	(0.1, 1) 73.83 $\pm$ 4.78 18.38	(0.01, 0.5, 0.125) 72.27 $\pm$ 4.04 21.96	(0.1, 1, 1) 70.05 $\pm$ 0.04 15.42	(10, 10, 0.03125) 70.98 $\pm$ 2.50 13.97	(0.7, 0.8, 100, 1, 0.125) 75.02 $\pm$ 3.82 7.56
planning	(1, 0.25) 70.87 $\pm$ 3.22 0.14	(1, 0.125) 70.90 $\pm$ 2.84 0.52	(0.1, 0.25) 68.65 $\pm$ 4.46 2.30	(100, 5, 1) 70.89 $\pm$ 5.92 4.49	(1, 0.1, 0.25) 71.41 $\pm$ 0.04 1.44	(0.01, 1, 0.125) 71.40 $\pm$ 5.92 1.10	(1, 0.3, 1, 0.1, 0.25) 71.95 $\pm$ 4.51 0.67
saheart	(100, 0.125) 64.69 $\pm$ 6.38 2.70	(1, 4) 64.70 $\pm$ 12.20 3.04	(10, 0.5) 68.38 $\pm$ 4.84 7.41	(0.01, 3, 0.0625) 67.54 $\pm$ 6.94 8.26	(1, 0.1, 4) 67.96 $\pm$ 0.04 6.27	(0.01, 0.01, 0.125) 68.81 $\pm$ 6.00 6.97	(0, 0, 0.01, 0.1, 0.125) 70.80 $\pm$ 5.28 5.63
sonar	(0.1, 0.5) 84.58 $\pm$ 5.44 0.69	(0.01, 8) 67.34 $\pm$ 7.72 0.46	(10, 0.5) 86.05 $\pm$ 2.65 1.66	(0.01, 0.5, 0.0625) 53.86 $\pm$ 6.07 2.20	(100, 10, 0.5) 86.05 $\pm$ 0.05 0.81	(1, 10, 0.125) 86.50 $\pm$ 4.36 1.13	(1.7, 0.4, 100, 1, 0.125) 88.48 $\pm$ 4.70 0.77
spectfheart	(0.01, 2) 78.41 $\pm$ 9.83 0.21	(1, 1) 77.18 $\pm$ 9.31 0.81	(1, 1) 76.80 $\pm$ 8.31 2.21	(10, 0.5, 0.125) 71.26 $\pm$ 23.98 3.42	(10, 10, 1) 79.04 $\pm$ 0.10 1.93	(1, 10, 0.125) 74.19 $\pm$ 9.83 2.15	(1, 1.4, 100, 10, 0.0625) 79.41 $\pm$ 9.83 1.26
tic-tac-toe	(0.01, 0.125) 100.00 $\pm$ 0.00 13.46	(0.01, 0.5) 81.00 $\pm$ 2.35 9.87	(10, 0.125) 100.00 $\pm$ 0.00 29.12	(10, 5, 0.125) 70.67 $\pm$ 3.26 32.99	(10, 10, 0.0625) 100.00 $\pm$ 0.00 9.94	(1, 0.1, 0.03125) 100.00 $\pm$ 0.00 10.81	(1, 1, 1, 1, 0.125) 100.00 $\pm$ 0.00 8.27
titanic	(0.01, 0.0625) 78.28 $\pm$ 0.62 32.54	(10, 0.125) 65.70 $\pm$ 15.35 44.32	(10, 0.25) 78.28 $\pm$ 0.62 96.15	(1, 0.05, 0.125) 77.10 $\pm$ 2.37 62.86	(1, 0.1, 0.5) 78.28 $\pm$ 0.01 40.64	(1, 0.1, 0.03125) 79.05 $\pm$ 1.43 49.23	(1.2, 0.3, 0.01, 1, 1) 67.70 $\pm$ 1.08 25.97
transfusion	(0.01, 0.0625) 78.20 $\pm$ 2.71 2.15	(0.1, 1) 72.57 $\pm$ 16.93 4.36	(1, 0.125) 79.41 $\pm$ 1.97 13.31	(1, 0.5, 0.0625) 77.27 $\pm$ 3.11 21.26	(1, 1, 1) 76.20 $\pm$ 0.03 13.38	(1, 1, 0.03125) 76.33 $\pm$ 2.94 10.44	(0, 0, 0.01, 0.1, 0.125) 77.00 $\pm$ 2.92 8.43
vehicle0	(10, 0.5) 78.72 $\pm$ 3.26 6.06	(0.01, 0.125) 73.64 $\pm$ 2.31 8.88	(10, 2) 87.47 $\pm$ 3.59 23.26	(1, 1, 1) 81.80 $\pm$ 6.11 27.13	(1, 1, 2) 90.07 $\pm$ 0.03 9.22	(0.1, 10, 0.03125) 90.78 $\pm$ 2.85 11.42	(1.7, 0.2, 100, 10, 0.125) 90.31 $\pm$ 2.55 7.61
vote	(0.1, 2) 89.43 $\pm$ 4.18 0.58	(1, 0.125) 70.34 $\pm$ 6.77 1.83	(0.01, 2) 89.89 $\pm$ 2.74 8.44	(0.1, 0.5, 0.0625) 90.11 $\pm$ 3.11 7.59	(100, 1, 0.03125) 90.80 $\pm$ 0.03 2.24	(10, 0.1, 0.0625) 90.57 $\pm$ 4.19 2.55	(2, 1, 1, 1, 0.5) 92.41 $\pm$ 3.51 1.59
vowel0	(1, 0.25) 96.36 $\pm$ 1.76 4.69	(0.1, 4) 91.10 $\pm$ 5.78 15.64	(10, 0.25) 100.00 $\pm$ 0.00 33.87	(0.01, 0.5, 0.0625) 90.90 $\pm$ 2.91 38.62	(10, 1, 0.25) 100.00 $\pm$ 0.00 22.85	(0.1, 1, 0.03125) 100.00 $\pm$ 0.00 24.02	(0.1, 0.1, 0.01, 1, 0.125) 100.00 $\pm$ 0.00 8.69
wine	(0.01, 0.25) 96.62 $\pm$ 1.30 0.44	(0.01, 2) 90.94 $\pm$ 2.57 0.84	(1, 2) 98.30 $\pm$ 2.55 1.71	(0.01, 0.5, 0.0625) 77.44 $\pm$ 9.39 1.94	(10, 10, 8) 97.16 $\pm$ 0.03 0.97	(100, 1, 0.0625) 98.86 $\pm$ 6.33 1.21	(0.1, 0.1, 0.01, 1, 0.25) 98.87 $\pm$ 1.54 0.62
WPBC	(0.1, 0.5) 76.78 $\pm$ 4.05 0.11	(0.01, 4) 75.24 $\pm$ 5.87 0.41	(10, 1) 74.77 $\pm$ 7.41 2.17	(100, 0.5, 0.125) 78.33 $\pm$ 4.98 2.25	(1, 1, 1) 75.77 $\pm$ 0.04 0.85	(10, 10, 0.0625) 77.77 $\pm$ 3.76 1.00	(0, 0.001, 1, 0.25) 76.74 $\pm$ 4.64 0.68
yeast	(1, 0.125) 90.77 $\pm$ 0.77 13.52	(10, 0.125) 72.72 $\pm$ 9.22 29.36	(0.01, 0.125) 94.00 $\pm$ 1.00 43.26	(1, 0.05, 0.5) 89.02 $\pm$ 1.08 67.67	(1, 0.1, 0.5) 94.95 $\pm$ 0.02 18.69	(0.1, 1, 0.03125) 94.47 $\pm$ 1.02 27.65	(1, 0.1, 0.01, 1, 0.0625) 91.31 $\pm$ 0.81 10.60
Average Accuracy	83.27	75.27	84.46	78.81	85.00	85.29	85.92
Average Rank	4.72	6.00	4.06	5.13	3.29	2.99	1.81

view, thereby reducing the risk of overfitting and enhancing the generalization performance of the model.

- The slack variable for the ϑ th view, denoted by $\xi_i^{[\vartheta]}$, quantifies the degree of margin violation. The constraints ensure that correctly classified samples lie on or beyond the unit margin from the decision hyperplane in their respective views.

In the proposed RoBoSS-MvSVM, the RoBoSS loss function addresses the key challenges in the MvL framework. Its bounded and robust design mitigates the influence of noisy samples, while its smooth formulation ensures a gradual and stable optimization process. To obtain the solution, the problem is first reformulated into a two-view framework to enable efficient implementation of the algorithm. As the number of views increases, RoBoSS-MvSVM continues to exploit the properties of the RoBoSS loss for robust classification, adaptively penalizing misclassifications across views while limiting the impact of noise and inter-view

inconsistencies. Furthermore, the smooth nature of the RoBoSS loss allows the model to be efficiently solved using gradient-based optimization techniques.

We employ the NAG method (Nesterov, 1983) to solve the optimization problem of RoBoSS-MvSVM. The NAG is recognized for its low computational complexity and high efficiency when dealing with large-scale problems (Ruder, 2016). NAG is an extension of the classical gradient-based optimization methods, which incorporates a momentum term to expedite the convergence. In contrast to standard stochastic or batch gradient descent methods, the momentum technique (Qian, 1999) updates the parameters by adding a fraction of the prior update to the current one, thereby mitigating the oscillations and speeding up the convergence in the relevant direction. The NAG method employs a look-ahead mechanism, in which it first predicts the future position of the parameters using the momentum and then evaluates the gradient at this predicted position.

Table 6

Sensitivity and specificity of the proposed RoBoSS-MvSVM and baseline models on the UCI and KEEL datasets.

Dataset	SVM-2K (Farquhar et al., 2005) Sensitivity Specificity	MvTSVM (Xie & Sun, 2015) Sensitivity Specificity	PSVM-2V (Tang et al., 2018b) Sensitivity Specificity	MvNPSVM (Tang et al., 2018a) Sensitivity Specificity	MvLDM (Hu et al., 2025) Sensitivity Specificity	MvTPMSVM (Quadir & Tanveer, 2024) Sensitivity Specificity	RoBoSS-MvSVM (Proposed) Sensitivity Specificity
appendicitis	6.67 ± 0.15 98.89 ± 0.02	28.67 ± 25.02 100.00 ± 0.00	62.33 ± 0.17 88.36 ± 0.06	6.67 ± 0.15 98.89 ± 0.02	57.33 ± 0.19 90.43 ± 0.06	48.33 ± 19.05 92.93 ± 3.24	71.33 ± 0.08 90.57 ± 0.07
auction	32.74 ± 0.15 92.20 ± 0.03	1.08 ± 1.25 98.60 ± 10.85	29.70 ± 0.35 96.01 ± 0.04	78.47 ± 0.21 27.38 ± 0.18	23.84 ± 1.25 94.21 ± 0.03	30.10 ± 3.58 93.60 ± 3.95	33.15 ± 0.04 96.19 ± 0.03
aus	79.43 ± 0.04 83.08 ± 0.04	38.39 ± 19.65 78.52 ± 14.01	80.84 ± 0.05 82.80 ± 0.05	92.34 ± 0.04 79.73 ± 0.04	90.22 ± 0.05 81.68 ± 0.04	77.54 ± 5.37 88.87 ± 4.21	70.07 ± 0.03 94.24 ± 0.04
bank	14.52 ± 1.19 96.00 ± 0.02	0.41 ± 0.25 97.20 ± 6.40	22.16 ± 0.04 92.15 ± 0.03	30.18 ± 0.54 88.55 ± 0.09	24.04 ± 0.07 94.80 ± 0.03	17.42 ± 1.11 93.50 ± 2.60	18.51 ± 0.10 92.54 ± 0.03
bankruptcy	26.14 ± 0.21 96.71 ± 0.02	72.70 ± 2.41 97.52 ± 5.95	12.41 ± 0.06 98.67 ± 0.02	48.60 ± 0.48 96.25 ± 0.04	13.02 ± 0.03 98.73 ± 0.02	17.24 ± 4.21 98.30 ± 1.85	12.31 ± 2.46 98.91 ± 0.02
breast cancer	94.74 ± 0.04 98.36 ± 0.01	93.89 ± 4.97 60.46 ± 39.02	81.42 ± 0.04 99.77 ± 0.01	80.60 ± 0.21 86.17 ± 0.35	90.86 ± 0.05 99.75 ± 0.01	94.35 ± 0.91 98.97 ± 0.25	93.46 ± 0.04 99.73 ± 0.01
bupa	56.65 ± 0.04 56.09 ± 0.04	48.47 ± 16.08 63.74 ± 12.34	57.12 ± 0.09 58.33 ± 0.08	47.31 ± 0.11 69.85 ± 0.08	6.16 ± 0.10 96.81 ± 0.06	52.65 ± 8.32 0.68 ± 14.34	58.69 ± 0.11 68.22 ± 0.13
card	68.69 ± 0.05 94.42 ± 0.02	59.77 ± 3.19 88.28 ± 6.14	80.83 ± 0.03 83.63 ± 0.04	80.57 ± 0.18 85.74 ± 0.07	79.98 ± 0.04 81.32 ± 0.09	71.54 ± 3.65 93.33 ± 2.44	87.54 ± 0.06 85.31 ± 0.03
churn	59.97 ± 0.06 99.05 ± 0.01	7.63 ± 18.40 92.86 ± 7.80	78.48 ± 0.08 97.93 ± 0.02	11.62 ± 0.17 99.18 ± 0.01	66.79 ± 0.07 97.96 ± 0.02	63.21 ± 6.95 99.59 ± 0.34	66.82 ± 0.05 98.79 ± 0.01
crx	77.01 ± 0.08 91.74 ± 0.04	74.15 ± 7.87 78.08 ± 4.95	78.76 ± 0.06 84.91 ± 0.04	86.95 ± 0.16 84.47 ± 0.06	69.51 ± 0.05 93.93 ± 0.03	73.98 ± 6.94 91.29 ± 2.28	89.35 ± 0.03 83.37 ± 0.03
divorce	94.98 ± 0.05 100.00 ± 0.00	59.71 ± 16.91 93.07 ± 7.19	96.16 ± 0.06 100.00 ± 0.00	98.67 ± 0.24 95.29 ± 0.15	96.16 ± 0.06 100.00 ± 0.00	94.98 ± 6.54 100.00 ± 0.00	97.49 ± 0.03 100.00 ± 0.00
ecoli1	75.13 ± 0.11 94.28 ± 0.03	63.56 ± 10.32 91.14 ± 5.17	59.09 ± 0.11 96.57 ± 0.02	37.35 ± 0.14 98.43 ± 0.25	68.89 ± 0.08 93.88 ± 0.04	72.77 ± 15.86 93.48 ± 1.66	78.17 ± 0.12 94.96 ± 0.04
german	99.17 ± 0.01 9.50 ± 0.03	96.52 ± 11.55 3.22 ± 15.56	84.49 ± 0.04 39.97 ± 0.08	93.68 ± 0.01 25.31 ± 0.05	93.02 ± 0.01 35.75 ± 0.05	97.58 ± 0.25 17.73 ± 1.58	85.23 ± 0.04 41.91 ± 0.10
glass1	74.95 ± 0.04 74.32 ± 0.12	27.09 ± 22.37 93.35 ± 4.99	67.07 ± 0.06 83.52 ± 0.07	4.27 ± 0.04 99.29 ± 0.02	65.33 ± 0.05 80.45 ± 0.05	62.15 ± 6.40 89.17 ± 7.14	73.83 ± 0.13 89.56 ± 0.08
haberman	92.56 ± 0.05 9.90 ± 0.03	93.57 ± 7.80 7.74 ± 9.94	95.61 ± 0.04 4.87 ± 0.05	98.27 ± 0.02 2.76 ± 0.03	95.16 ± 0.04 8.27 ± 0.06	97.77 ± 2.14 1.90 ± 3.25	100.00 ± 0.00 1.05 ± 0.44
heart	76.22 ± 0.07 76.85 ± 0.13	65.35 ± 22.19 63.32 ± 30.96	75.01 ± 0.05 76.20 ± 0.16	92.21 ± 0.06 57.10 ± 0.35	78.98 ± 0.09 75.28 ± 0.16	76.22 ± 7.23 76.11 ± 19.90	76.41 ± 0.08 73.80 ± 0.11
hepatitis	100.00 ± 0.00 5.00 ± 0.11	100.00 ± 0.00 5.00 ± 15.42	87.98 ± 0.04 49.43 ± 0.06	100.00 ± 0.00 0.00 ± 1.25	88.82 ± 0.09 51.10 ± 0.16	91.10 ± 6.43 44.57 ± 19.57	91.26 ± 0.07 49.43 ± 0.06
ionosphere	48.71 ± 0.19 98.65 ± 0.01	49.53 ± 0.31 66.10 ± 0.25	74.90 ± 0.13 97.77 ± 0.02	81.69 ± 0.45 75.64 ± 3.54	75.20 ± 0.12 98.23 ± 0.02	78.09 ± 1.66 97.33 ± 14.23	76.41 ± 0.08 98.66 ± 0.03
mammographic	82.22 ± 0.05 74.65 ± 0.06	28.77 ± 21.10 90.47 ± 9.40	86.70 ± 0.06 71.51 ± 0.07	93.33 ± 0.14 59.67 ± 0.12	85.18 ± 0.05 74.17 ± 0.06	74.77 ± 7.40 82.51 ± 4.90	86.78 ± 0.05 75.01 ± 0.06
monk-2	93.05 ± 0.05 69.36 ± 0.14	64.61 ± 5.38 52.32 ± 3.79	92.97 ± 0.06 76.03 ± 0.12	75.16 ± 0.17 78.99 ± 0.03	92.64 ± 0.06 86.80 ± 0.13	81.32 ± 9.76 98.68 ± 1.86	92.57 ± 0.02 79.11 ± 0.02
musk	89.34 ± 0.04 97.12 ± 0.02	38.40 ± 12.60 90.18 ± 6.75	88.05 ± 0.05 97.60 ± 0.02	85.90 ± 0.14 75.70 ± 0.13	88.05 ± 0.05 97.58 ± 0.02	87.77 ± 5.90 97.58 ± 2.10	87.49 ± 0.05 98.16 ± 0.02
newthyroid	100.00 ± 0.00 69.38 ± 0.07	98.67 ± 2.10 86.90 ± 6.05	100.00 ± 0.00 70.71 ± 0.06	96.73 ± 0.31 20.19 ± 0.12	99.31 ± 0.02 80.00 ± 0.14	97.26 ± 3.21 91.67 ± 9.89	100.00 ± 0.00 88.33 ± 0.06
ozone	22.61 ± 0.13 96.84 ± 0.01	24.64 ± 9.03 66.85 ± 11.14	9.48 ± 0.07 98.46 ± 0.01	59.50 ± 0.54 97.90 ± 0.24	4.33 ± 0.06 99.68 ± 0.00	16.79 ± 5.96 97.93 ± 0.53	20.80 ± 0.85 97.65 ± 0.25
parkinson	89.01 ± 0.12 83.50 ± 0.30	100.00 ± 0.00 11.77 ± 4.47	97.40 ± 0.03 83.14 ± 0.10	82.79 ± 0.20 67.93 ± 0.34	97.40 ± 0.03 87.49 ± 0.03	93.96 ± 8.89 97.50 ± 1.55	96.63 ± 0.04 95.50 ± 0.06
phoneme	62.50 ± 0.07 87.29 ± 0.03	0.62 ± 24.80 99.53 ± 1.20	71.94 ± 0.09 90.80 ± 0.04	89.28 ± 0.21 59.03 ± 0.20	77.66 ± 0.08 82.37 ± 0.04	54.18 ± 8.20 96.84 ± 1.75	48.14 ± 0.07 29.27 ± 0.15
pima	33.52 ± 0.07 93.94 ± 0.03	9.40 ± 8.07 90.45 ± 22.75	35.95 ± 0.09 94.39 ± 0.01	62.51 ± 0.05 77.69 ± 0.06	18.59 ± 0.07 97.81 ± 0.01	53.11 ± 3.99 80.60 ± 1.08	63.66 ± 0.05 81.27 ± 0.03
planning	97.83 ± 0.02 3.21 ± 0.04	97.72 ± 3.42 3.82 ± 5.24	93.87 ± 0.03 5.71 ± 0.06	99.26 ± 0.25 0.00 ± 0.21	98.54 ± 0.02 3.21 ± 0.04	99.29 ± 4.25 0.03 ± 1.14	99.29 ± 0.35 3.21 ± 0.04
saheart	40.80 ± 0.08 77.49 ± 0.07	26.22 ± 33.22 86.45 ± 14.18	38.13 ± 0.11 85.09 ± 0.05	60.50 ± 0.07 71.42 ± 0.08	33.43 ± 0.18 88.01 ± 0.09	36.44 ± 0.14 86.15 ± 2.45	42.01 ± 0.08 86.15 ± 0.04
sonar	79.96 ± 0.09 88.07 ± 0.10	41.97 ± 15.91 88.42 ± 7.38	77.84 ± 0.08 92.70 ± 0.05	1.11 ± 0.02 100.00 ± 0.00	81.02 ± 0.05 90.07 ± 0.10	83.98 ± 5.21 88.02 ± 10.82	85.39 ± 0.07 90.82 ± 0.07
spectfheart	100.00 ± 0.00 0.00 ± 0.21	92.18 ± 18.55 16.22 ± 36.09	94.76 ± 0.07 13.33 ± 0.18	81.03 ± 0.42 20.00 ± 0.45	98.69 ± 0.02 5.56 ± 0.08	89.09 ± 0.68 29.68 ± 2.58	77.77 ± 0.04 33.50 ± 0.36
tic-tac-toe	100.00 ± 0.00 100.00 ± 0.00	91.02 ± 1.83 62.27 ± 3.60	100.00 ± 0.00 100.00 ± 0.00	99.36 ± 0.47 16.80 ± 0.01	100.00 ± 0.00 100.00 ± 0.00	100.00 ± 0.00 100.00 ± 0.00	100.00 ± 0.00 100.00 ± 0.00
titanic	39.75 ± 0.07 96.74 ± 0.04	64.54 ± 15.48 66.19 ± 30.78	39.75 ± 0.07 96.74 ± 0.04	46.62 ± 0.07 91.69 ± 0.02	48.82 ± 0.03 92.36 ± 0.02	38.00 ± 1.76 98.66 ± 0.28	20.00 ± 0.21 80.00 ± 0.01
transfusion	20.41 ± 0.03 96.33 ± 0.01	23.06 ± 37.73 87.58 ± 34.63	32.77 ± 0.03 94.08 ± 0.02	11.72 ± 0.03 97.72 ± 0.01	0.53 ± 0.01 99.83 ± 0.00	4.50 ± 3.24 0.99 ± 0.48	4.36 ± 0.03 99.65 ± 0.01
vehicle0	11.53 ± 0.08 99.38 ± 0.01	0.67 ± 0.00 0.96 ± 0.86	56.86 ± 0.15 97.24 ± 0.01	32.72 ± 0.34 97.06 ± 0.04	68.57 ± 0.13 97.09 ± 0.02	93.18 ± 5.41 90.22 ± 2.71	82.86 ± 0.06 92.93 ± 0.03
vote	95.19 ± 0.03 80.63 ± 0.09	93.91 ± 7.76 32.26 ± 7.99	90.68 ± 0.03 88.59 ± 0.03	88.50 ± 0.05 92.72 ± 0.03	92.20 ± 0.03 88.73 ± 0.04	90.69 ± 6.36 90.55 ± 3.87	92.60 ± 0.04 92.05 ± 0.05
vowel0	57.60 ± 0.20 100.00 ± 0.00	94.83 ± 3.73 98.21 ± 3.18	100.00 ± 0.00 100.00 ± 0.00	36.70 ± 0.36 96.20 ± 0.37	100.00 ± 0.00 100.00 ± 0.00	1.00 ± 2.24 1.00 ± 0.03	100.00 ± 0.00 100.00 ± 0.00

Table 6
Continue.

Dataset	SVM-2K (Farquhar et al., 2005) Sensitivity Specificity	MvTSVM (Xie & Sun, 2015) Sensitivity Specificity	PSVM-2V (Tang et al., 2018b) Sensitivity Specificity	MvNPSVM (Tang et al., 2018a) Sensitivity Specificity	MvLDM (Hu et al., 2025) Sensitivity Specificity	MvTPMSVM (Quadri & Tanveer, 2024) Sensitivity Specificity	RoBoSS-MvSVM (Proposed) Sensitivity Specificity
wine	100.00 ± 0.00 94.91 ± 0.02	73.16 ± 6.14 100.00 ± 0.00	96.33 ± 0.05 99.13 ± 0.02	34.24 ± 0.20 100.00 ± 0.00	94.79 ± 0.05 98.26 ± 0.04	98.33 ± 13.28 99.13 ± 0.00	100.00 ± 0.00 98.36 ± 0.02
WPBC	4.04 ± 0.06 99.33 ± 0.01	87.62 ± 4.47 77.37 ± 5.99	49.67 ± 0.23 82.77 ± 0.03	14.00 ± 0.31 97.93 ± 0.05	5.08 ± 0.07 98.08 ± 0.03	28.43 ± 17.67 92.70 ± 2.77	28.43 ± 0.18 91.32 ± 0.04
yeast	18.19 ± 0.05 99.70 ± 0.01	13.05 ± 23.51 80.12 ± 11.08	51.95 ± 0.06 92.25 ± 0.00	33.70 ± 0.48 95.80 ± 0.18	69.80 ± 0.20 98.11 ± 0.01	67.60 ± 1.44 97.81 ± 0.21	72.63 ± 0.04 93.65 ± 0.01
Average	(64.51, 79.05)	(55.01, 68.37)	(69.99, 81.06)	(62.93, 71.40)	(67.64, 83.02)	(66.84, 76.66)	(71.66, 81.11)

3.3. Optimization for RoBoSS-MvSVM

The optimization problem for the proposed RoBoSS-MvSVM within the two-view learning framework is formulated as follows:

$$\begin{aligned}
\min D \sum_{i=1}^m & \left((u_1 \psi_1(x_i^{[1]})) - (u_2 \psi_2(x_i^{[2]})) \right)^2 \\
& + \sum_{\theta=1}^2 \sum_{i=1}^m \frac{C_\theta}{m} \left(\lambda_\theta \left(1 - (a_\theta \{\xi_i^{[\theta]}\}_+ + 1) \exp(-a_\theta \{\xi_i^{[\theta]}\}_+) \right) \right) \\
& + \frac{1}{2} \|u_1\|^2 + \frac{\eta}{2} \|u_2\|^2 \\
\text{s.t. } & y_i \left(u_1 \psi_1(x_i^{[1]}) \right) = 1 - \xi_i^{[1]}, \\
& y_i \left(u_2 \psi_2(x_i^{[2]}) \right) = 1 - \xi_i^{[2]}, \quad i = 1, 2, \dots, m.
\end{aligned} \quad (9)$$

The parameter η serves as an adaptive combination weight that regulates the relative contribution of each view during both optimization and prediction. By adjusting the regularization intensity and the influence of the corresponding decision function, η enables the model to emphasize more discriminative views while suppressing less informative ones. We apply the representer theorem (Dinuzzo & Schölkopf, 2012) to obtain the solutions for u_1 and u_2 in the problem (9), which can be expressed as follows:

$$u_\theta = \sum_{j=1}^m \alpha_j^{[\theta]} \psi_\theta(x_j^{[\theta]}), \quad (10)$$

where $\alpha^{[\theta]} = [\alpha_1^{[\theta]}, \alpha_2^{[\theta]}, \dots, \alpha_m^{[\theta]}]^T$ for $\theta = 1, 2$ are the coefficient vectors corresponding to views 1 and 2. Substituting the Eq. (10) into (9), we obtain

$$\begin{aligned}
\min_{\alpha^{[1]}, \alpha^{[2]}} & f(\alpha^{[1]}, \alpha^{[2]}) = \frac{1}{2} \alpha^{[1]T} \mathcal{K}_1 \alpha^{[1]} \\
& + \frac{\eta}{2} \alpha^{[2]T} \mathcal{K}_2 \alpha^{[2]} \\
& + D(\mathcal{K}_1 \alpha^{[1]} - \mathcal{K}_2 \alpha^{[2]})^T (\mathcal{K}_1 \alpha^{[1]} - \mathcal{K}_2 \alpha^{[2]}) \\
& + \frac{C_1}{m} \sum_{i=1}^m \lambda_1 \left(1 - (a_1 \{\xi_i^{[1]}\}_+ + 1) \exp(-a_1 \{\xi_i^{[1]}\}_+) \right) \\
& + \frac{C_2}{m} \sum_{i=1}^m \lambda_2 \left(1 - (a_2 \{\xi_i^{[2]}\}_+ + 1) \exp(-a_2 \{\xi_i^{[2]}\}_+) \right) \\
\text{s.t. } & Y \mathcal{K}_1 \alpha^{[1]} = 1 - \xi^{[1]}, \\
& Y \mathcal{K}_2 \alpha^{[2]} = 1 - \xi^{[2]},
\end{aligned} \quad (11)$$

where $\mathcal{K}_1 = K_1(x_i^{[1]}, x_j^{[1]}) = \psi_1(x_i^{[1]}) \cdot \psi_1(x_j^{[1]})$ and $\mathcal{K}_2 = K_2(x_i^{[2]}, x_j^{[2]}) = \psi_2(x_i^{[2]}) \cdot \psi_2(x_j^{[2]})$ represent the kernel functions for the first and second views, respectively. The slack variables are given by $\xi^{[\theta]} = [\xi_1^{[\theta]}, \xi_2^{[\theta]}, \dots, \xi_m^{[\theta]}]^T$ for $\theta = 1, 2$. The label matrix is given as $Y = \text{diag}(y_1, y_2, \dots, y_m)$ where Y is an $m \times m$ diagonal matrix.

We utilize the NAG algorithm to tackle the optimization problem (11). However, the choice of learning rate is a critical factor in the NAG algorithm. An excessively small learning rate results in slow convergence, while a high learning rate may lead to overshooting the optimal

solution. To address this challenge, we adopt an exponential decay approach, inspired by the simulated annealing method (Kirkpatrick et al., 1983) to update the learning rate as follows: $\beta_{\text{new}} = \beta_{\text{old}} \exp(-\theta t)$, where θ controls the decay rate and t denotes the current iteration number. Algorithm 1 presents the NAG algorithm used to compute $\alpha^{[1]}, \alpha^{[2]}$. After obtaining the optimal solutions $\alpha^{[1]*}, \alpha^{[2]*}$, the class label of a new sample $(x_i^{[1]}, x_i^{[2]})$ can be predicted as:

$$f(x^{[1]}, x^{[2]}) = \text{sign}(h(x^{[1]}, x^{[2]})), \quad (12)$$

where

$$h(x^{[1]}, x^{[2]}) = \sum_{i=1}^m \alpha_i^{[1]*} K_1(x_i^{[1]}, x^{[1]}) + \eta \sum_{i=1}^m \alpha_i^{[2]*} K_2(x_i^{[2]}, x^{[2]}). \quad (13)$$

4. Theoretical analysis

In this section, we analyze the computational complexity of the proposed RoBoSS-MvSVM and present the theoretical analysis of the RoBoSS loss function. Furthermore, to assess the generalization capability of the model, we employ Rademacher complexity (Bartlett & Mendelson, 2023) to derive the generalization error bound for RoBoSS-MvSVM.

4.1. Computational complexity

In this subsection, we analyzed the computational complexity of the NAG algorithm utilized to address the optimization problem of the proposed RoBoSS-MvSVM. Let m denote the total number of patterns, d denote the average number of features across both views, and N denotes the total number of iterations. The computational cost for updating the velocity variable v in each view is given by $\mathcal{O}(2(N+d)m^2)$. The complexity of updating the parameter β is $\mathcal{O}(Nm)$, while the update of α requires $\mathcal{O}(N)$ operations. Considering all these components, the overall computational complexity of the NAG algorithm for RoBoSS-MvSVM can be expressed as $\mathcal{O}(2(N+d)m^2)$. For comparison, PSVM-2V has a complexity of $\mathcal{O}((6m)^3)$, SVM-2K has $\mathcal{O}((4m)^3)$, and MvTSVM requires $2 \times \mathcal{O}((2m)^3)$. The analysis confirms that the proposed multi-view RoBoSS-MvSVM ensures comparative time complexity while achieving improved robustness and better MvL performance.

4.2. RoBoSS loss function theoretical analysis

This subsection investigates the theoretical properties of the RoBoSS loss function, highlighting its classification-calibrated property. Bartlett et al. (2006) introduced classification-calibrated as a principled framework for assessing the statistical efficacy of a loss function. A classification-calibrated loss function ensures that the model's predicted probabilities closely reflect the accurate class probabilities, thereby improving the reliability of predictions and enhancing the interpretability of classification outcomes.

Consider a training dataset $\mathcal{T} = \{(x_i, y_i)\}_{i=1}^m$ where each input instance x_i belongs to a space $X \subseteq \mathbb{R}^d$, and its corresponding label $y_i \in$

Table 7

F-measure and G-mean of the proposed RoBoSS-MvSVM and baseline models on the UCI and KEEL datasets.

Dataset	SVM-2K (Farquhar et al., 2005) F-measure G-mean	MvTSVM (Xie & Sun, 2015) F-measure G-mean	PSVM-2V (Tang et al., 2018b) F-measure G-mean	MvNPSVM (Tang et al., 2018a) F-measure G-mean	MvLDM (Hu et al., 2025) F-measure G-mean	MvTPMSVM (Quadir & Tanveer, 2024) F-measure G-mean	RoBoSS-MvSVM (Proposed) F-measure G-mean
appendicitis	8.00 ± 0.17 25.68 ± 0.25 35.35 ± 0.04	44.56 ± 18.90 53.54 ± 30.03 1.95 ± 6.80	59.96 ± 0.12 74.21 ± 0.10 37.87 ± 0.05	8.00 ± 0.18 25.68 ± 0.25 23.43 ± 0.16	58.80 ± 0.16 72.01 ± 0.14 29.94 ± 0.05	56.04 ± 22.36 67.02 ± 14.15 34.69 ± 3.90	69.85 ± 0.12 80.38 ± 0.06 41.52 ± 0.04
auction	54.94 ± 0.06 79.11 ± 0.05 81.24 ± 0.03	10.31 ± 14.20 46.57 ± 11.80 54.90 ± 25.85	53.40 ± 0.06 79.86 ± 0.05 81.81 ± 0.03	46.35 ± 0.14 84.91 ± 0.03 85.80 ± 0.03	47.39 ± 0.05 84.61 ± 0.05 85.84 ± 0.03	53.08 ± 4.80 80.89 ± 7.41 83.01 ± 3.76	56.46 ± 0.04 79.03 ± 0.04 81.26 ± 0.02
aus	19.89 ± 0.06 37.33 ± 0.08 23.39 ± 0.07	0.68 ± 5.90 6.32 ± 18.60 58.26 ± 7.40	24.20 ± 0.06 45.19 ± 0.07 16.30 ± 0.08	36.28 ± 0.12 51.69 ± 0.17 39.80 ± 0.14	29.32 ± 0.07 47.74 ± 0.07 17.28 ± 0.08	20.86 ± 2.85 40.36 ± 6.90 20.59 ± 3.25	21.05 ± 0.05 41.39 ± 0.06 16.97 ± 0.07
bank	50.28 ± 0.07 95.75 ± 0.03 96.53 ± 0.02	84.20 ± 12.40 75.71 ± 17.63 75.35 ± 38.40	34.99 ± 0.07 89.42 ± 0.02 90.13 ± 0.02	68.39 ± 0.12 78.33 ± 0.11 83.34 ± 0.25	35.85 ± 0.06 94.93 ± 0.03 95.20 ± 0.03	41.17 ± 5.10 95.97 ± 2.74 96.63 ± 0.49	34.89 ± 0.06 96.37 ± 0.02 96.54 ± 0.02
bankruptcy	52.13 ± 0.04 56.37 ± 0.02 78.29 ± 0.04	48.74 ± 12.58 55.58 ± 4.34 70.43 ± 8.95	53.22 ± 0.08 57.72 ± 0.05 80.41 ± 0.01	52.02 ± 0.24 57.48 ± 0.20 81.41 ± 0.09	10.67 ± 0.13 24.41 ± 0.19 79.03 ± 0.05	53.88 ± 9.36 5.97 ± 8.36 79.68 ± 1.87	58.30 ± 0.08 63.28 ± 0.08 85.05 ± 0.03
bupa	80.53 ± 0.03 72.73 ± 0.06 77.08 ± 0.06	72.64 ± 2.21 10.54 ± 19.20 26.62 ± 16.90	82.22 ± 0.02 82.78 ± 0.07 87.67 ± 0.05	83.12 ± 0.08 19.89 ± 0.15 33.95 ± 0.14	80.65 ± 0.06 75.31 ± 0.06 80.89 ± 0.05	81.71 ± 1.70 76.40 ± 6.60 79.34 ± 6.20	86.42 ± 0.03 77.09 ± 0.06 81.25 ± 0.05
card	82.33 ± 0.05 84.05 ± 0.05 97.43 ± 0.03	74.57 ± 5.39 76.09 ± 5.06 72.29 ± 9.24	80.03 ± 0.04 81.78 ± 0.03 98.04 ± 0.03	84.43 ± 0.08 85.70 ± 0.07 97.41 ± 0.06	78.47 ± 0.05 80.80 ± 0.04 98.04 ± 0.03	80.08 ± 4.46 82.18 ± 4.74 97.43 ± 0.00	85.36 ± 0.03 86.31 ± 0.02 98.73 ± 0.02
churn	97.46 ± 0.03 77.17 ± 0.10 84.16 ± 0.07	74.55 ± 9.96 66.90 ± 12.09 76.11 ± 6.47	98.06 ± 0.03 69.54 ± 0.08 75.54 ± 0.07	96.97 ± 0.08 44.10 ± 0.35 60.63 ± 0.26	98.06 ± 0.03 73.00 ± 0.06 80.42 ± 0.04	97.46 ± 3.41 74.96 ± 22.36 82.47 ± 15.09	98.74 ± 0.02 80.29 ± 0.05 86.16 ± 0.06
ecoli1	83.34 ± 0.02 30.70 ± 0.04 68.35 ± 0.09	81.11 ± 3.86 17.64 ± 13.89 39.37 ± 36.89	80.38 ± 0.02 58.11 ± 0.05 68.25 ± 0.07	82.97 ± 0.03 48.69 ± 0.05 7.87 ± 0.16	84.34 ± 0.02 57.67 ± 0.04 65.07 ± 0.07	83.81 ± 3.10 41.59 ± 0.25 68.53 ± 6.10	81.11 ± 0.02 59.77 ± 0.08 77.42 ± 0.06
german	74.63 ± 0.06 82.27 ± 0.02 30.27 ± 0.05	50.29 ± 31.76 82.61 ± 1.88 26.92 ± 19.02	74.85 ± 0.04 83.20 ± 0.02 21.57 ± 0.15	20.59 ± 0.15 84.23 ± 0.02 16.47 ± 0.11	72.50 ± 0.04 83.44 ± 0.02 28.06 ± 0.15	74.44 ± 15.71 83.96 ± 3.22 13.65 ± 2.48	81.31 ± 0.06 84.89 ± 0.39 10.26 ± 0.84
haberman	78.72 ± 0.05 76.54 ± 0.06 88.85 ± 0.05	65.64 ± 20.25 64.33 ± 15.20 88.85 ± 4.12	77.92 ± 0.05 75.60 ± 0.07 87.52 ± 0.02	83.17 ± 0.05 72.56 ± 0.35 88.49 ± 0.05	79.88 ± 0.09 77.10 ± 0.10 88.04 ± 0.05	78.46 ± 7.43 76.17 ± 10.96 88.49 ± 7.65	85.21 ± 0.07 81.28 ± 0.07 89.28 ± 0.04
hepatitis	22.36 ± 0.22 64.61 ± 0.18 69.32 ± 0.14	22.36 ± 26.73 53.22 ± 9.70 57.22 ± 0.25	65.95 ± 0.04 83.84 ± 0.08 85.57 ± 0.07	0.00 ± 0.00 77.45 ± 0.02 78.61 ± 0.68	67.37 ± 0.09 84.43 ± 0.07 85.95 ± 0.07	63.72 ± 28.06 85.50 ± 5.88 87.18 ± 22.78	67.16 ± 0.06 85.42 ± 0.06 86.83 ± 0.05
ionosphere	78.62 ± 0.06 78.35 ± 0.05 84.21 ± 0.08	15.18 ± 21.60 51.02 ± 13.80 62.50 ± 7.53	79.89 ± 0.06 78.74 ± 0.05 86.52 ± 0.08	79.12 ± 0.15 74.62 ± 0.09 78.94 ± 0.16	80.10 ± 0.05 79.48 ± 0.05 90.47 ± 0.08	77.33 ± 7.10 78.54 ± 4.95 88.96 ± 2.83	81.33 ± 0.05 80.68 ± 0.04 87.38 ± 0.02
mammographic	80.34 ± 0.08 87.12 ± 0.05 93.15 ± 0.04	58.14 ± 4.10 39.97 ± 13.40 58.84 ± 10.60	84.08 ± 0.08 87.56 ± 0.04 92.70 ± 0.04	77.05 ± 0.13 90.49 ± 0.11 80.64 ± 0.21	89.67 ± 0.07 87.52 ± 0.04 92.69 ± 0.04	89.58 ± 5.48 87.37 ± 5.40 92.55 ± 5.60	85.58 ± 0.01 88.58 ± 0.04 92.67 ± 0.03
monk-2	93.76 ± 0.02 83.30 ± 0.04 20.22 ± 0.12	96.84 ± 2.73 92.60 ± 2.42 3.84 ± 0.49	94.04 ± 0.01 84.09 ± 0.03 12.71 ± 0.10	83.69 ± 0.68 44.19 ± 0.45 51.37 ± 0.41	95.66 ± 0.03 89.13 ± 0.07 6.93 ± 0.10	97.12 ± 3.52 94.42 ± 8.14 18.86 ± 17.89	97.86 ± 0.02 93.99 ± 0.03 20.42 ± 0.36
newthyroid	46.79 ± 0.24 91.97 ± 0.08 86.21 ± 0.19	40.58 ± 10.24 87.54 ± 3.64 34.31 ± 14.00	30.56 ± 0.16 96.05 ± 0.03 89.99 ± 0.06	76.32 ± 0.47 87.13 ± 0.10 74.99 ± 0.16	20.78 ± 0.18 96.67 ± 0.01 92.31 ± 0.02	40.55 ± 16.28 96.55 ± 3.58 95.71 ± 6.70	45.07 ± 0.36 97.62 ± 0.03 96.07 ± 0.05
parkinson	64.75 ± 0.07 73.86 ± 0.07 46.36 ± 0.09	0.20 ± 0.75 7.86 ± 1.05 15.28 ± 21.29	74.12 ± 0.08 80.82 ± 0.07 48.99 ± 0.09	62.01 ± 0.19 72.59 ± 0.13 61.39 ± 0.05	67.02 ± 0.42 79.98 ± 0.06 30.23 ± 0.10	67.02 ± 4.20 72.43 ± 4.10 56.18 ± 7.02	31.52 ± 0.17 37.54 ± 0.12 64.03 ± 0.05
phoneme	56.12 ± 0.07 82.77 ± 0.03 17.71 ± 0.16	29.16 ± 11.08 82.76 ± 2.33 19.32 ± 16.62	58.25 ± 0.07 81.07 ± 0.03 23.14 ± 0.16	69.69 ± 0.04 82.97 ± 0.04 0.00 ± 0.00	42.64 ± 0.08 83.12 ± 0.03 17.77 ± 0.16	65.43 ± 4.71 83.48 ± 5.92 1.78 ± 4.58	71.83 ± 0.04 83.48 ± 0.03 17.84 ± 0.16
planning	44.64 ± 0.09 56.23 ± 0.07 83.27 ± 0.06	36.82 ± 25.29 47.62 ± 30.13 53.53 ± 17.18	45.89 ± 0.10 56.96 ± 0.08 83.88 ± 0.04	56.53 ± 0.11 65.74 ± 0.07 2.11 ± 0.05	44.58 ± 0.17 54.24 ± 0.17 84.57 ± 0.06	44.82 ± 4.40 56.03 ± 3.67 85.46 ± 6.88	49.88 ± 0.08 60.16 ± 0.07 87.87 ± 0.05
saheart	83.92 ± 0.05 87.90 ± 0.06 0.00 ± 0.00	60.92 ± 11.32 86.30 ± 6.65 38.67 ± 34.26	84.95 ± 0.04 87.08 ± 0.05 35.55 ± 0.29	10.54 ± 0.11 82.95 ± 0.36 40.26 ± 0.10	85.43 ± 0.05 88.31 ± 0.06 23.41 ± 0.20	85.97 ± 5.07 85.52 ± 9.83 51.42 ± 0.25	88.07 ± 0.05 79.66 ± 0.06 51.05 ± 0.47
spectfheart	100.00 ± 0.00 100.00 ± 0.00 54.64 ± 0.04	86.34 ± 2.34 75.29 ± 2.58 55.52 ± 9.25	100.00 ± 0.00 100.00 ± 0.00 54.64 ± 0.04	81.61 ± 0.02 40.86 ± 0.47 56.87 ± 0.65	100.00 ± 0.00 100.00 ± 0.00 59.29 ± 0.02	100.00 ± 0.00 100.00 ± 0.00 53.97 ± 2.59	100.00 ± 0.00 100.00 ± 0.00 10.57 ± 0.08
tic-tac-toe	62.01 ± 0.04 30.94 ± 0.04 44.34 ± 0.03	65.36 ± 29.38 31.26 ± 23.27 44.94 ± 11.16	62.01 ± 0.04 43.36 ± 0.03 55.53 ± 0.02	65.38 ± 0.47 20.15 ± 0.04 33.84 ± 0.12	67.15 ± 0.02 1.03 ± 0.02 7.25 ± 0.07	61.23 ± 1.93 8.29 ± 1.15 2.11 ± 0.16	40.00 ± 0.25 8.15 ± 0.04 20.83 ± 0.06
transfusion	20.44 ± 0.12 33.85 ± 0.13 91.82 ± 0.03	1.18 ± 0.28 0.80 ± 0.69 79.49 ± 6.62	68.69 ± 0.11 74.35 ± 0.10 91.67 ± 0.02	44.01 ± 0.92 56.35 ± 0.34 91.72 ± 0.02	77.06 ± 0.07 81.59 ± 0.07 92.51 ± 0.03	82.80 ± 4.88 91.69 ± 1.90 92.22 ± 2.51	80.51 ± 0.04 87.75 ± 0.03 93.80 ± 0.03
vehicle0	87.60 ± 0.05 73.10 ± 0.17 75.89 ± 0.14	55.04 ± 7.58 95.65 ± 4.40 96.51 ± 12.91	89.63 ± 0.03 100.00 ± 0.00 100.00 ± 0.00	90.59 ± 0.03 35.99 ± 0.85 59.42 ± 0.68	90.45 ± 0.03 100.00 ± 0.00 100.00 ± 0.00	90.62 ± 3.70 1.00 ± 0.00 1.00 ± 0.00	92.33 ± 0.04 100.00 ± 0.00 100.00 ± 0.00
vowel0							

Table 7

Continue.

Dataset	SVM-2K (Farquhar et al., 2005) F-measure G-mean	MvTSVM (Xie & Sun, 2015) F-measure G-mean	PSVM-2V (Tang et al., 2018b) F-measure G-mean	MvNPSVM (Tang et al., 2018a) F-measure G-mean	MvLDM (Hu et al., 2025) F-measure G-mean	MvTPMSVM (Quadir & Tanveer, 2024) F-measure G-mean	RoBoSS-MvSVM (Proposed) F-measure G-mean
wine	95.18 ± 0.02	84.50 ± 2.54	97.32 ± 0.04	51.02 ± 0.15	95.85 ± 0.05	98.33 ± 3.61	98.29 ± 0.02
WPBC	97.42 ± 0.01	85.53 ± 3.31	97.72 ± 0.03	58.52 ± 0.31	96.51 ± 0.04	98.73 ± 11.81	99.18 ± 0.01
	7.34 ± 0.10	75.76 ± 12.40	47.04 ± 0.20	14.00 ± 0.27	8.53 ± 0.11	36.76 ± 21.71	35.87 ± 0.19
	20.03 ± 0.17	82.34 ± 13.17	64.12 ± 0.19	37.03 ± 0.35	22.32 ± 0.19	51.34 ± 15.99	50.96 ± 0.16
yeast	30.07 ± 0.08	10.30 ± 6.48	65.75 ± 0.05	33.65 ± 1.58	75.46 ± 0.11	72.98 ± 2.30	64.91 ± 0.03
	42.58 ± 0.06	32.34 ± 16.04	69.23 ± 0.04	56.82 ± 0.47	82.76 ± 0.12	81.31 ± 8.03	82.47 ± 0.02
Average	(65.56, 62.80)	(53.40, 50.82)	(71.77, 70.69)	(59.54, 56.45)	(68.82, 67.53)	(69.11, 65.89)	(71.15, 71.12)

Algorithm 1 NAG-based algorithm to solve RoBoSS-MvSVM.

Input: Training patterns: $\{x_i^{[1]}, x_i^{[2]}, y_i\}_{i=1}^m, y_i \in \{-1, +1\}$; RoBoSS parameters λ_θ, a_θ , learning rate decay factor θ , momentum parameter r , maximum iteration N ;

Initialize: model parameter α_0^θ , velocity $v_0^{[\theta]}$, learning rate β ;

Output: Classifiers parameters: $\alpha^{[1]}, \alpha^{[2]}$

1: Determine $\xi_i^{[1]}, \xi_i^{[2]}$.

$$\xi_i^{[1]} = 1 - y_i \left(\sum_{j=1}^m \alpha_j^{[1]} K_1(x_i^{[1]}, x_j^{[1]}) \right)$$

$$\xi_i^{[2]} = 1 - y_i \left(\sum_{j=1}^m \alpha_j^{[2]} K_2(x_i^{[2]}, x_j^{[2]}) \right), \quad i = 1, 2, \dots, m.$$

2: Look-ahead update: $\tilde{\alpha}_t^{[\theta]} = \alpha_t^{[\theta]} + r v_t$, for $\theta = 1, 2$;

3: Compute gradient:

$$\nabla f(\tilde{\alpha}_t^{[1]}) = \mathcal{K}_1 \tilde{\alpha}_t^{[1]} + D \mathcal{K}_1 (\mathcal{K}_1 \tilde{\alpha}_t^{[1]} - \mathcal{K}_2 \tilde{\alpha}_t^{[2]}) - \frac{C_1}{m} \lambda_1 \sum_{j=1}^m a_1^2 \xi_j^{[1]} \exp(-a_1 \xi_j^{[1]}) y_j \mathcal{K}_1^j,$$

$$\nabla f(\tilde{\alpha}_t^{[2]}) = \mathcal{K}_2 \tilde{\alpha}_t^{[2]} + D \mathcal{K}_2 (\mathcal{K}_1 \tilde{\alpha}_t^{[1]} - \mathcal{K}_2 \tilde{\alpha}_t^{[2]}) - \frac{C_2}{m} \lambda_2 \sum_{j=1}^m a_2^2 \xi_j^{[2]} \exp(-a_2 \xi_j^{[2]}) y_j \mathcal{K}_2^j,$$

where $\mathcal{K}_\theta$ denotes the kernel matrix and $\mathcal{K}_\theta^j$ denotes the components of j th row in kernel matrix.

4: Update velocity: $v_t^{[\theta]} = r v_{t-1}^{[\theta]} - \beta_t \nabla f(\tilde{\alpha}_t^{[\theta]})$;

5: Update model parameters: $\alpha_{t+1}^{[\theta]} = \tilde{\alpha}_t^{[\theta]} + v_t^{[\theta]}$;

6: Update learning rate: $\beta_{t+1} = \beta_t \exp(-\theta t)$;

7: Update current iteration number: $t = t + 1$.

Until:

$t = N$.

Return: $\alpha_t^{[1]}, \alpha_t^{[2]}$.

$Y = \{-1, +1\}$. The data points (x_i, y_i) are assumed to be drawn independently from a joint probability measure $\mathcal{P}$ over the space $X \times Y$. The aim is to learn a binary decision function $\mathcal{H} : X \rightarrow Y$ that maps input instances from the space X to one of the two class labels. The objective of this classification task is to generate a classifier that minimizes the associated classification risk. The expected classification risk associated with a hypothesis $\mathcal{H}$ can be expressed as:

$$\mathcal{R}(\mathcal{H}) = \int_X \mathcal{P}(\mathcal{H}(x) \neq y | x) d\mathcal{P}_X, \quad (14)$$

where $\mathcal{P}(y | x)$ denotes the conditional probability of label y given an input x , and $d\mathcal{P}_X$ represents the marginal distribution of the input sample x . We further assume that the conditional distribution $\mathcal{P}(y | x)$ follows a Bernoulli model, defined by the probabilities $\mathbb{P}(y = -1 | x)$ and $\mathbb{P}(y = 1 | x)$. Now, the objective is to develop a classifier that accurately categorizes inputs while minimizing the overall classification error. Among all possible classifiers, the Bayes classifier is shown to achieve the minimum classification risk, i.e., $f_{\mathcal{H}} = \arg \min_{\mathcal{H}: X \rightarrow Y} \mathcal{R}(\mathcal{H})$. For $\mathbb{P}(x) \neq 1/2$, the Bayes classifier can be defined as:

$$f_{\mathcal{H}}(x) = \begin{cases} +1, & \text{if } \mathbb{P}(y = 1 | x) > 1/2, \\ -1, & \text{otherwise.} \end{cases} \quad (15)$$

The expected error of a classifier $f : X \rightarrow \mathbb{R}$ under the specified loss function $\mathcal{L}$ can be formulated as:

$$\mathcal{R}_{\mathcal{L}, \mathcal{P}}(f) = \int_{X \times Y} \mathcal{L}(1 - yf(x)) d\mathcal{P}. \quad (16)$$

Among all measurable functions, the function $f_{\mathcal{L}, \mathcal{P}}$ that attains the minimum expected error is defined as:

$$f_{\mathcal{L}, \mathcal{P}}(x) = \arg \min_{f(x) \in \mathbb{R}} \int_Y \mathcal{L}(1 - yf(x)) d\mathcal{P}(y|x), \quad \forall x \in X. \quad (17)$$

Therefore, if we can show that the sign of $f_{\mathcal{L}_{\text{RoBoSS}}, \mathcal{P}}$ aligns with the sign of the Bayes classifier, then the classification-calibrated property of the RoBoSS loss is established.

Theorem 1. The RoBoSS loss function $\mathcal{L}_{\text{RoBoSS}}(\cdot)$ possesses the property of classification-calibrated, that is, $f_{\mathcal{L}_{\text{RoBoSS}}, \mathcal{P}}$ aligns with the sign of the Bayes classifier.

Proof. After straightforward computation, we conclude that

$$\begin{aligned} & \int_Y \mathcal{L}_{\text{RoBoSS}}(1 - yf(x)) d\mathcal{P}(y|x) \\ &= \mathcal{L}_{\text{RoBoSS}}(1 + f(x)) \mathbb{P}(y = -1 | x) + \mathcal{L}_{\text{RoBoSS}}(1 - f(x)) \mathbb{P}(y = 1 | x) \end{aligned}$$

Table 8

Comparison of the proposed RoBoSS-MvSVM with baseline models on AWA datasets.

Dataset	SVM-2K (Farquhar et al., 2005) (C_1, σ)	MvTSVM (Xie & Sun, 2015) (C_1, σ)	PSVM-2V (Tang et al., 2018b) (C_1, σ)	MvNPSVM (Tang et al., 2018a) (C_1, D, σ)	MvLDM (Hu et al., 2025) (C_1, v, σ)	MvTPMSVM (Quadir & Tanveer, 2024) (C_1, C_2, σ)	RoBoSS-MvSVM (Proposed) ($a, \lambda, C_1, D, \sigma$)
	Accuracy (%) Time (s)	Accuracy (%) Time (s)	Accuracy (%) Time (s)	Accuracy (%) Time (s)	Accuracy (%) Time (s)	Accuracy (%) Time (s)	Accuracy (%) Time (s)
Chimpanzee vs Giant panda	(0.1, 0.125) 63.00 ± 4.56 2.86	(0.1, 4) 71.31 ± 3.42 3.05	(1, 1) 84.75 ± 4.11 38.36	(0.01, 5, 0.0625) 65.91 ± 7.40 17.83	(10, 1, 0.5) 85.25 ± 0.04 3.38	(0.1, 0.1, 0.03125) 61.25 ± 6.19 3.95	(1, 0.3, 0.01, 1, 0.125) 63.50 ± 4.24 3.03
Chimpanzee vs Hippopotamus	(0.1, 0.25) 75.75 ± 4.29 2.72	(0.1, 4) 70.25 ± 5.55 2.96	(1, 0.25) 75.00 ± 4.59 38.36	(1, 0.05, 0.0625) 77.34 ± 11.64 19.07	(1, 0.1, 0.0625) 76.25 ± 0.05 3.27	(0.1, 0.1, 0.0625) 76.00 ± 3.19 2.92	(2, 1, 0.01, 10, 0.25) 78.50 ± 3.60 2.42
Chimpanzee vs Humpback whale	(0.1, 0.125) 89.50 ± 3.26 2.82	(0.1, 4) 86.00 ± 4.37 3.71	(0.1, 0.125) 89.50 ± 3.24 45.64	(0.01, 3, 1) 87.07 ± 4.42 19.36	(1, 1, 0.0625) 91.50 ± 0.04 3.49	(0.1, 10, 0.125) 90.25 ± 2.56 3.82	(1.2, 0.1, 10, 10, 1) 91.75 ± 3.26 3.88
Chimpanzee vs Leopard	(0.1, 0.0625) 59.50 ± 3.19 3.18	(0.1, 0.125) 69.17 ± 3.42 2.67	(0.1, 0.25) 70.75 ± 4.05 43.75	(1, 5, 0.125) 69.30 ± 13.39 18.93	(1, 1, 0.0625) 70.75 ± 0.04 3.48	(0.1, 0.1, 0.03125) 69.46 ± 4.11 2.75	(1, 0.1, 100, 1, 1) 71.07 ± 2.98 2.42
Chimpanzee vs Persian cat	(0.1, 0.125) 64.25 ± 4.20 2.77	(0.1, 4) 73.00 ± 6.77 3.08	(1, 0.25) 68.50 ± 5.41 37.95	(1, 0.05, 0.0625) 69.88 ± 4.24 18.14	(10, 1, 1) 86.75 ± 0.05 3.04	(0.1, 1, 0.125) 76.97 ± 3.19 2.88	(1, 1, 10, 1, 1) 75.25 ± 3.89 2.72
Chimpanzee vs Pig	(0.1, 0.25) 62.75 ± 4.09 2.97	(0.1, 2) 49.75 ± 3.11 2.89	(0.1, 0.25) 73.75 ± 2.50 28.16	(100, 5, 0.125) 67.49 ± 16.62 19.38	(10, 1, 0.0625) 73.75 ± 0.04 3.30	(0.1, 0.1, 0.0625) 75.88 ± 3.69 3.77	(1, 0.1, 10, 1, 1) 68.25 ± 3.85 2.12
Chimpanzee vs Raccoon	(0.1, 0.125) 70.83 ± 4.79 3.03	(1, 4) 50.50 ± 3.42 3.27	(1, 0.25) 69.75 ± 4.63 40.82	(1, 5, 1) 68.07 ± 18.55 19.06	(1, 1, 0.0625) 70.25 ± 0.04 3.63	(10, 1, 0.03125) 72.40 ± 3.71 3.77	(0.1, 0.1, 10, 10, 1) 73.00 ± 4.63 2.78
Chimpanzee vs Rat	(0.1, 0.125) 63.00 ± 2.09 2.70	(0.1, 4) 69.67 ± 5.05 3.94	(1, 0.25) 66.50 ± 1.63 40.50	(1, 0.05, 0.125) 74.64 ± 8.62 19.13	(1, 0.1, 0.0625) 66.75 ± 0.03 3.20	(0.1, 1, 0.125) 68.00 ± 1.43 3.80	(0, 0, 10, 1, 1) 71.50 ± 2.71 4.04
Chimpanzee vs Seal	(1, 0.25) 71.33 ± 3.47 2.79	(0.1, 4) 74.75 ± 3.47 3.05	(1, 1) 86.75 ± 2.93 25.11	(1, 5, 0.125) 78.60 ± 5.05 18.13	(10, 1, 0.5) 86.00 ± 0.03 3.28	(0.1, 1, 0.125) 79.00 ± 5.41 3.81	(1.2, 0.1, 0.01, 1, 0.25) 78.25 ± 3.47 2.74
Giant panda vs Hippopotamus	(0.1, 1) 76.25 ± 3.11 3.01	(0.1, 2) 70.00 ± 3.58 5.42	(0.1, 0.25) 74.00 ± 3.81 41.39	(1, 5, 0.125) 77.69 ± 6.82 19.03	(10, 1, 1) 79.25 ± 0.03 3.88	(10, 10, 0.0625) 72.50 ± 3.60 3.60	(1.1, 0.3, 1, 1, 0.125) 73.00 ± 3.19 3.45
Giant panda vs Humpback whale	(1, 0.25) 88.00 ± 4.01 3.46	(0.1, 4) 83.33 ± 2.85 4.94	(0.1, 0.125) 89.25 ± 5.08 40.34	(0.01, 5, 0.125) 88.93 ± 4.73 20.40	(1, 1, 0.0625) 91.50 ± 0.05 3.83	(0.01, 1, 0.125) 87.50 ± 3.24 3.61	(1, 0.3, 0.01, 1, 0.125) 89.25 ± 4.01 3.57
Giant panda vs Leopard	(1, 1) 65.50 ± 6.02 3.13	(0.1, 2) 60.50 ± 3.42 4.18	(1, 0.25) 66.00 ± 3.79 43.37	(0.01, 5, 0.125) 65.79 ± 9.37 19.65	(1, 0.1, 0.0625) 67.75 ± 0.04 3.59	(0.01, 0.1, 0.03125) 68.30 ± 7.58 3.59	(1.3, 0.2, 0.01, 1, 1) 71.00 ± 5.38 3.91
Giant panda vs Raccoon	(1, 0.25) 68.25 ± 4.89 3.26	(1, 2) 61.25 ± 7.98 5.86	(0.1, 0.125) 61.00 ± 6.75 44.12	(100, 5, 1) 79.82 ± 9.13 19.64	(1, 1, 0.0625) 71.25 ± 0.06 4.36	(0.01, 0.1, 0.0625) 63.00 ± 4.20 3.61	(1, 1, 0.01, 1, 1) 70.50 ± 6.16 3.34
Giant panda vs Seal	(0.1, 0.125) 76.75 ± 4.20 2.86	(1, 4) 79.75 ± 3.24 4.07	(0.1, 0.125) 76.50 ± 4.87 41.53	(0.01, 0.5, 0.0625) 83.80 ± 3.47 20.21	(1, 1, 0.5) 90.50 ± 0.05 3.11	(0.1, 0.1, 0.125) 80.75 ± 5.56 2.82	(1, 1, 1, 1, 0.125) 77.25 ± 5.20 4.76
Giant panda vs Persian cat	(1, 1) 66.50 ± 4.37 2.80	(10, 4) 69.25 ± 2.93 4.15	(0.1, 0.25) 70.03 ± 5.26 39.95	(100, 5, 0.125) 63.37 ± 5.26 18.95	(10, 1, 0.0625) 64.50 ± 0.06 3.36	(0.1, 1, 0.03125) 70.16 ± 4.79 2.80	(2, 0.6, 1, 10, 1) 70.72 ± 4.63 3.83
Giant panda vs Pig	(1, 0.25) 64.50 ± 8.95 3.12	(0.1, 2) 62.50 ± 6.81 4.34	(0.1, 0.25) 67.00 ± 6.85 41.14	(0.01, 5, 0.0625) 60.78 ± 7.67 18.66	(1, 1, 0.0625) 67.50 ± 0.07 3.38	(0.01, 1, 0.0625) 65.75 ± 5.59 3.57	(0.7, 0.2, 0.01, 1, 1) 69.50 ± 8.37 3.51
Giant panda vs Rat	(0.1, 0.125) 62.25 ± 4.45 3.15	(0.1, 2) 65.25 ± 5.49 4.52	(0.01, 1) 80.75 ± 3.24 42.43	(0.01, 3, 0.0625) 62.19 ± 4.37 24.42	(10, 1, 0.5) 84.75 ± 0.04 4.19	(0.1, 1, 0.125) 65.50 ± 9.19 3.60	(1.4, 0.4, 10, 1, 0.0625) 64.00 ± 5.45 3.25
Hippopotamus vs Raccoon	(0.1, 0.25) 70.50 ± 3.35 3.57	(10, 2) 50.75 ± 3.69 4.34	(0.1, 0.125) 67.75 ± 4.54 38.15	(0.01, 5, 0.0625) 69.14 ± 14.35 24.74	(10, 1, 0.0625) 74.25 ± 0.04 3.42	(0.01, 1, 0.0625) 69.25 ± 3.64 3.61	(1, 0.2, 10, 1, 0.125) 71.89 ± 4.01 5.93
Hippopotamus vs Humpback whale	(0.1, 0.125) 74.50 ± 6.29 3.25	(0.1, 4) 73.50 ± 4.79 4.23	(0.1, 0.25) 78.75 ± 5.55 44.52	(0.01, 0.05, 0.0625) 77.81 ± 4.09 25.46	(1, 1, 0.25) 78.05 ± 0.06 3.55	(0.1, 0.1, 0.125) 78.25 ± 5.18 3.63	(0.5, 0.3, 10, 1, 0.125) 80.25 ± 6.34 3.42
Hippopotamus vs Rat	(0.1, 0.125) 63.25 ± 5.49 2.99	(1, 4) 63.00 ± 3.81 4.49	(0.1, 0.25) 65.25 ± 6.77 42.73	(0.01, 5, 0.125) 63.95 ± 4.87 24.37	(1, 1, 0.0625) 64.00 ± 0.07 3.13	(0.1, 10, 0.125) 67.25 ± 4.89 3.60	(1.3, 0.1, 0.01, 10, 1) 69.25 ± 4.01 3.28
Humpback whale vs Rat	(1, 1) 65.25 ± 5.30 3.19	(0.1, 4) 65.50 ± 15.48 4.66	(0.01, 0.125) 81.25 ± 6.56 42.99	(0.01, 0.5, 0.125) 79.90 ± 0.68 25.17	(10, 0.1, 0.0625) 83.75 ± 0.07 3.37	(0.1, 0.1, 0.125) 82.75 ± 4.20 2.62	(1.4, 0.3, 0.01, 1, 0.25) 85.25 ± 5.11 2.21
Leopard vs Hippopotamus	(10, 1) 70.00 ± 4.81 3.48	(0.1, 2) 67.25 ± 2.65 3.89	(0.1, 0.125) 70.75 ± 3.35 43.09	(0.01, 5, 0.0625) 72.75 ± 3.42 23.96	(1, 1, 0.125) 76.50 ± 0.04 3.39	(1, 1, 0.125) 75.50 ± 6.77 2.61	(1.8, 0.8, 0.01, 1, 0.125) 71.50 ± 5.42 2.81
Leopard vs Humpback whale	(0.1, 0.25) 84.17 ± 3.26 3.69	(10, 4) 80.00 ± 3.69 4.20	(0.1, 0.125) 86.33 ± 3.49 44.33	(0.01, 5, 0.0625) 88.94 ± 4.73 24.09	(10, 1, 0.0625) 89.58 ± 0.04 3.51	(1, 0.01, 0.03125) 87.25 ± 5.11 2.61	(1.2, 1, 1, 1, 0.125) 89.75 ± 3.47 2.67
Leopard vs Pig	(0.1, 0.125) 62.25 ± 5.11 3.48	(0.1, 4) 69.50 ± 2.44 4.54	(0.1, 0.125) 63.75 ± 3.38 43.14	(0.01, 0.5, 1) 56.07 ± 6.59 24.27	(1, 1, 0.0625) 72.00 ± 0.03 3.39	(1, 1, 0.03125) 74.90 ± 3.89 2.62	(0.1, 0.1, 0.01, 1, 0.125) 63.25 ± 4.73 2.83

Table 8

Continue.

Dataset	SVM-2K (Farquhar et al., 2005) (C_1, σ) Accuracy (%) Time (s)	MvTSVM (Xie & Sun, 2015) (C_1, σ) Accuracy (%) Time (s)	PSVM-2V (Tang et al., 2018b) (C_1, σ) Accuracy (%) Time (s)	MvNPSVM (Tang et al., 2018a) (C_1, D, σ) Accuracy (%) Time (s)	MvLDM (Hu et al., 2025) (C_1, v, σ) Accuracy (%) Time (s)	MvTPMSVM (Quadir & Tanveer, 2024) (C_1, C_2, σ) Accuracy (%) Time (s)	RoBoSS-MvSVM (Proposed) ($a, \lambda, C_1, D, \sigma$) Accuracy (%) Time (s)
Leopard vs Raccoon	(0.1, 0.125) 59.50 ± 4.01 3.64	(0.1, 2) 60.50 ± 5.05 3.84	(0.1, 0.25) 61.44 ± 3.47 42.86	(0.01, 5, 1) 54.12 ± 9.21 24.49	(10, 1, 0.5) 60.75 ± 0.04 3.52	(1, 1, 0.03125) 67.27 ± 4.20 3.63	(1, 1, 0.01, 1, 0.125) 69.75 ± 3.69 2.70
Leopard vs Rat	(0.1, 0.125) 59.00 ± 6.70 3.35	(0.1, 2) 52.50 ± 5.97 4.05	(0.1, 0.125) 58.50 ± 7.13 43.35	(0.01, 3, 0.0625) 59.31 ± 4.54 24.68	(1, 1, 0.0625) 65.25 ± 0.06 3.70	(0.1, 0.01, 0.03125) 60.50 ± 4.73 3.63	(0.7, 0.8, 1, 1, 0.0625) 60.75 ± 6.88 2.59
Persian cat vs Humpback whale	(0.1, 1) 84.50 ± 3.47 2.90	(0.1, 4) 70.83 ± 5.03 5.18	(0.01, 0.125) 75.25 ± 4.37 44.28	(0.01, 5, 0.125) 86.75 ± 5.11 24.88	(10, 1, 0.125) 81.50 ± 0.05 3.27	(0.01, 0.01, 0.0625) 79.75 ± 3.81 2.62	(1, 0.3, 0.01, 0.25, 1) 89.25 ± 3.47 2.85
Persian cat vs Raccoon	(10, 1) 61.00 ± 2.71 3.33	(0.1, 2) 70.24 ± 3.42 4.84	(0.1, 0.25) 58.91 ± 2.56 38.59	(1, 5, 0.0625) 61.85 ± 14.67 22.23	(1, 1, 0.0625) 59.50 ± 0.03 3.18	(1, 1, 0.03125) 58.75 ± 3.85 3.65	(0.1, 0.3, 0.01, 0.1, 1) 74.50 ± 2.93 2.74
Persian cat vs Hippopotamus	(1, 1) 67.25 ± 5.63 3.05	(0.1, 2) 70.00 ± 3.42 4.76	(0.1, 0.25) 68.00 ± 5.42 37.24	(1, 5, 0.125) 78.90 ± 13.82 21.24	(1, 0.1, 0.0625) 67.50 ± 0.05 2.96	(0.1, 1, 0.03125) 81.12 ± 4.47 3.64	(1.7, 0.4, 0.01, 1, 1) 81.50 ± 5.63 2.76
Persian cat vs Leopard	(0.1, 0.125) 62.00 ± 2.59 3.40	(0.1, 0.125) 62.33 ± 3.42 4.29	(0.1, 0.25) 69.00 ± 3.24 44.67	(0.01, 3, 1) 67.78 ± 1.44 23.61	(10, 1, 0.25) 70.50 ± 0.02 3.25	(0.1, 0.1, 0.03125) 63.75 ± 3.58 3.63	(1, 1.4, 0.01, 10, 1) 71.57 ± 3.19 2.74
Persian cat vs Pig	(0.1, 0.125) 61.00 ± 2.98 3.31	(0.1, 2) 61.08 ± 3.42 4.63	(0.1, 0.125) 61.25 ± 3.11 37.70	(100, 5, 0.125) 64.36 ± 11.48 24.14	(1, 1, 0.0625) 61.25 ± 0.04 3.35	(1, 10, 0.03125) 60.50 ± 4.01 3.61	(1, 1, 0.01, 1, 1) 66.81 ± 3.19 2.71
Persian cat vs Rat	(0.1, 0.125) 59.00 ± 1.85 3.20	(10, 2) 55.87 ± 3.35 4.45	(0.1, 0.125) 59.00 ± 2.34 36.80	(0.01, 5, 0.125) 53.88 ± 19.85 23.73	(1, 1, 1) 60.25 ± 0.04 3.16	(0.1, 1, 0.03125) 59.50 ± 2.88 3.64	(1.2, 0.3, 0.01, 1, 0.0625) 64.25 ± 2.34 2.87
Pig vs Hippopotamus	(0.1, 0.25) 64.17 ± 4.45 3.30	(0.1, 4) 67.75 ± 6.46 4.38	(0.1, 0.25) 67.50 ± 5.97 38.98	(100, 5, 1) 67.62 ± 18.97 24.38	(10, 1, 0.0625) 66.75 ± 0.06 3.24	(0.1, 0.01, 0.0625) 68.25 ± 5.70 3.62	(0, 0, 10, 0.1, 0.125) 65.00 ± 3.81 2.45
Pig vs Raccoon	(0.1, 0.25) 58.50 ± 3.49 3.60	(0.1, 4) 53.25 ± 6.35 4.26	(0.1, 0.125) 56.00 ± 1.85 43.59	(100, 0.5, 0.125) 60.08 ± 20.42 24.79	(1, 1, 0.0625) 60.75 ± 0.04 3.42	(0.1, 0.1, 0.03125) 52.25 ± 3.89 3.61	(1.7, 0.2, 0.01, 10, 1) 63.25 ± 3.42 2.50
Pig vs Humpback whale	(10, 0.125) 86.25 ± 5.18 3.57	(0.1, 4) 84.21 ± 4.63 4.27	(0.1, 0.25) 87.25 ± 2.27 44.82	(0.01, 5, 1) 81.04 ± 3.11 24.88	(1, 1, 0.0625) 86.75 ± 0.03 3.27	(1, 10, 0.125) 85.50 ± 4.73 3.62	(2, 1, 10, 1, 0.125) 89.25 ± 4.97 2.65
Pig vs Rat	(0.1, 0.125) 58.00 ± 4.01 3.34	(0.1, 2) 59.50 ± 4.54 4.40	(1, 0.25) 55.75 ± 5.76 43.80	(100, 5, 0.125) 51.98 ± 3.81 24.27	(1, 1, 0.0625) 55.75 ± 0.05 3.47	(0.01, 0.01, 0.03125) 55.50 ± 8.51 3.62	(0.1, 0.3, 10, 1, 0.125) 69.25 ± 4.79 2.53
Raccoon vs Humpback whale	(0.1, 0.125) 86.25 ± 2.34 3.51	(0.1, 4) 84.75 ± 2.85 4.38	(0.1, 0.125) 86.00 ± 2.98 43.67	(100, 3, 0.0625) 83.42 ± 2.27 24.52	(1, 1, 0.125) 86.75 ± 0.03 3.24	(1, 0.01, 0.125) 86.25 ± 3.79 3.62	(0.1, 0.5, 0.01, 1, 1) 87.50 ± 2.34 6.18
Raccoon vs Rat	(0.1, 0.125) 57.75 ± 5.55 3.55	(0.1, 2) 53.00 ± 4.37 4.38	(0.1, 0.25) 59.00 ± 6.03 43.84	(0.01, 5, 0.0625) 60.20 ± 2.80 25.44	(1, 1, 0.125) 59.50 ± 0.07 3.44	(1, 10, 0.125) 61.75 ± 3.79 3.61	(0.4, 0.1, 0.01, 1, 1) 60.75 ± 6.27 2.94
Seal vs Hippopotamus	(10, 1) 60.00 ± 7.52 3.44	(10, 4) 44.75 ± 3.42 4.84	(1, 0.125) 57.25 ± 5.55 43.53	(0.01, 5, 0.125) 53.75 ± 22.82 23.60	(1, 1, 0.125) 59.75 ± 0.08 4.62	(1, 10, 0.03125) 55.75 ± 4.47 3.61	(1.2, 1, 10, 1, 0.125) 60.50 ± 7.74 2.62
Seal vs Humpback whale	(0.1, 0.25) 74.75 ± 5.35 3.22	(0.1, 4) 66.00 ± 5.18 4.61	(0.1, 1) 78.33 ± 6.16 43.14	(0.01, 0.05, 0.0625) 69.42 ± 13.71 22.35	(1, 1, 0.125) 76.25 ± 0.06 3.97	(1, 10, 0.0625) 74.25 ± 7.07 3.62	(1, 1, 0.01, 10, 0.25) 80.75 ± 5.70 2.68
Seal vs Leopard	(0.1, 0.25) 70.50 ± 3.69 3.39	(0.1, 0.125) 50.00 ± 3.42 4.57	(0.1, 0.125) 77.25 ± 4.45 15.52	(0.01, 5, 0.0625) 76.53 ± 10.22 25.38	(1, 0.1, 0.0625) 75.70 ± 0.06 3.87	(1, 1, 0.03125) 77.25 ± 4.45 3.64	(0.1, 0.4, 10, 1, 0.125) 78.00 ± 4.38 2.63
Seal vs Persian cat	(1, 1) 69.25 ± 4.56 3.06	(1, 4) 71.75 ± 5.49 4.53	(0.1, 0.25) 72.75 ± 7.98 36.29	(0.01, 0.5, 0.0625) 70.76 ± 2.27 24.64	(1, 1, 1) 73.00 ± 0.08 3.70	(1, 100, 0.125) 72.25 ± 5.42 3.66	(1.7, 0.2, 0.01, 1, 1) 76.25 ± 4.56 3.09
Seal vs Pig	(10, 0.25) 71.25 ± 6.10 3.21	(1, 4) 50.25 ± 3.42 4.65	(1, 0.125) 69.17 ± 4.09 18.98	(0.01, 5, 0.0625) 66.67 ± 14.96 25.13	(1, 1, 1) 73.25 ± 0.05 4.45	(1, 1, 0.03125) 68.00 ± 3.01 4.61	(1, 0.8, 0.01, 1, 0.125) 77.00 ± 6.56 2.84
Seal vs Raccoon	(0.1, 0.25) 75.25 ± 4.09 3.47	(1, 4) 50.50 ± 3.42 5.01	(0.1, 0.125) 74.75 ± 7.04 41.43	(1, 5, 1) 74.61 ± 8.32 23.96	(10, 1, 0.0625) 78.75 ± 0.07 3.91	(0.1, 1, 0.03125) 72.50 ± 7.60 3.62	(1.2, 0.3, 0.01, 10, 1) 79.00 ± 4.45 2.73
Seal vs Rat	(1, 0.125) 67.00 ± 1.68 3.17	(0.1, 4) 70.00 ± 5.66 4.41	(0.1, 0.125) 66.75 ± 2.71 43.17	(0.01, 3, 0.0625) 63.98 ± 7.40 22.84	(1, 1, 0.0625) 69.00 ± 0.04 4.78	(1, 0.1, 0.125) 66.75 ± 4.73 4.62	(1, 0.1, 1, 10, 0.125) 70.75 ± 2.44 2.79
Average Accuracy	68.67	65.44	71.17	70.14	73.57	71.23	73.85
Average Rank	5.14	5.60	4.11	4.62	2.74	3.79	1.99

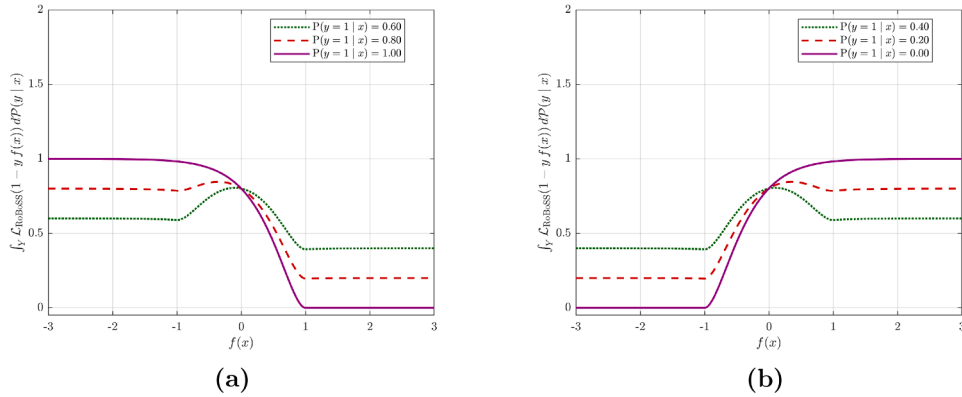
Table 9

Sensitivity and specificity of the proposed RoBoSS-MvSVM and baseline models on the AwA datasets.

Dataset	SVM-2K (Farquhar et al., 2005) Sensitivity Specificity	MvTSVM (Xie & Sun, 2015) Sensitivity Specificity	PSVM-2V (Tang et al., 2018b) Sensitivity Specificity	MvNPSVM (Tang et al., 2018a) Sensitivity Specificity	MvLDM (Hu et al., 2025) Sensitivity Specificity	MvTPMSVM (Quadir & Tanveer, 2024) Sensitivity Specificity	RoBoSS-MvSVM (Proposed) Sensitivity Specificity
Chimpanzee vs Giant panda	40.10 ± 0.10	65.89 ± 0.01	79.59 ± 0.09	48.53 ± 0.41	80.64 ± 0.05	47.27 ± 11.81	39.16 ± 0.10
Chimpanzee vs Hippopotamus	86.18 ± 0.04	76.72 ± 2.14	90.12 ± 0.04	83.30 ± 0.34	90.11 ± 0.05	75.12 ± 5.78	86.21 ± 0.04
Chimpanzee vs Humpback whale	73.53 ± 0.06	90.74 ± 8.13	77.31 ± 0.07	79.89 ± 0.31	79.97 ± 0.04	69.63 ± 4.14	76.20 ± 0.06
Chimpanzee vs Leopard	78.14 ± 0.11	50.39 ± 13.59	72.64 ± 0.10	74.79 ± 0.29	72.67 ± 0.11	81.97 ± 7.87	77.19 ± 0.10
Chimpanzee vs Persian cat	95.13 ± 0.03	89.11 ± 5.14	95.13 ± 0.03	98.64 ± 0.03	97.08 ± 0.02	95.36 ± 5.02	96.59 ± 0.03
Chimpanzee vs Pig	84.07 ± 0.07	82.94 ± 6.13	84.07 ± 0.07	75.51 ± 0.10	85.93 ± 0.07	84.74 ± 7.50	87.56 ± 0.07
Chimpanzee vs Raccoon	33.51 ± 0.07	60.11 ± 0.00	51.56 ± 0.07	72.00 ± 0.31	47.84 ± 0.07	53.51 ± 6.39	47.62 ± 0.06
Chimpanzee vs Rat	85.41 ± 0.03	79.32 ± 1.08	91.00 ± 0.09	66.59 ± 0.03	93.39 ± 0.06	85.41 ± 2.66	97.14 ± 0.03
Chimpanzee vs Seal	96.39 ± 0.04	95.62 ± 4.44	97.47 ± 0.02	55.26 ± 0.03	88.99 ± 0.02	98.48 ± 3.99	80.24 ± 0.04
Giant panda vs Hippopotamus	31.56 ± 0.11	49.83 ± 16.09	39.19 ± 0.13	84.50 ± 0.37	84.29 ± 0.12	55.45 ± 9.50	72.93 ± 0.10
Giant panda vs Humpback whale	29.72 ± 0.08	99.02 ± 1.04	69.18 ± 0.06	74.11 ± 0.39	69.18 ± 0.06	55.84 ± 8.42	54.68 ± 0.09
Giant panda vs Leopard	95.92 ± 0.03	0.54 ± 3.47	78.48 ± 0.02	60.86 ± 0.34	78.56 ± 0.02	95.92 ± 3.79	87.12 ± 0.05
Giant panda vs Pig	50.97 ± 0.09	99.44 ± 0.00	60.05 ± 0.12	98.92 ± 0.43	55.07 ± 0.13	63.19 ± 8.33	51.37 ± 0.08
Giant panda vs Raccoon	89.92 ± 0.03	1.57 ± 2.48	79.36 ± 0.06	37.22 ± 0.21	85.42 ± 0.08	81.61 ± 3.11	95.69 ± 0.03
Giant panda vs Rat	77.83 ± 0.09	60.66 ± 8.08	88.64 ± 0.06	90.50 ± 0.22	93.26 ± 0.06	74.38 ± 11.81	89.63 ± 0.09
Giant panda vs Seal	48.73 ± 0.05	80.43 ± 3.48	44.46 ± 0.07	58.78 ± 0.44	40.55 ± 0.08	60.73 ± 10.78	47.98 ± 0.07
Hippopotamus vs Raccoon	78.72 ± 0.05	59.20 ± 6.94	88.61 ± 0.03	95.47 ± 0.17	88.59 ± 0.05	68.44 ± 11.51	89.55 ± 0.05
Hippopotamus vs Rat	65.77 ± 0.07	90.65 ± 4.50	85.31 ± 0.04	77.31 ± 0.25	83.76 ± 0.04	88.47 ± 1.10	66.59 ± 0.07
Hippopotamus vs Seal	93.29 ± 0.04	87.32 ± 8.67	81.22 ± 0.08	93.25 ± 0.20	81.86 ± 0.07	89.14 ± 6.41	83.73 ± 0.02
Humpback whale vs Rat	60.24 ± 0.06	53.35 ± 7.27	67.12 ± 0.04	62.13 ± 0.36	77.21 ± 0.04	55.88 ± 7.87	61.58 ± 0.06
Humpback whale vs Seal	98.43 ± 0.02	83.98 ± 2.75	97.50 ± 0.02	98.64 ± 0.02	96.99 ± 0.02	98.48 ± 3.91	97.50 ± 0.02
Humpback whale vs Persian cat	77.27 ± 0.07	83.58 ± 6.07	80.90 ± 0.09	79.21 ± 0.15	85.98 ± 0.08	76.02 ± 6.28	80.90 ± 0.07
Humpback whale vs Pig	44.59 ± 0.07	56.32 ± 0.01	59.62 ± 0.08	39.93 ± 0.45	55.16 ± 0.06	61.67 ± 10.97	48.65 ± 0.06
Humpback whale vs Raccoon	85.48 ± 0.05	65.98 ± 4.12	72.51 ± 0.04	91.65 ± 0.17	80.47 ± 0.01	74.93 ± 3.36	95.42 ± 0.04
Humpback whale vs Rat	65.74 ± 0.13	79.30 ± 17.01	42.20 ± 0.08	60.95 ± 0.44	71.70 ± 0.08	45.77 ± 8.49	81.46 ± 0.13
Humpback whale vs Seal	71.17 ± 0.06	43.10 ± 14.94	80.22 ± 0.09	98.70 ± 0.49	70.91 ± 0.07	78.64 ± 9.14	62.49 ± 0.05
Persian cat vs Hippopotamus	89.56 ± 0.03	75.16 ± 3.86	88.16 ± 0.03	88.65 ± 0.07	93.62 ± 0.03	89.12 ± 7.75	90.67 ± 0.05
Persian cat vs Humpback whale	64.03 ± 0.07	84.40 ± 5.71	65.09 ± 0.09	78.74 ± 0.29	87.77 ± 0.09	71.66 ± 11.11	64.11 ± 0.08
Persian cat vs Leopard	91.13 ± 0.03	66.11 ± 2.46	97.39 ± 0.04	54.58 ± 0.01	97.39 ± 0.03	91.49 ± 4.14	81.26 ± 0.03
Persian cat vs Pig	41.67 ± 0.09	69.98 ± 9.88	44.00 ± 0.09	72.16 ± 0.27	31.55 ± 0.09	48.83 ± 8.07	65.61 ± 0.09
Persian cat vs Raccoon	81.12 ± 0.15	79.70 ± 11.84	76.27 ± 0.08	51.91 ± 0.41	78.34 ± 0.09	66.89 ± 13.14	73.49 ± 0.16
Persian cat vs Rat	49.23 ± 0.10	45.55 ± 17.93	58.19 ± 0.09	69.66 ± 0.49	57.19 ± 0.09	63.52 ± 12.48	67.60 ± 0.14
Persian cat vs Seal	90.66 ± 0.07	99.44 ± 3.81	93.17 ± 0.05	96.82 ± 0.05	90.59 ± 0.02	95.56 ± 15.75	78.05 ± 0.11
Raccoon vs Hippopotamus	34.19 ± 0.09	30.71 ± 11.16	68.91 ± 0.05	27.56 ± 0.32	78.85 ± 0.07	33.49 ± 8.42	49.99 ± 0.09
Raccoon vs Rat	64.37 ± 0.05	89.18 ± 2.75	55.62 ± 0.04	56.80 ± 0.35	70.38 ± 0.03	51.00 ± 4.90	57.06 ± 0.08
Raccoon vs Seal	78.05 ± 0.06	12.41 ± 1.02	80.31 ± 0.06	81.49 ± 0.27	78.08 ± 0.07	87.52 ± 4.43	83.33 ± 0.05
Raccoon vs Persian cat	85.55 ± 0.05	87.37 ± 11.83	87.54 ± 0.04	89.60 ± 0.14	88.04 ± 0.03	79.32 ± 4.08	86.95 ± 0.06
Raccoon vs Pig	63.74 ± 0.08	59.98 ± 14.22	69.98 ± 0.09	66.02 ± 0.28	69.45 ± 0.10	76.02 ± 9.04	65.51 ± 0.08
Raccoon vs Raccoon	75.44 ± 0.06	52.73 ± 10.40	88.90 ± 0.06	87.14 ± 0.13	91.95 ± 0.04	82.06 ± 3.57	84.11 ± 0.07
Raccoon vs Rat	51.33 ± 0.08	73.53 ± 10.57	41.75 ± 0.10	40.76 ± 0.35	36.29 ± 0.09	50.03 ± 7.40	56.84 ± 0.09
Rat vs Hippopotamus	91.47 ± 0.07	69.88 ± 10.48	81.56 ± 0.07	83.38 ± 0.09	84.49 ± 0.06	73.24 ± 6.84	85.09 ± 0.07
Rat vs Humpback whale	39.30 ± 0.05	62.08 ± 33.39	80.99 ± 0.06	76.42 ± 0.20	83.02 ± 0.07	91.88 ± 6.19	85.60 ± 0.05
Rat vs Leopard	89.59 ± 0.04	95.90 ± 7.44	90.40 ± 0.03	92.66 ± 0.06	85.45 ± 0.05	76.67 ± 6.45	92.66 ± 0.04
Rat vs Pig	52.32 ± 0.08	38.67 ± 2.99	51.08 ± 0.08	56.18 ± 0.32	67.61 ± 0.07	71.99 ± 9.55	50.03 ± 0.08
Rat vs Raccoon	98.46 ± 0.02	84.76 ± 2.59	93.89 ± 0.01	97.73 ± 0.02	95.45 ± 0.02	94.79 ± 5.25	97.94 ± 0.02
Rat vs Rat	70.35 ± 0.07	75.05 ± 7.25	80.35 ± 0.07	80.16 ± 0.13	85.91 ± 0.07	79.39 ± 6.65	79.32 ± 0.07
Rat vs Seal	91.07 ± 0.06	79.46 ± 9.64	91.17 ± 0.05	52.20 ± 0.18	86.99 ± 0.03	74.70 ± 7.32	92.65 ± 0.05
Rat vs Persian cat	33.80 ± 0.08	59.38 ± 9.61	36.96 ± 0.08	59.95 ± 0.43	57.04 ± 0.07	75.09 ± 9.77	34.33 ± 0.08
Rat vs Pig	87.93 ± 0.05	95.15 ± 8.76	79.67 ± 0.08	49.27 ± 0.22	76.72 ± 0.01	64.45 ± 5.23	86.07 ± 0.06
Rat vs Raccoon	30.92 ± 0.09	26.52 ± 5.02	44.13 ± 0.09	58.97 ± 0.45	45.14 ± 0.08	70.09 ± 4.78	33.71 ± 0.10
Rat vs Rat	95.79 ± 0.04	100.00 ± 1.26	93.41 ± 0.04	97.73 ± 0.03	97.41 ± 0.02	85.61 ± 8.32	86.41 ± 0.03
Rat vs Seal	22.10 ± 0.09	4.98 ± 10.91	23.67 ± 0.10	20.89 ± 0.22	33.18 ± 0.08	35.65 ± 8.62	35.87 ± 0.09
Seal vs Hippopotamus	79.83 ± 0.06	81.33 ± 2.83	63.06 ± 0.04	86.50 ± 0.16	78.13 ± 0.05	67.56 ± 5.61	96.65 ± 0.06
Seal vs Rat	90.51 ± 0.07	60.27 ± 7.26	87.38 ± 0.08	87.00 ± 0.12	84.90 ± 0.08	91.96 ± 5.54	82.71 ± 0.07
Seal vs Raccoon	23.21 ± 0.07	66.71 ± 0.02	44.33 ± 0.08	83.00 ± 0.34	24.48 ± 0.06	28.49 ± 7.68	57.61 ± 0.07
Seal vs Persian cat	97.89 ± 0.05	73.77 ± 0.25	73.50 ± 0.04	40.69 ± 0.06	94.45 ± 0.05	89.09 ± 6.65	93.51 ± 0.04
Seal vs Pig	37.13 ± 0.10	70.81 ± 0.01	39.49 ± 0.10	66.00 ± 0.33	38.55 ± 0.09	71.12 ± 8.06	82.91 ± 0.10
Seal vs Raccoon	98.64 ± 0.02	70.22 ± 1.04	96.51 ± 0.01	91.80 ± 0.09	96.51 ± 0.01	91.12 ± 3.23	81.04 ± 0.02
Seal vs Rat	25.44 ± 0.05	64.45 ± 0.03	39.28 ± 0.08	71.00 ± 0.27	42.72 ± 0.07	32.17 ± 5.32	48.68 ± 0.05
Seal vs Seal	98.55 ± 0.02	60.21 ± 5.42	98.52 ± 0.01	64.56 ± 0.34	97.99 ± 0.01	95.51 ± 1.94	97.14 ± 0.02
Pig vs Hippopotamus	26.54 ± 0.04	64.24 ± 0.01	27.10 ± 0.04	63.22 ± 0.27	25.41 ± 0.05	30.66 ± 4.93	60.32 ± 0.05
Pig vs Rat	95.65 ± 0.03	57.92 ± 6.32	95.65 ± 0.02	65.49 ± 0.09	97.05 ± 0.02	90.54 ± 1.69	77.62 ± 0.03
Pig vs Raccoon	35.96 ± 0.03	51.96 ± 2.29	36.52 ± 0.04	40.01 ± 0.46	35.34 ± 0.08	40.09 ± 6.06	40.70 ± 0.03
Pig vs Seal	82.14 ± 0.04	59.77 ± 3.27	81.64 ± 0.04	67.76 ± 0.43	85.10 ± 0.02	79.26 ± 5.48	79.82 ± 0.04
Pig vs Persian cat	64.33 ± 0.09	50.48 ± 19.67	67.28 ± 0.05	84.01 ± 0.44	65.79 ± 0.06	79.13 ± 12.07	79.71 ± 0.08
Pig vs Pig	64.80 ± 0.12	85.19 ± 7.70	67.37 ± 0.10	51.24 ± 0.46	67.37 ± 0.08	56.74 ± 12.25	49.90 ± 0.12
Pig vs Raccoon	42.56 ± 0.11	83.65 ± 18.47	43.97 ± 0.08	68.19 ± 0.47	58.73 ± 0.11	40.28 ± 8.26	68.62 ± 0.15
Pig vs Rat	75.66 ± 0.08	24.55 ± 22.74	69.09 ± 0.09	51.98 ± 0.44	63.43 ± 0.13	64.77 ± 5.30	61.80 ± 0.15
Pig vs Seal	93.87 ± 0.02	79.78 ± 6.58	94.00 ± 0.02	87.56 ± 0.06	95.05 ± 0.02	91.68 ± 6.62	94.83 ± 0.02
Pig vs Persian cat	78.43 ± 0.10	90.11 ± 8.60	80.50 ± 0.06	74.51 ± 0.15	78.45 ± 0.07	78.59 ± 11.39	77.60 ± 0.10
Pig vs Pig	90.07 ± 0.04	98.38 ± 6.25	91.53 ± 0.05	47.41 ± 0.09	96.01 ± 0.01	69.97 ± 13.35	93.64 ± 0.05
Pig vs Raccoon	26.15 ± 0.07	20.04 ± 8.87	20.31 ± 0.11	56.56 ± 0.30	15.71 ± 0.05	41.93 ± 12.91	24.51 ± 0.09

Table 9
Continue.

Dataset	SVM-2K (Farquhar et al., 2005)	MvTSVM (Xie & Sun, 2015)	PSVM-2V (Tang et al., 2018b)	MvNPSVM (Tang et al., 2018a)	MvLDM (Hu et al., 2025)	MvTPMSVM (Quadir & Tanveer, 2024)	RoBoSS-MvSVM (Proposed)
	Sensitivity	Sensitivity	Sensitivity	Sensitivity	Sensitivity	Sensitivity	Sensitivity
	Specificity	Specificity	Specificity	Specificity	Specificity	Specificity	Specificity
Raccoon vs Humpback whale	93.95 ± 0.04	88.97 ± 2.91	93.47 ± 0.02	78.58 ± 0.08	91.49 ± 0.01	94.41 ± 3.29	94.06 ± 0.04
	78.35 ± 0.06	80.47 ± 6.60	78.35 ± 0.08	88.26 ± 0.15	81.96 ± 0.07	77.64 ± 6.82	81.83 ± 0.06
Raccoon vs Rat	91.45 ± 0.04	95.90 ± 5.90	89.48 ± 0.03	92.27 ± 0.08	94.90 ± 0.03	78.84 ± 6.58	95.75 ± 0.04
	24.15 ± 0.06	9.65 ± 6.06	28.64 ± 0.08	28.12 ± 0.30	24.29 ± 0.08	41.61 ± 10.12	22.81 ± 0.07
Seal vs Hippopotamus	48.68 ± 0.06	80.33 ± 0.00	49.87 ± 0.08	42.39 ± 0.52	39.59 ± 0.13	54.37 ± 7.00	75.08 ± 0.08
	70.75 ± 0.10	9.61 ± 0.01	65.09 ± 0.07	65.12 ± 0.49	75.10 ± 0.06	57.64 ± 7.69	40.00 ± 0.10
Seal vs Humpback whale	81.19 ± 0.09	65.68 ± 18.59	85.80 ± 0.13	77.84 ± 0.33	78.79 ± 0.11	77.11 ± 12.71	84.06 ± 0.09
	68.57 ± 0.09	67.95 ± 20.51	72.26 ± 0.09	61.00 ± 0.37	74.07 ± 0.08	71.47 ± 13.78	67.13 ± 0.09
Seal vs Leopard	42.37 ± 0.10	99.53 ± 0.01	61.49 ± 0.10	54.92 ± 0.26	68.59 ± 0.12	63.49 ± 11.43	63.41 ± 0.11
	98.57 ± 0.04	0.53 ± 0.54	93.20 ± 0.06	98.14 ± 0.04	82.81 ± 0.06	91.10 ± 4.04	92.67 ± 0.04
Seal vs Persian cat	90.55 ± 0.05	66.82 ± 6.64	90.60 ± 0.05	93.47 ± 0.07	91.60 ± 0.04	83.94 ± 5.42	77.48 ± 0.05
	49.82 ± 0.06	76.47 ± 7.50	54.77 ± 0.13	48.06 ± 0.39	54.24 ± 0.14	59.03 ± 7.44	77.19 ± 0.06
Seal vs Pig	48.21 ± 0.13	100.00 ± 0.01	56.95 ± 0.12	51.35 ± 0.36	58.82 ± 0.12	65.15 ± 9.59	57.19 ± 0.14
	94.13 ± 0.04	0.49 ± 0.85	80.62 ± 0.05	81.98 ± 0.26	86.64 ± 0.06	71.33 ± 8.36	82.92 ± 0.03
Seal vs Raccoon	56.86 ± 0.08	100.00 ± 0.01	61.02 ± 0.11	98.43 ± 0.23	71.14 ± 0.11	62.49 ± 10.53	68.99 ± 0.08
	93.49 ± 0.05	0.99 ± 1.98	88.53 ± 0.08	50.79 ± 0.09	86.53 ± 0.07	82.72 ± 6.25	90.45 ± 0.06
Seal vs Rat	76.01 ± 0.06	42.74 ± 12.18	76.15 ± 0.07	76.77 ± 0.23	78.04 ± 0.07	43.39 ± 7.75	76.67 ± 0.07
	58.08 ± 0.04	96.89 ± 2.84	57.71 ± 0.09	51.19 ± 0.40	60.13 ± 0.09	88.61 ± 6.88	61.18 ± 0.06
Average	(70.18, 67.45)	(78.96, 52.15)	(73.70, 68.90)	(74.83, 65.86)	(75.03, 72.16)	(69.34, 72.68)	(76.56, 69.61)

**Fig. 3.** Graph of $\int_Y \mathcal{L}_{\text{RoBoSS}}(1 - yf(x))d\mathcal{P}(y|x)$ with respect to $f(x)$ for different $\mathbb{P}(y = 1 | x)$ values.

$$= \begin{cases} g_1(x)\mathbb{P}(y = 1 | x), & f(x) \leq -1, \\ (g_1(x) - g_2(x))\mathbb{P}(y = 1 | x) + g_2(x), & -1 < f(x) < 1, \\ g_2(x)(1 - \mathbb{P}(y = 1 | x)), & f(x) \geq 1. \end{cases} \quad (18)$$

where

$$g_1(x) = \lambda(1 - (a(1 - f(x)) + 1)\exp(-a(1 - f(x))))$$

and

$$g_2(x) = \lambda(1 - (a(1 + f(x)) + 1)\exp(-a(1 + f(x))))$$

Figs. 3(a) and 3(b) depict the graph of $\int_Y \mathcal{L}_{\text{RoBoSS}}(1 - yf(x))d\mathcal{P}(y|x)$ with respect to $f(x)$ under two scenarios: when $\mathbb{P}(y = 1 | x) > 1/2$ and when $\mathbb{P}(y = 1 | x) < 1/2$, respectively. These graphs show that the expected loss attains its minimum at a positive value of $f(x)$ when $\mathbb{P}(y = 1 | x) > 1/2$ (Fig. 3(a)), and at a negative value of $f(x)$ when $\mathbb{P}(y = 1 | x) < 1/2$ (Fig. 3(b)). Hence, the RoBoSS loss function is classification-calibrated as it preserves the same sign as the Bayes classifier. Hence the result. $\square$

The classification-calibrated property of the RoBoSS loss guarantees that the minimization of the empirical RoBoSS risk results in a classifier whose decision boundary coincides with the Bayes optimal classifier. Consequently, this property ensures that the proposed RoBoSS-MvSVM produces statistically valid predictions and exhibits reliable decision behavior, even in the presence of noisy or extreme samples.

4.3. Generalization capability analysis

We now present generalization of the error bound for RoBoSS-MvSVM derived using Rademacher complexity. We begin by introducing the definition of Rademacher complexity (Bartlett & Mendelson, 2023), followed by a three pertinent lemmas (Quadir et al., 2025a; Tang et al., 2018b). Here, $\mathbb{E}_\epsilon$ denotes the expectation taken over the random vector ϵ .

Definition 1. Let $\mathcal{T} = \{x_i, i = 1, 2, \dots, m\}$ be a sample of size m drawn independently from a probability distribution $\mathcal{P}$ on a set X and $\mathcal{H}$ be a class of real-valued functions with domain X . Then, the empirical Rademacher complexity of $\mathcal{H}$ is defined as

$$\hat{\mathfrak{R}}_m(\mathcal{H}) = \mathbb{E}_\epsilon \left[\sup_{h \in \mathcal{H}} \left| \frac{2}{m} \sum_{i=1}^m \epsilon_i h(x_i) \right| : x_1, x_2, \dots, x_m \right], \quad (19)$$

where $\epsilon = \{\epsilon_1, \epsilon_2, \dots, \epsilon_m\}$ are independent Rademacher random variables, each taking values in $\{+1, -1\}$. The Rademacher complexity of $\mathcal{H}$ is expressed as

$$\mathfrak{R}_m(\mathcal{H}) = \mathbb{E}_{\mathcal{T}} [\hat{\mathfrak{R}}_m(\mathcal{H})] = \mathbb{E}_{\mathcal{T}, \epsilon} \left[\sup_{h \in \mathcal{H}} \left| \frac{2}{m} \sum_{i=1}^m \epsilon_i h(x_i) \right| \right]. \quad (20)$$

Lemma 1. Let $\eta \in (0, 1)$, and suppose $\mathcal{H}$ be a family of functions from the domain X to $[0, 1]$. Let $\mathcal{T} = \{x_1, x_2, \dots, x_m\}$ be a sample drawn independently from a distribution $\mathcal{P}$. Then, with probability at least $1 - \eta$ over

Table 10

F-measure and G-mean of the proposed RoBoSS-MvSVM and baseline models on the AwA datasets.

Dataset	SVM-2K (Farquhar et al., 2005)	MvTSVM (Xie & Sun, 2015)	PSVM-2V (Tang et al., 2018b)	MvNPSVM (Tang et al., 2018a)	MvLDM (Hu et al., 2025)	MvTPMSVM (Quadir & Tanveer, 2024)	RoBoSS-MvSVM (Proposed)
	F-measure G-mean	F-measure G-mean	F-measure G-mean	F-measure G-mean	F-measure G-mean	F-measure G-mean	F-measure G-mean
Chimpanzee vs Giant panda	51.99 ± 0.09 58.79 ± 0.07	70.08 ± 3.42 71.10 ± 1.25	83.92 ± 0.05 84.69 ± 0.04	62.30 ± 0.22 63.58 ± 0.34	84.59 ± 0.04 85.25 ± 0.04	54.77 ± 8.24 59.59 ± 7.17	51.12 ± 0.09 58.11 ± 0.07
Chimpanzee vs Hippopotamus	75.41 ± 0.04 75.80 ± 0.05	75.78 ± 7.38 67.62 ± 7.26	75.75 ± 0.05 74.94 ± 0.04	77.91 ± 0.21 77.30 ± 0.29	77.43 ± 0.05 76.23 ± 0.05	74.11 ± 6.32 75.55 ± 3.23	76.71 ± 0.04 76.69 ± 0.04
Chimpanzee vs Humpback whale	90.21 ± 0.03 89.43 ± 0.04	86.54 ± 4.84 85.97 ± 4.45	90.21 ± 0.03 89.43 ± 0.04	88.91 ± 0.10 86.30 ± 0.10	92.12 ± 0.03 91.34 ± 0.04	90.90 ± 6.31 89.89 ± 2.85	92.18 ± 0.03 91.97 ± 0.04
Chimpanzee vs Leopard	45.21 ± 0.09 53.49 ± 0.07	65.25 ± 3.42 69.05 ± 4.12	64.62 ± 0.04 68.50 ± 0.04	70.47 ± 0.43 69.24 ± 0.03	62.27 ± 0.06 66.84 ± 0.05	60.46 ± 4.26 67.60 ± 4.97	54.03 ± 0.08 68.01 ± 0.07
Chimpanzee vs Persian cat	72.94 ± 0.03 55.16 ± 0.10	78.39 ± 5.12 69.03 ± 11.07	75.71 ± 0.03 61.81 ± 0.11	64.98 ± 0.19 68.33 ± 0.37	87.21 ± 1.35 86.61 ± 0.11	75.93 ± 1.35 73.90 ± 7.55	79.50 ± 0.03 76.50 ± 0.09
Chimpanzee vs Pig	44.39 ± 0.07 53.40 ± 0.05	66.35 ± 3.27 7.32 ± 5.65	72.51 ± 0.03 73.69 ± 0.03	69.46 ± 0.24 67.16 ± 0.34	72.55 ± 0.05 73.72 ± 0.04	67.91 ± 5.48 73.18 ± 4.88	60.47 ± 0.08 69.02 ± 0.06
Chimpanzee vs Raccoon	60.03 ± 0.10 67.70 ± 0.08	66.77 ± 3.42 12.49 ± 3.47	66.61 ± 0.06 69.03 ± 0.05	76.27 ± 0.18 60.68 ± 0.21	65.40 ± 0.06 68.59 ± 0.04	63.98 ± 5.77 71.81 ± 5.89	66.36 ± 0.09 70.11 ± 0.07
Chimpanzee vs Rat	67.89 ± 0.03 61.59 ± 0.01	69.23 ± 4.16 69.85 ± 5.06	72.65 ± 0.02 62.77 ± 0.04	78.22 ± 0.24 72.94 ± 0.44	73.86 ± 0.02 61.49 ± 1.83	73.53 ± 5.18 67.21 ± 1.83	76.87 ± 0.03 65.58 ± 0.03
Chimpanzee vs Seal	75.94 ± 0.04 71.95 ± 0.04	70.25 ± 6.13 73.26 ± 3.80	87.14 ± 0.03 86.95 ± 0.03	87.40 ± 0.17 85.91 ± 0.25	86.41 ± 0.03 86.14 ± 0.03	75.59 ± 3.15 77.81 ± 5.51	80.58 ± 0.03 77.22 ± 0.04
Giant panda vs Hippopotamus	80.32 ± 0.04 74.97 ± 0.03	74.68 ± 8.79 68.25 ± 3.79	75.85 ± 0.04 73.83 ± 0.04	80.80 ± 0.22 76.11 ± 0.36	80.10 ± 0.03 79.50 ± 0.03	76.53 ± 5.00 70.58 ± 4.05	75.54 ± 0.03 71.81 ± 0.04
Giant panda vs Humpback whale	89.35 ± 0.03 87.21 ± 0.04	88.65 ± 5.43 83.78 ± 2.86	90.19 ± 0.04 88.81 ± 0.05	89.87 ± 0.13 88.39 ± 0.15	92.07 ± 0.04 91.32 ± 0.05	89.20 ± 4.68 86.52 ± 3.41	90.19 ± 0.03 88.81 ± 0.04
Giant panda vs Leopard	54.80 ± 0.09 61.74 ± 0.07	62.74 ± 3.42 60.96 ± 0.14	63.71 ± 0.05 65.75 ± 0.04	51.27 ± 0.50 60.50 ± 0.17	63.11 ± 0.05 66.62 ± 0.04	60.03 ± 8.46 67.98 ± 8.62	64.25 ± 0.07 68.13 ± 0.06
Giant panda vs Raccoon	68.55 ± 0.11 68.40 ± 0.08	67.13 ± 8.16 58.46 ± 7.37	52.12 ± 0.07 58.18 ± 0.07	75.19 ± 0.28 77.56 ± 0.49	71.49 ± 0.06 71.30 ± 0.06	57.33 ± 7.46 59.99 ± 4.77	77.70 ± 0.12 71.35 ± 0.10
Giant panda vs Seal	79.45 ± 0.04 75.73 ± 0.05	78.86 ± 6.38 79.64 ± 3.28	79.10 ± 0.04 75.75 ± 0.05	85.35 ± 0.18 83.55 ± 0.29	90.90 ± 0.05 90.65 ± 0.05	82.99 ± 6.68 79.92 ± 6.14	80.09 ± 0.04 76.24 ± 0.06
Giant panda vs Persian cat	77.55 ± 0.03 61.62 ± 0.10	70.42 ± 2.38 68.02 ± 6.03	73.16 ± 0.04 65.47 ± 0.08	59.42 ± 0.14 62.76 ± 0.27	73.33 ± 0.05 55.43 ± 0.08	71.07 ± 3.90 66.84 ± 7.48	80.62 ± 0.03 73.01 ± 0.10
Giant panda vs Pig	70.08 ± 0.13 63.19 ± 0.10	67.97 ± 19.77 60.25 ± 6.78	69.98 ± 0.06 66.62 ± 0.07	62.94 ± 0.28 60.13 ± 0.49	70.85 ± 0.06 66.93 ± 0.07	66.26 ± 6.59 65.18 ± 5.64	74.11 ± 0.12 70.49 ± 0.09
Giant panda vs Rat	70.75 ± 0.04 55.68 ± 0.06	74.09 ± 9.02 55.26 ± 5.81	83.14 ± 0.03 80.13 ± 0.03	73.21 ± 0.16 51.65 ± 0.32	85.82 ± 0.04 84.52 ± 0.06	75.24 ± 7.49 56.57 ± 8.67	68.41 ± 0.05 62.47 ± 0.06
Hippopotamus vs Raccoon	70.41 ± 0.05 70.88 ± 0.04	64.54 ± 3.67 33.26 ± 6.45	63.48 ± 0.05 66.83 ± 0.05	67.04 ± 0.17 68.03 ± 0.27	73.18 ± 0.05 74.13 ± 0.04	62.85 ± 7.40 66.81 ± 4.16	65.69 ± 0.06 68.96 ± 0.05
Hippopotamus vs Humpback whale	77.12 ± 0.06 73.84 ± 0.06	77.31 ± 5.66 72.39 ± 6.38	80.65 ± 0.04 78.27 ± 0.06	81.41 ± 0.19 76.91 ± 0.28	80.78 ± 0.05 78.19 ± 0.07	79.07 ± 7.09 77.65 ± 5.58	78.67 ± 0.06 75.47 ± 0.06
Hippopotamus vs Rat	67.33 ± 0.06 62.23 ± 0.05	60.07 ± 6.93 62.27 ± 3.52	71.95 ± 0.06 60.92 ± 0.08	72.93 ± 0.17 59.60 ± 0.35	71.96 ± 0.05 57.77 ± 0.08	75.34 ± 6.80 64.08 ± 4.97	75.92 ± 0.04 69.15 ± 0.04
Humpback whale vs Rat	76.16 ± 0.06 59.96 ± 0.05	69.98 ± 21.21 65.87 ± 17.63	81.29 ± 0.07 81.28 ± 0.07	81.16 ± 0.15 79.83 ± 0.20	83.80 ± 0.07 83.75 ± 0.07	80.89 ± 6.47 82.03 ± 4.20	85.21 ± 0.05 85.35 ± 0.05
Leopard vs Hippopotamus	78.85 ± 0.04 68.46 ± 0.06	74.95 ± 3.60 60.90 ± 3.09	75.62 ± 0.03 67.95 ± 0.04	79.74 ± 0.19 72.15 ± 0.32	78.52 ± 0.04 76.01 ± 0.04	77.86 ± 6.52 74.29 ± 7.16	76.49 ± 0.05 68.09 ± 0.06
Leopard vs Humpback whale	86.45 ± 0.03 83.23 ± 0.04	81.08 ± 5.69 79.75 ± 3.78	88.31 ± 0.03 86.86 ± 0.04	89.88 ± 0.12 88.51 ± 0.13	91.30 ± 0.04 90.55 ± 0.04	88.10 ± 5.30 86.75 ± 5.03	89.78 ± 0.03 88.14 ± 0.04
Leopard vs Pig	70.82 ± 0.04 55.48 ± 0.06	72.38 ± 3.89 68.69 ± 3.01	71.84 ± 0.04 58.05 ± 0.04	43.37 ± 0.22 55.94 ± 0.43	75.74 ± 0.03 70.44 ± 0.03	65.54 ± 6.18 74.90 ± 4.41	71.77 ± 0.04 56.40 ± 0.06
Leopard vs Raccoon	68.49 ± 0.04 52.14 ± 0.07	71.21 ± 4.15 50.24 ± 4.89	67.70 ± 0.03 59.30 ± 0.05	41.50 ± 0.23 53.90 ± 0.45	66.30 ± 0.03 58.85 ± 0.06	59.44 ± 5.69 67.21 ± 4.06	68.31 ± 0.03 53.87 ± 0.07
Leopard vs Rat	70.05 ± 0.05 46.01 ± 0.09	67.79 ± 7.72 22.31 ± 6.73	69.31 ± 0.06 47.02 ± 0.08	71.20 ± 0.11 45.19 ± 0.22	73.76 ± 0.05 56.85 ± 0.07	68.53 ± 4.61 55.24 ± 5.43	68.84 ± 0.05 55.67 ± 0.09
Persian cat vs Humpback whale	84.84 ± 0.03 85.00 ± 0.03	85.39 ± 4.49 70.01 ± 9.53	72.11 ± 0.03 74.23 ± 0.04	86.87 ± 0.12 86.75 ± 0.12	81.14 ± 0.04 81.45 ± 0.05	77.12 ± 4.90 78.82 ± 3.47	90.64 ± 0.03 89.41 ± 0.03
Persian cat vs Raccoon	35.74 ± 0.08 47.67 ± 0.07	71.45 ± 3.42 70.15 ± 1.47	57.49 ± 0.09 57.08 ± 0.06	71.28 ± 0.52 58.12 ± 0.06	37.94 ± 0.06 48.08 ± 0.05	41.03 ± 11.95 50.38 ± 7.20	71.10 ± 0.08 73.40 ± 0.07
Persian cat vs Hippopotamus	53.88 ± 0.10 60.52 ± 0.08	74.07 ± 3.42 70.51 ± 6.24	55.18 ± 0.10 61.74 ± 0.08	75.67 ± 0.42 77.84 ± 0.09	54.22 ± 0.09 61.00 ± 0.07	76.72 ± 6.01 80.50 ± 5.82	83.38 ± 0.10 81.97 ± 0.08
Persian cat vs Leopard	40.11 ± 0.06 50.07 ± 0.05	71.32 ± 3.42 62.29 ± 4.25	55.84 ± 0.08 62.21 ± 0.06	74.39 ± 0.55 67.70 ± 0.54	59.06 ± 0.06 64.70 ± 0.05	47.12 ± 4.05 55.43 ± 4.27	54.77 ± 0.07 68.77 ± 0.06
Persian cat vs Pig	40.54 ± 0.06 50.39 ± 0.04	70.25 ± 3.42 61.00 ± 0.56	41.23 ± 0.06 50.91 ± 0.04	64.37 ± 0.51 64.35 ± 0.09	39.48 ± 0.07 49.66 ± 0.06	43.72 ± 5.25 52.69 ± 4.44	53.03 ± 0.06 68.42 ± 0.04
Persian cat vs Rat	46.76 ± 0.03 54.35 ± 0.03	56.70 ± 3.38 55.73 ± 11.78	47.18 ± 0.04 54.60 ± 0.03	37.26 ± 0.33 52.07 ± 0.43	46.88 ± 0.08 54.84 ± 0.06	49.85 ± 8.08 56.37 ± 4.03	50.70 ± 0.04 57.00 ± 0.03
Pig vs Hippopotamus	66.12 ± 0.05 64.57 ± 0.06	61.13 ± 8.17 65.58 ± 9.90	67.46 ± 0.06 67.32 ± 0.06	72.18 ± 0.24 65.61 ± 0.46	66.52 ± 0.07 66.57 ± 0.06	71.15 ± 4.52 67.00 ± 6.74	69.48 ± 0.04 63.07 ± 0.06
Pig vs Raccoon	52.64 ± 0.08 56.74 ± 0.05	64.86 ± 6.27 45.32 ± 25.46	50.69 ± 0.02 55.12 ± 0.02	63.60 ± 0.25 59.53 ± 0.44	60.57 ± 0.06 61.04 ± 0.04	45.81 ± 6.24 51.08 ± 4.77	71.83 ± 0.09 65.12 ± 0.05
Pig vs Humpback whale	87.34 ± 0.04 85.80 ± 0.06	84.81 ± 7.35 84.79 ± 4.68	88.11 ± 0.02 86.98 ± 0.03	83.83 ± 0.13 80.77 ± 0.15	87.90 ± 0.02 86.36 ± 0.03	86.67 ± 7.34 84.88 ± 5.30	87.28 ± 0.04 85.78 ± 0.06

Table 10

F-measure and G-mean of the proposed RoBoSS-MvSVM and baseline models on the AwA datasets.

Dataset	SVM-2K (Farquhar et al., 2005)	MvTSVM (Xie & Sun, 2015)	PSVM-2V (Tang et al., 2018b)	MvNPSVM (Tang et al., 2018a)	MvLDM (Hu et al., 2025)	MvTPMSVM (Quadri & Tanveer, 2024)	RoBoSS-MvSVM (Proposed)
	F-measure	F-measure	F-measure	F-measure	F-measure	F-measure	F-measure
	G-mean	G-mean	G-mean	G-mean	G-mean	G-mean	G-mean
Pig vs Rat	68.29 ± 0.03	70.84 ± 8.19	67.53 ± 0.04	49.27 ± 0.13	68.52 ± 0.04	61.40 ± 7.10	69.61 ± 0.03
Raccoon vs Humpback whale	48.53 ± 0.07	44.40 ± 4.44	43.12 ± 0.09	51.78 ± 0.30	38.84 ± 0.07	54.17 ± 9.11	47.91 ± 0.08
	87.33 ± 0.02	88.03 ± 5.25	87.08 ± 0.02	82.69 ± 0.13	87.49 ± 0.03	87.59 ± 4.82	89.03 ± 0.02
	85.79 ± 0.03	84.61 ± 2.88	85.58 ± 0.03	83.28 ± 0.15	86.60 ± 0.04	85.62 ± 3.99	87.73 ± 0.03
Raccoon vs Rat	68.43 ± 0.05	67.04 ± 5.45	68.63 ± 0.05	71.20 ± 0.14	70.19 ± 0.05	68.22 ± 5.02	71.06 ± 0.05
	47.00 ± 0.06	30.42 ± 4.36	50.62 ± 0.08	50.94 ± 0.30	48.01 ± 0.09	57.28 ± 5.39	46.74 ± 0.08
Seal vs Hippopotamus	58.63 ± 0.07	59.20 ± 3.42	54.07 ± 0.07	40.67 ± 0.42	49.41 ± 0.12	55.33 ± 4.88	66.96 ± 0.07
	58.69 ± 0.07	27.78 ± 4.15	56.98 ± 0.06	52.54 ± 0.49	54.52 ± 0.10	55.99 ± 4.42	54.81 ± 0.07
Seal vs Humpback whale	76.68 ± 0.06	67.10 ± 10.01	80.64 ± 0.07	75.19 ± 0.25	77.05 ± 0.06	75.35 ± 9.64	77.76 ± 0.06
	74.61 ± 0.05	66.80 ± 6.85	78.74 ± 0.06	68.91 ± 0.37	76.40 ± 0.06	74.24 ± 6.88	75.12 ± 0.06
Seal vs Leopard	58.91 ± 0.06	66.58 ± 3.42	73.11 ± 0.06	70.05 ± 0.06	78.03 ± 0.08	73.71 ± 5.14	74.33 ± 0.07
	64.62 ± 0.05	7.24 ± 6.21	75.70 ± 0.05	73.42 ± 0.04	75.36 ± 0.07	76.06 ± 5.56	76.66 ± 0.06
Seal vs Persian cat	76.93 ± 0.04	70.68 ± 4.05	77.20 ± 0.06	77.85 ± 0.21	77.59 ± 0.06	76.72 ± 3.77	79.41 ± 0.04
	67.16 ± 0.06	71.48 ± 5.96	70.44 ± 0.10	67.02 ± 0.39	70.49 ± 0.10	70.39 ± 6.88	77.34 ± 0.06
Seal vs Pig	62.38 ± 0.10	66.78 ± 3.42	64.36 ± 0.07	63.15 ± 0.21	68.69 ± 0.09	67.32 ± 7.32	65.86 ± 0.10
	67.37 ± 0.07	6.98 ± 4.28	67.76 ± 0.05	64.88 ± 0.26	71.39 ± 0.07	68.17 ± 3.09	68.86 ± 0.08
Seal vs Raccoon	69.51 ± 0.06	66.89 ± 3.42	70.85 ± 0.08	79.69 ± 0.11	77.15 ± 0.08	69.31 ± 9.85	77.93 ± 0.06
	72.91 ± 0.05	9.94 ± 2.87	73.50 ± 0.07	70.70 ± 0.09	78.46 ± 0.07	71.90 ± 8.15	79.00 ± 0.05
Seal vs Rat	69.73 ± 0.03	58.77 ± 5.69	69.72 ± 0.03	70.65 ± 0.19	71.74 ± 0.04	57.37 ± 5.58	71.19 ± 0.03
	66.44 ± 0.01	64.35 ± 9.11	66.29 ± 0.03	62.69 ± 0.40	68.50 ± 0.04	62.01 ± 4.66	68.49 ± 0.02
Average	(67.67, 64.81)	(70.99, 57.23)	(71.00, 68.70)	(70.94, 68.25)	(72.99, 70.92)	(69.00, 69.16)	(73.44, 70.94)

random choice of samples of size m , the following inequality

$$\mathbb{E}_{\mathcal{P}}[h(x)] \leq \hat{\mathbb{E}}_{\mathcal{T}}[h(x)] + \mathfrak{R}_m(\mathcal{H}) + 3\sqrt{\frac{\ln(2/\eta)}{2m}}. \quad (21)$$

satisfies for all $h \in \mathcal{H}$.

Lemma 2. Let $K : X \times X \rightarrow \mathbb{R}$ be a kernel function, and $\mathcal{T} = \{x_i\}_{i=1}^m$ denote the training set from X . Then, for the class function $\mathcal{H}_B = \{h|h : x \rightarrow \langle u, \psi(x) \rangle : \|u\| \leq B\}$ where $\psi(\cdot)$ denotes the feature map associated with the kernel K , the empirical Rademacher complexity of $\mathcal{H}_B$ satisfies

$$\mathfrak{R}_m(\mathcal{H}_B) = \frac{2B}{m} \sqrt{\sum_{i=1}^m K(x_i, x_i)}. \quad (22)$$

Lemma 3. Let $\mathcal{A} : \mathbb{R} \rightarrow \mathbb{R}$ be a Lipschitz continuous function with Lipschitz constant $L > 0$, and $\mathcal{A}(0) = 0$. Then, the Rademacher complexity of composite class $\mathcal{A} \circ \mathcal{H}$ satisfies

$$\mathfrak{R}_m(\mathcal{A} \circ \mathcal{H}) \leq 2L \mathfrak{R}_m(\mathcal{H}). \quad (23)$$

Theorem 2. Fix $M \in \mathbb{R}^+$ and $\eta \in (0, 1)$, and consider a two-view training set $\mathcal{T} = \{(x_i, y_i)\}_{i=1}^m$ that contains m samples drawn independently from a distribution $\mathcal{P}$, where each $x_i = (x_i^{[1]}, \eta x_i^{[2]})^T$ and $y_i \in \{-1, +1\}$. Define the function classes as follows: $\mathcal{H} = \{h|h : x \rightarrow u^T \hat{\psi}(x) : \|u\| \leq M\}$ and $\hat{\mathcal{H}} = \{\hat{h}|\hat{h} : (x, y) \rightarrow yh(x), h(x) \in \mathcal{H}\}$ where $\hat{\psi}(x) = \begin{pmatrix} \psi_1(x^{[1]}) \\ \eta \psi_2(x^{[2]}) \end{pmatrix}$, $u = \begin{pmatrix} u_1 \\ u_2 \end{pmatrix}$ and $h(x) = u_1^T \psi_1(x^{[1]}) + \eta u_2^T \psi_2(x^{[2]}) = u^T \hat{\psi}(x) \in \mathcal{H}$. Then, with a probability of at least $1 - \eta$ over the training set $\mathcal{T}$, any $h(x) \in \mathcal{H}$ satisfies

$$\mathbb{P}_{\mathcal{P}}[yh(x) \leq 0] \leq \frac{1}{m(1+\eta)} \sum_{i=1}^m (\xi_i^{[1]} + \eta \xi_i^{[2]}) + 3\sqrt{\frac{\ln(2/\eta)}{2m}} + \frac{4M}{m(1+\eta)} \sqrt{\sum_{i=1}^m (K_1(x_i^{[1]}, x_i^{[1]}) + \eta^2 K_2(x_i^{[2]}, x_i^{[2]}))}. \quad (24)$$

Proof. Consider a loss function $\phi : \mathbb{R} \rightarrow [0, 1]$ defined by

$$\phi(x) = \begin{cases} 1, & \text{if } x < 0, \\ 1 - \frac{x}{1+\eta}, & \text{if } 0 \leq x \leq 1+\eta, \\ 0, & \text{otherwise.} \end{cases} \quad (25)$$

From this definition of loss function, we have

$$\mathbb{P}_{\mathcal{P}}[yh(x) \leq 0] \leq \mathbb{E}_{\mathcal{P}}[\phi(\hat{h}(x, y))]. \quad (26)$$

Using Lemma 1, we get

$$\mathbb{E}_{\mathcal{P}}[\phi(\hat{h}(x, y)) - 1] \leq \hat{\mathbb{E}}_{\mathcal{T}}[\phi(\hat{h}(x, y)) - 1] + \mathfrak{R}_m((\phi - 1) \circ \hat{\mathcal{H}}) + 3\sqrt{\frac{\ln(2/\eta)}{2m}}. \quad (27)$$

Therefore,

$$\mathbb{E}_{\mathcal{P}}[\phi(\hat{h}(x, y))] \leq \hat{\mathbb{E}}_{\mathcal{T}}[\phi(\hat{h}(x, y))] + \mathfrak{R}_m((\phi - 1) \circ \hat{\mathcal{H}}) + 3\sqrt{\frac{\ln(2/\eta)}{2m}}. \quad (28)$$

Now, one can easily observe that

$$\phi(yh(x)) \leq \frac{1}{1+\eta} [1 + \eta - yh(x)]_+. \quad (29)$$

Then, by taking the expectation both the sides and applying the first and second constraints from the RoBoSS-MvSVM (9), we obtain

$$\begin{aligned} \mathbb{E}_{\mathcal{T}}[\phi(\hat{h}(x, y))] &\leq \frac{1}{m(1+\eta)} \sum_{i=1}^m [1 + \eta - y_i h(x_i)]_+ \\ &= \frac{1}{m(1+\eta)} \sum_{i=1}^m [1 + \eta - y_i (u_1^T \psi_1(x_i^{[1]}) + \eta u_2^T \psi_2(x_i^{[2]}))]_+ \\ &\leq \frac{1}{m(1+\eta)} \sum_{i=1}^m \left\{ [1 - y_i u_1^T \psi_1(x_i^{[1]})]_+ + \eta [1 - y_i u_2^T \psi_2(x_i^{[2]})]_+ \right\} \\ &\leq \frac{1}{m(1+\eta)} (\xi_i^{[1]} + \eta \xi_i^{[2]}). \end{aligned} \quad (30)$$

Now, since $\phi(x)$ is a Lipschitz continuous function with constant $\frac{1}{1+\eta}$ and satisfies $\phi(0) = 0$, we can use Lemma 3 to obtain the following inequality:

$$\mathfrak{R}_m((\phi - 1) \circ \hat{\mathcal{H}}) \leq \frac{2}{1+\eta} \mathfrak{R}_m(\hat{\mathcal{H}}). \quad (31)$$

Further, using the definition of empirical Rademacher complexity (Definition 1), we have

$$\begin{aligned} \mathfrak{R}_m(\hat{\mathcal{H}}) &= \mathbb{E}_{\epsilon} \left[\sup_{h \in \hat{\mathcal{H}}} \left| \frac{2}{m} \sum_{i=1}^m \epsilon_i \hat{h}(x_i, y_i) \right| \right] \\ &= \mathbb{E}_{\epsilon} \left[\sup_{h \in \hat{\mathcal{H}}} \left| \frac{2}{m} \sum_{i=1}^m \epsilon_i y_i h(x_i) \right| \right] \end{aligned}$$

Table 11
Comparison of RoBoSS-MvSVM with existing methods with Gaussian noise.

Dataset	Noise	SVM-2K (Farquhar et al., 2005) Accuracy (%)	MvTSVM (Xie & Sun, 2015) Accuracy (%)	PSVM-2V (Tang et al., 2018b) Accuracy (%)	MvNPSVM (Tang et al., 2018a) Accuracy (%)	MvLDM (Hu et al., 2025) Accuracy (%)	MvTPMSVM (Quadri & Tanveer, 2024) Accuracy (%)	RoBoSS-MvSVM (Proposed) Accuracy (%)
appendicitis	5%	80.22	84.85	82.12	78.82	84.94	84.98	85.89
	10%	81.17	82.80	83.03	78.02	83.07	84.03	83.98
	15%	80.22	81.85	79.26	76.20	81.54	81.75	82.08
	20%	81.17	80.85	80.26	76.20	83.98	82.00	83.07
Average aus		80.69	82.59	81.17	77.31	83.38	83.19	83.76
	5%	82.32	41.45	82.32	83.64	85.80	83.62	82.46
	10%	82.75	49.71	81.59	81.25	85.65	84.06	82.46
	15%	82.75	41.88	82.46	80.20	85.65	84.20	82.75
Average ecolil		81.16	44.64	79.86	76.80	85.65	83.77	82.17
	5%	82.25	44.42	81.56	80.47	85.69	83.91	82.46
	10%	89.89	80.95	87.80	82.85	86.60	89.29	89.89
	15%	88.54	82.73	87.80	81.24	88.10	89.28	89.89
Average glass1		87.61	78.56	88.69	80.12	86.01	90.18	90.48
	5%	89.57	82.12	88.69	82.78	87.50	88.41	89.59
	10%	88.90	81.09	88.25	81.75	87.05	89.29	89.96
	15%	71.44	68.65	72.53	63.81	72.15	73.42	73.75
Average hepatitis		73.47	68.65	72.54	62.47	74.27	74.14	74.24
	5%	72.48	68.65	74.59	61.12	74.65	73.60	74.70
	10%	66.30	65.85	76.15	58.00	75.22	76.16	74.71
	15%	70.92	67.95	73.95	61.35	74.08	74.33	74.35
Average monk-2		80.00	82.58	79.35	77.50	81.29	81.29	81.94
	5%	80.00	80.65	80.65	74.80	81.29	81.29	82.58
	10%	80.00	81.94	81.29	73.54	80.00	81.29	81.94
	15%	80.00	80.00	78.71	71.50	81.94	83.23	81.94
Average newthyroid		80.00	81.29	80.00	74.34	81.13	81.77	82.10
	5%	81.01	58.33	84.01	72.86	88.66	89.60	83.33
	10%	81.47	56.94	83.33	70.20	88.67	89.14	84.49
	15%	81.69	58.34	84.71	71.54	89.36	88.22	86.57
Average sonar		82.40	56.95	84.26	67.00	88.67	89.37	84.49
	5%	81.64	57.64	84.08	70.40	88.84	89.08	84.72
	10%	90.70	95.81	90.70	72.70	93.95	96.24	96.26
	15%	90.70	94.88	90.70	71.76	94.88	95.81	96.72
Average vote		90.23	94.42	90.70	71.76	92.56	95.81	96.26
	5%	90.70	92.56	89.30	71.76	94.42	94.88	91.40
	10%	90.58	94.42	90.35	72.00	93.95	95.70	95.16
	15%	84.10	66.81	85.09	52.24	85.56	86.99	84.59
Average WPBC		82.02	68.73	85.56	52.24	86.53	87.47	82.20
	5%	84.10	67.28	85.56	51.28	87.49	87.94	84.59
	10%	85.54	70.19	84.12	50.00	84.61	86.99	82.20
	15%	83.94	68.25	85.08	51.44	86.05	87.35	83.39
Average		89.43	68.74	89.89	88.02	91.03	91.03	92.41
	5%	89.43	68.97	89.89	87.35	91.26	90.57	91.72
	10%	89.20	67.82	89.89	89.34	91.49	91.49	91.95
	15%	89.20	69.43	89.89	85.32	91.72	90.57	92.64
Average		89.31	68.74	89.89	87.51	91.38	90.92	92.18
	5%	76.72	75.73	73.26	76.40	75.77	76.73	76.76
	10%	76.78	76.24	72.28	74.00	75.77	77.77	76.74
	15%	76.78	73.98	73.27	72.45	72.33	74.45	77.77
Average		77.28	71.41	71.74	70.50	71.25	72.21	76.71
	20%	76.89	74.34	72.64	73.34	73.78	75.29	76.99

$$\begin{aligned}
&= \mathbb{E}_\epsilon \left[\sup_{h \in \mathcal{H}} \left| \frac{2}{m} \sum_{i=1}^m \epsilon_i h(x_i) \right| \right] \\
&= \hat{\mathfrak{R}}_m(\mathcal{H}).
\end{aligned} \tag{32}$$

From Lemma 2, we get

$$\begin{aligned}
\hat{\mathfrak{R}}_m((\phi - 1) \circ \hat{H}) &\leq \frac{2}{1 + \eta} \hat{\mathfrak{R}}_m(\hat{H}) \\
&= \frac{4M}{m(1 + \eta)} \sqrt{\sum_{i=1}^m K(x_i, x_i)} \\
&= \frac{4M}{m(1 + \eta)} \sqrt{\sum_{i=1}^m (K_1(x_i^{[1]}, x_i^{[1]}) + \eta^2 K_2(x_i^{[2]}, x_i^{[2]}))}.
\end{aligned} \tag{33}$$

Hence, combining the results of expressions (26), (28), (30), and (33), we yield the final error bound for RoBoSS-MvSVM. This completes the proof. $\square$

Here, the classification error function $h(x) \in \mathcal{H}$ is constructed based on the integrated decision function employed in RoBoSS-MvSVM (9). We obtained a margin-based upper bound on the misclassification probability by combining the empirical Rademacher complexity of $\mathcal{H}$ with the empirical expectation of $\hat{\mathcal{H}}$. As the number of samples (m) increases, RoBoSS-MvSVM exhibits a tighter and more stable bound over the generalization error. Consequently, a reduction in training error directly results in a lower generalization error, providing theoretical evidence that the proposed RoBoSS-MvSVM can yield more reliable and robust performance in real-world applications.

5. Experiment analysis

This section presents a detailed empirical assessment of the proposed RoBoSS-MvSVM model through extensive experiments on a wide range of datasets. The experiments are conducted on 3 synthetic datasets, 39 benchmark datasets from the UCI (Dua & Graff, 2017) and KEEL (Derrac et al., 2015) repositories, and 45 binary classification datasets

Table 12
Comparison of RoBoSS-MvSVM with existing methods with label noise.

Dataset	Noise	SVM-2K (Farquhar et al., 2005) Accuracy (%)	MvTSVM (Xie & Sun, 2015) Accuracy (%)	PSVM-2V (Tang et al., 2018b) Accuracy (%)	MvNPSVM (Tang et al., 2018a) Accuracy (%)	MvLDM (Hu et al., 2025) Accuracy (%)	MvTPMSVM (Quadir & Tanveer, 2024) Accuracy (%)	RoBoSS-MvSVM (Proposed) Accuracy (%)
appendicitis	5%	82.98	81.08	78.31	77.24	82.94	83.02	83.07
	10%	80.22	82.03	78.35	77.43	83.89	84.03	84.03
	15%	72.12	70.13	73.64	73.23	74.07	75.30	73.64
	20%	63.98	68.23	70.87	72.95	72.89	70.26	73.59
Average aus		74.83	75.37	75.29	75.21	78.45	78.15	78.58
	5%	78.99	67.39	79.57	81.88	85.51	83.77	84.17
	10%	78.99	60.14	78.70	79.83	84.35	82.90	84.21
	15%	77.10	51.01	77.39	77.76	81.51	80.72	82.33
Average ecolil		72.32	48.12	72.03	75.72	80.80	77.54	81.93
	5%	76.85	56.67	76.92	78.80	83.04	81.23	83.16
	10%	90.09	61.31	87.80	79.48	86.61	87.81	90.19
	15%	88.60	57.21	86.91	76.27	85.69	84.53	89.59
Average glass1		87.40	47.30	86.91	73.31	82.49	82.76	87.81
	5%	84.51	67.22	85.03	77.47	80.22	80.07	86.02
	10%	87.65	58.26	86.66	76.63	83.75	83.79	88.40
	15%	76.18	60.79	74.33	64.64	76.63	77.57	71.45
Average hepatitis		74.11	63.73	71.47	61.53	75.69	75.22	71.46
	5%	70.72	59.79	66.16	57.52	73.82	68.22	65.36
	10%	64.92	57.48	60.55	51.39	69.60	68.23	63.95
	15%	71.48	60.45	68.13	58.77	73.94	72.31	68.06
Average monk-2		79.35	65.16	77.42	72.66	76.77	78.71	80.00
	5%	78.06	57.42	75.48	68.24	76.13	78.06	79.94
	10%	70.65	55.48	67.10	62.57	68.39	72.26	76.77
	15%	68.71	51.29	66.32	62.90	67.68	71.48	75.71
Average newthyroid		74.19	57.34	71.58	66.59	72.24	75.13	78.10
	5%	79.85	58.56	72.91	69.70	88.20	84.74	81.71
	10%	79.40	59.50	71.77	65.58	85.43	78.72	81.48
	15%	77.78	52.99	67.60	62.21	82.65	75.97	75.93
Average sonar		80.78	51.41	66.21	57.24	84.95	69.70	74.54
	5%	79.45	55.62	69.62	63.68	85.31	77.29	78.41
	10%	89.77	74.19	90.23	62.32	90.70	93.02	93.49
	15%	89.77	75.35	90.23	62.00	90.23	87.44	92.09
Average vote		86.51	70.70	86.51	57.50	86.77	73.49	87.44
	5%	89.30	77.67	88.84	55.83	86.98	45.12	89.77
	10%	88.84	74.48	88.95	59.41	88.67	74.77	90.70
	15%	84.11	58.13	87.48	80.87	86.99	86.03	82.17
Average WPBC		73.17	56.24	79.30	77.24	79.53	73.18	79.80
	5%	75.03	50.44	76.95	69.70	75.93	77.93	71.17
	10%	71.01	50.46	70.05	67.66	71.11	71.89	72.10
	15%	75.83	53.82	78.45	73.87	78.39	77.26	76.31
Average		88.51	68.28	89.66	86.99	90.34	86.90	92.64
	5%	87.36	66.44	89.20	83.79	87.36	78.39	91.72
	10%	87.82	61.84	89.66	82.94	83.22	69.66	91.26
	15%	87.13	66.67	88.74	77.35	80.23	62.76	90.11
Average		87.70	65.80	89.31	82.77	85.29	74.43	91.44
	5%	76.78	71.73	73.78	76.23	77.78	78.26	75.72
	10%	72.29	74.74	69.74	71.91	74.27	73.26	75.76
	15%	71.78	66.12	71.27	70.68	70.31	73.76	71.73
Average		71.78	71.74	64.67	69.90	72.78	70.22	73.21
	20%	73.16	71.08	69.87	72.18	73.79	73.87	74.10

Table 13
Ablation studies.

Dataset → Method ↓	Breast cancer Accuracy (%)	Divorce Accuracy (%)	Haberman Accuracy (%)	Heart Accuracy (%)	Ionosphere Accuracy (%)
Without consensus	96.31 ± 1.68	98.82 ± 1.61	73.52 ± 3.22	80.74 ± 5.34	88.68 ± 1.38
Without complementarity	96.49 ± 1.52	97.65 ± 2.46	73.52 ± 3.22	82.22 ± 6.23	88.89 ± 4.67
Without RoBoSS	67.66 ± 4.35	92.94 ± 6.77	54.18 ± 4.65	79.26 ± 3.33	88.04 ± 1.90
RoBoSS-MvSVM	97.36 ± 1.68	98.82 ± 1.61	73.85 ± 2.12	82.96 ± 6.33	90.60 ± 3.83

obtained from the AwA dataset (Lampert et al., 2013). The multi-view feature adopted for the UCI/KEEL and AwA datasets is summarized in Table 3. Additionally, the performance of the proposed approach is compared against several existing MvL algorithms, namely SVM-2K (Farquhar et al., 2005), MvTSVM (Xie & Sun, 2015), PSVM-2V (Tang et al., 2018b), MvNPSVM (Tang et al., 2018a), MvLDM (Hu et al., 2025), and MvTPMSVM (Quadir & Tanveer, 2024).

5.1. Experimental setup

All the computational experiments are performed on a high-performance computing cluster equipped with Intel Xeon Platinum 8268 processors, using MATLAB R2021b. To ensure the reliability of the results, a five-fold cross-validation strategy is employed, where each dataset is divided into five equal subsets. In each fold, the model is

trained on four subsets and tested on the remaining one, with this process repeated five times. Consequently, model performance is evaluated based on the average accuracy across the five folds. We used a grid search method to obtain the optimal hyperparameters selected based on the highest average accuracy achieved across the folds. The

Gaussian function ($K(x_1, x_2) = e^{-\frac{\|x_1 - x_2\|^2}{2\sigma^2}}$) is used as the kernel function across all classifiers in this study. The kernel parameter σ is selected from the set $\{2^{-4}, 2^{-3}, \dots, 2^4\}$. The regularization parameters are set as $C_1 = C_2$, with C_1 chosen from $\{10^{-2}, 10^{-1}, \dots, 10^2\}$. The parameter D is also selected from the same set. For the proposed model employing the RoBoSS loss function, the additional parameters a and λ are chosen from the set $\{0, 0.1, \dots, 2\}$, while the parameter η is fixed at 1 for experimental purposes. For the baseline models MvTSVM and MvNPSVM, the parameters are set as $C_1 = C_2 = C_3 = C_4$, selected from $\{10^{-2}, 10^{-1}, \dots, 10^2\}$. In MvNPSVM, the parameter D is selected from the set $\{0.05, 0.5, 1, 3, 5\}$. For MvLDM, the parameters are configured as $C_1 = C_2$ and $v_1 = v_2$, each selected from $\{10^{-2}, 10^{-1}, \dots, 10^2\}$. In PSVM-2V and SVM-2K, the parameters are set as $C_1 = C_2 = D$, with values drawn from $\{10^{-2}, 10^{-1}, \dots, 10^2\}$. For MvTPMSVM, the parameters C_1 and C_2 are independently selected from the set $\{10^{-2}, 10^{-1}, \dots, 10^2\}$. The effectiveness of the proposed model is assessed using standard evaluation metrics such as accuracy, sensitivity, specificity, F-measure, and G-mean. Mathematically,

$$\text{Accuracy} = \frac{\text{TN} + \text{TP}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}},$$

$$\text{Sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}},$$

$$\text{F-measure} = \frac{2\text{TP}}{2\text{TP} + \text{FN} + \text{FP}}, \quad \text{G-mean} = \sqrt{\text{sensitivity} \times \text{specificity}}.$$

5.2. Experiments on synthetic datasets

In this subsection, we utilize the three synthetic two-view datasets (Wang et al., 2023) to demonstrate the view-consistency handling ability of the proposed RoBoSS-MvSVM. The synthetic 1 dataset features a double vortex and concentric circle. The Synthetic 2 dataset exhibits a checkboard pattern and a Gaussian cloud, and the Synthetic 3 dataset is composed of a double moon and a double line structure. These distributions are visualized in Fig. 4. The sample sizes of the three synthetic datasets are 800, 1200, and 2000, respectively.

Table 4 presents the classification accuracies along with the standard deviation of RoBoSS-MvSVM and baseline methods using a Gaussian kernel. The proposed RoBoSS-MvSVM achieves the highest overall average accuracy of 97%, outperforming all baseline approaches. Its strong and consistent performance across all synthetic datasets highlights the robustness and effectiveness of the proposed method.

5.3. Experiments on UCI and KEEL datasets

To assess the generalization performance of the proposed RoBoSS-MvSVM in comparison with existing approaches, the experiments are conducted on 39 benchmark datasets sourced from the UCI and KEEL repositories. These datasets span various application domains, including healthcare, finance, bioinformatics, and social sciences. Among them, 29 datasets comprise up to 1000 training samples while the remaining 10 datasets contain between 1000 and 7000 samples, enabling evaluation across both small and large-sized datasets. Since the datasets obtained from the UCI and KEEL repositories are not inherently multi-view, we construct a two-view representation for each dataset. The original feature set is designated as view 1, while view 2 is generated by applying principal component analysis (PCA) and retaining the top 95% of the cumulative variance from the original data (Wang et al., 2023). The accuracy results along with standard deviation, run time, and optimal hyperparameters for the proposed RoBoSS-MvSVM and existing methods are presented in Table 5. The highest accuracy for each dataset is

highlighted in bold. The proposed method achieves the highest accuracy on 30 out of 39 datasets, with an overall average accuracy of 85.92%, surpassing all existing classifiers. In comparison, the average accuracies obtained by MvTPMSVM, MvLDM, PSVM-2V, SVM-2K, MvNPSVM, and MvTSVM are 85.29%, 85.00%, 84.46%, 83.27%, 78.81%, and 75.27%, respectively. In addition, the proposed method achieves lower computational time than existing techniques, particularly on larger datasets, due to its iterative optimization strategy. These results clearly highlight the effectiveness and superiority of the proposed approach in handling multi-view classification problems.

To ensure a more neutral assessment of the classifiers, we also assign rankings for each dataset, as accuracy metrics could be biased by exceptional performance on a single dataset, potentially compensating for low accuracy on other datasets. The classifiers are ranked on each dataset based on their performance, where the classifier with the highest accuracy receives a rank of 1, the second-best receives a rank of 2, and so on. Once again, the proposed method achieved the lowest average rank of 1.81, demonstrating its superiority over the other classifiers. The average ranks of the existing methods are 2.99, 3.29, 4.06, 4.72, 5.13, and 6.00 for MvTPMSVM, MvLDM, PSVM-2V, SVM-2K, MvNPSVM, and MvTSVM, respectively. Furthermore, we also evaluated the classifier's performance using additional metrics such as sensitivity, specificity, F-measure, and G-mean. As shown in Tables 6 and 7, the proposed method achieved the highest average sensitivity (71.66%) and G-mean (71.12%) values, indicating its strong ability to correctly classify positive samples while maintaining balanced performance across both classes. While the proposed method ranks second in terms of average specificity (81.11%) and F-measure (71.15%) after MvLDM (83.02%) and PSVM-2V (71.77%), respectively, it still exhibited the most consistent overall performance. These results are further illustrated in Fig. 5, where the bar graph of the average values of sensitivity, specificity, F-measure, and G-mean clearly highlights the superior and more consistent performance of the proposed method across all evaluation metrics.

5.4. Experiments on AWA datasets

This section provides a comparative evaluation of the proposed RoBoSS-MvSVM method against existing approaches utilizing the AWA dataset. The AWA dataset comprises 30475 images spanning 50 distinct animal categories, each associated with six pre-extracted feature representations. Similar to other studies (Hu et al., 2025; Tang et al., 2018b), we focus on a subset of 10 test classes: chimpanzee, giant panda, seal, Persian cat, leopard, pig, hippopotamus, raccoon, humpback whale, and rat, resulting in a total of 6180 images. We use the 2000-dimensional L_1 -normalized SURF features as the first view, and the 252-dimensional HOG descriptors as the second view. A one-versus-one classification strategy is adopted to handle all possible pairs among the selected 10 classes, resulting in 45 binary classification tasks. For each class, 200 samples are randomly selected for training purposes. The performance of the proposed method and existing approaches is evaluated based on the standard metrics. The Table 8 reports the accuracy values along with the standard deviation, run time, and optimal parameters for each method. The proposed method achieves the highest average accuracy of 73.85%, outperforming all baseline classifiers. In comparison, the average accuracy scores of MvLDM, MvTPMSVM, PSVM-2V, MvNPSVM, SVM-2K, and MvTSVM are 73.57%, 71.23%, 71.17%, 70.14%, 68.67%, and 65.44%, respectively.

We also compute the average rank of each classifier across the 45 datasets to provide a more balanced comparison. A lower average rank indicates more consistent top-tier performance. The proposed method obtains the lowest average rank of 1.99, followed by MvLDM (2.74), MvTPMSVM (3.79), PSVM-2V (4.11), MvNPSVM (4.62), SVM-2K (5.14), and MvTSVM (5.60). Furthermore, the performance of the proposed method is evaluated using sensitivity, F-score, and G-mean, as presented in Tables 9 and 10. The results demonstrate that the proposed method achieves the highest average F-measure (73.44%)

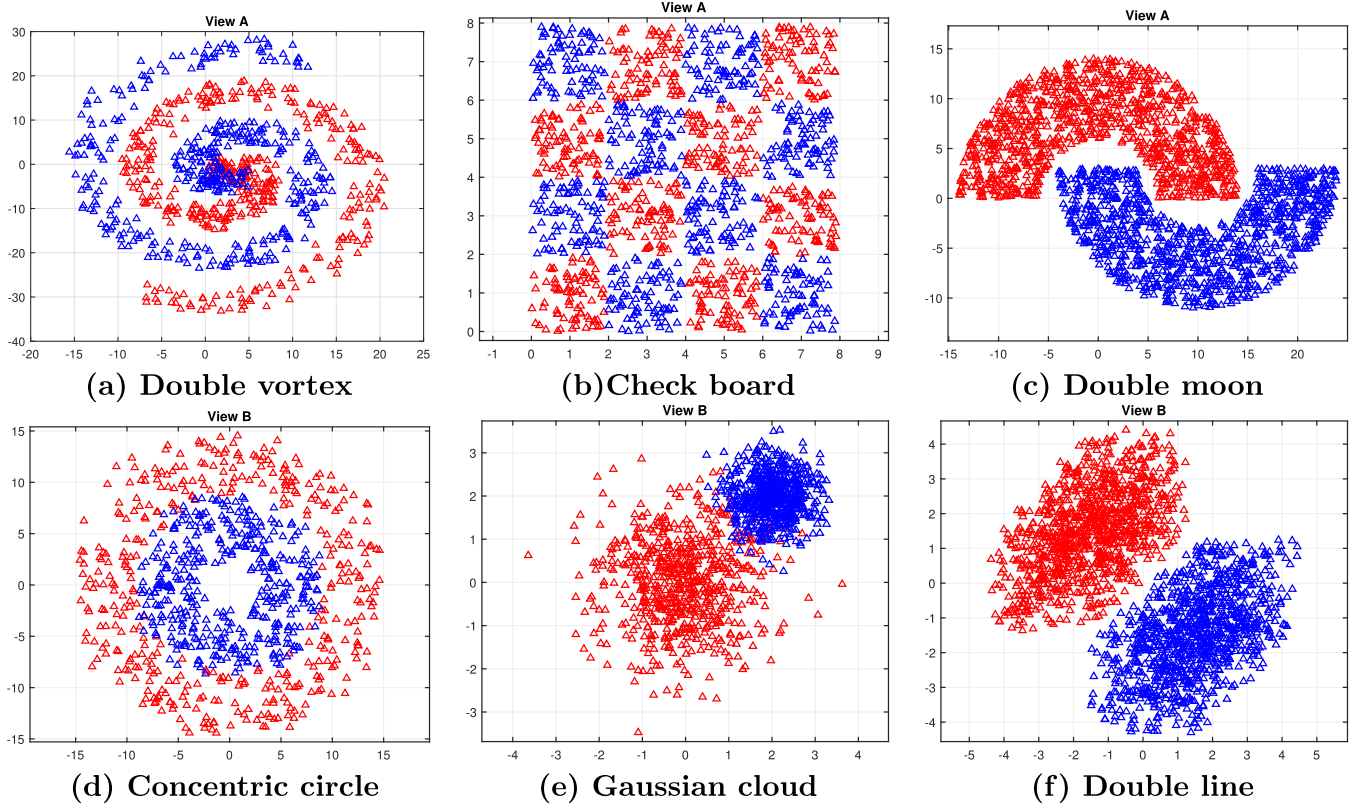


Fig. 4. Overview of three synthetic multi-view datasets.

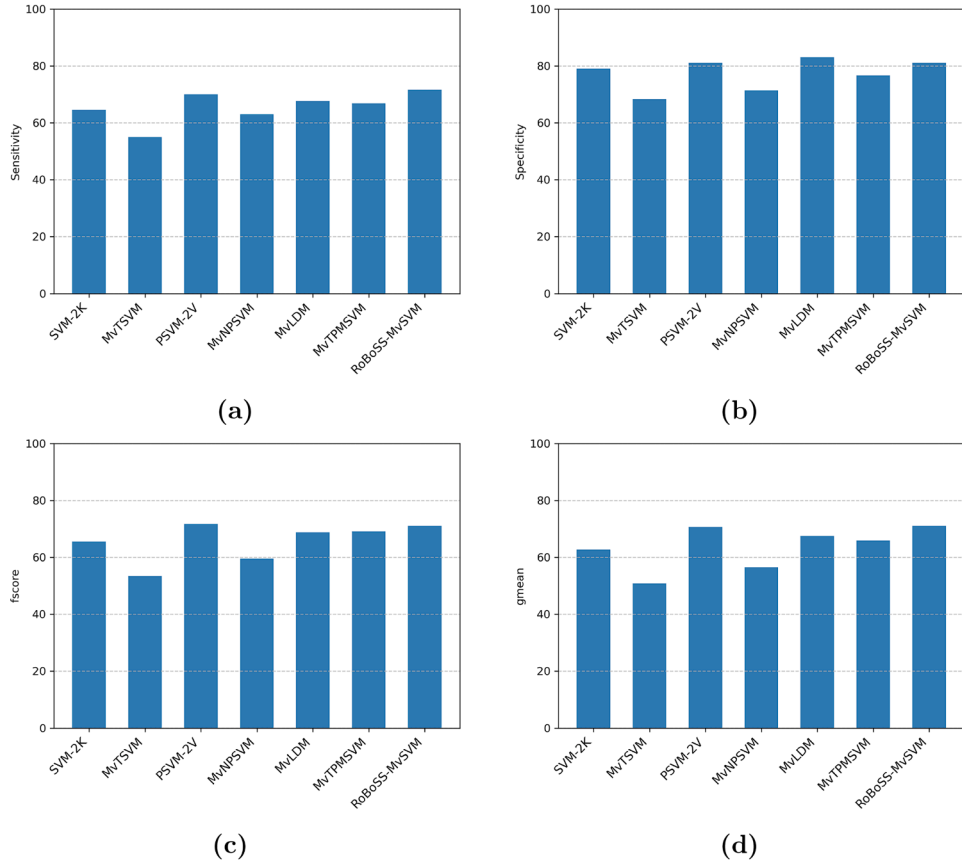


Fig. 5. Average sensitivity, specificity, F-measure, and G-mean on UCI and KEEL datasets.

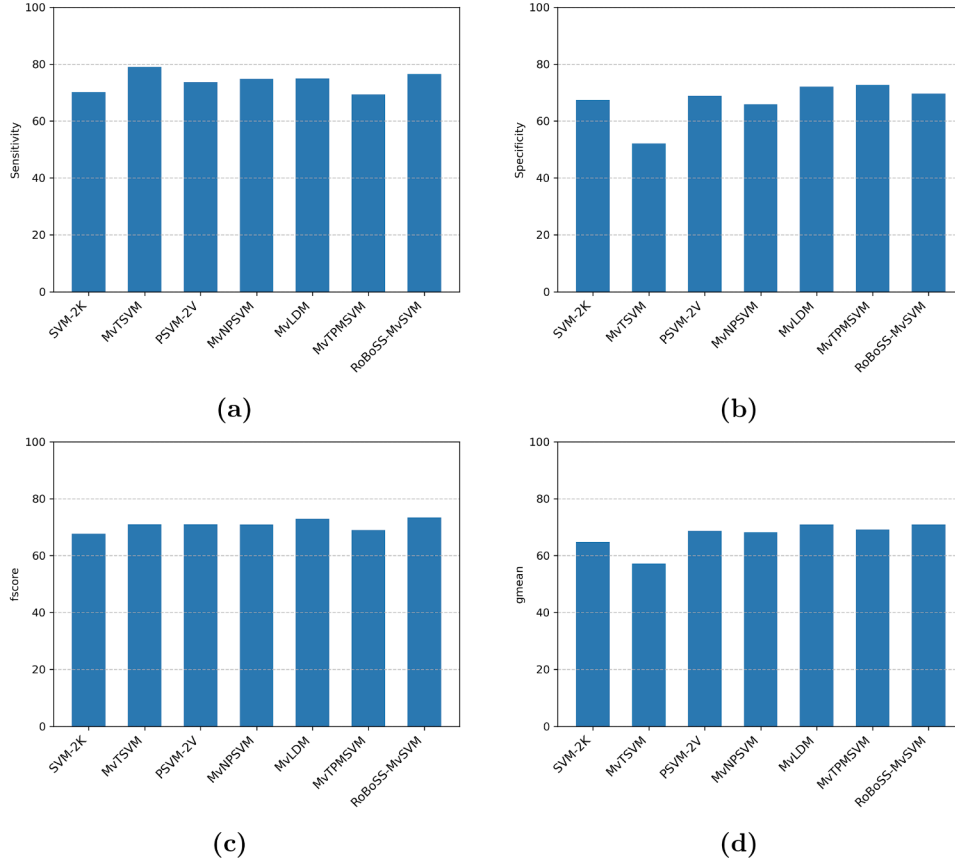


Fig. 6. Average sensitivity, specificity, F-measure, and G-mean on AWA datasets.

and G-mean (70.94%), compared to all existing methods. Furthermore, it attains the second-highest average sensitivity rate of 76.56% after MvTSVM (78.96%), while also maintaining a balanced average specificity of 69.61%. Fig. 6 further validates these findings by providing a graphical summary of the comparative performance across all four evaluation metrics. These comprehensive experimental results demonstrate that the proposed method significantly outperforms existing approaches across all evaluation metrics, establishing its superior effectiveness in diverse MvL scenarios.

5.5. Convergence analysis

To examine the convergence behavior of the proposed RoBoSS-MvSVM, we plot the change of the objective value with respect to the number of iterations on six benchmark datasets, namely monk-2, parkinson, specfheart, vowel0, wine, and WPBC, as shown in Fig. 7. It can be observed that the objective value decreases rapidly during the initial iterations and then gradually stabilizes, indicating that the proposed iterative method converges to a stable solution. Furthermore, a steady and decreasing trend across all datasets indicates a stable and reliable training process. Overall, the stabilization of the objective function and a clear monotonic downward trend confirms the effectiveness and robustness of the proposed optimization strategy for learning RoBoSS-MvSVM.

5.6. Sensitivity analysis

In this section, we conduct the sensitivity analyses of the proposed RoBoSS-MvSVM algorithm by examining the influence of its key hyperparameters. First, we evaluate the impact of the penalty parameter C on the performance of the proposed method and compare it with baseline

models. The experiments are conducted on six datasets: ecoli1, glass1, haberman, monk-2, newthyroid, and planning. The value of C is varied over the set $\{10^{-2}, 10^{-1}, \dots, 10^2\}$, while all other hyperparameters are fixed at their respective optimal values. The classification accuracy obtained in this experiment for each dataset is presented in Fig. 8. Compared to other classifiers, the proposed method demonstrates strong performance across a wide range of C values, particularly for small to moderate values. This suggests that allowing slightly larger penalties for misclassified samples can enhance the model's ability to distinguish challenging samples while preserving robustness. In the second experiment, we analyze the influence of the penalty parameter C and kernel width σ on six randomly selected multi-view datasets (appendicitis, breast cancer, heart, parkinson, sonar, vote) by simultaneously varying the two hyperparameters. The parameter C is selected from the set $\{10^{-2}, 10^{-1}, \dots, 10^2\}$ and σ is selected from the set $\{2^{-4}, 2^{-3}, \dots, 2^4\}$. The experimental results are presented in Fig. 9, where classification accuracy is plotted along the z -axis, with C and σ corresponding to the x - and y -axes, respectively. The results indicate that the proposed method achieves consistently high performance for moderate to large values of C combined with intermediate values of σ , while performance declines for very small C and extreme σ values. These observations highlight the importance of appropriate hyperparameter selection for optimal generalization performance.

5.7. Experiments on UCI and KEEL datasets with added Gaussian noise

To assess the robustness of the proposed RoBoSS-MvSVM against noisy data, experiments are performed on ten benchmark datasets by introducing Gaussian noise at different intensity levels (5%, 10%, 15%, and 20%). The classification performance of the proposed method, compared against several baseline models, is summarized in Table 11. The

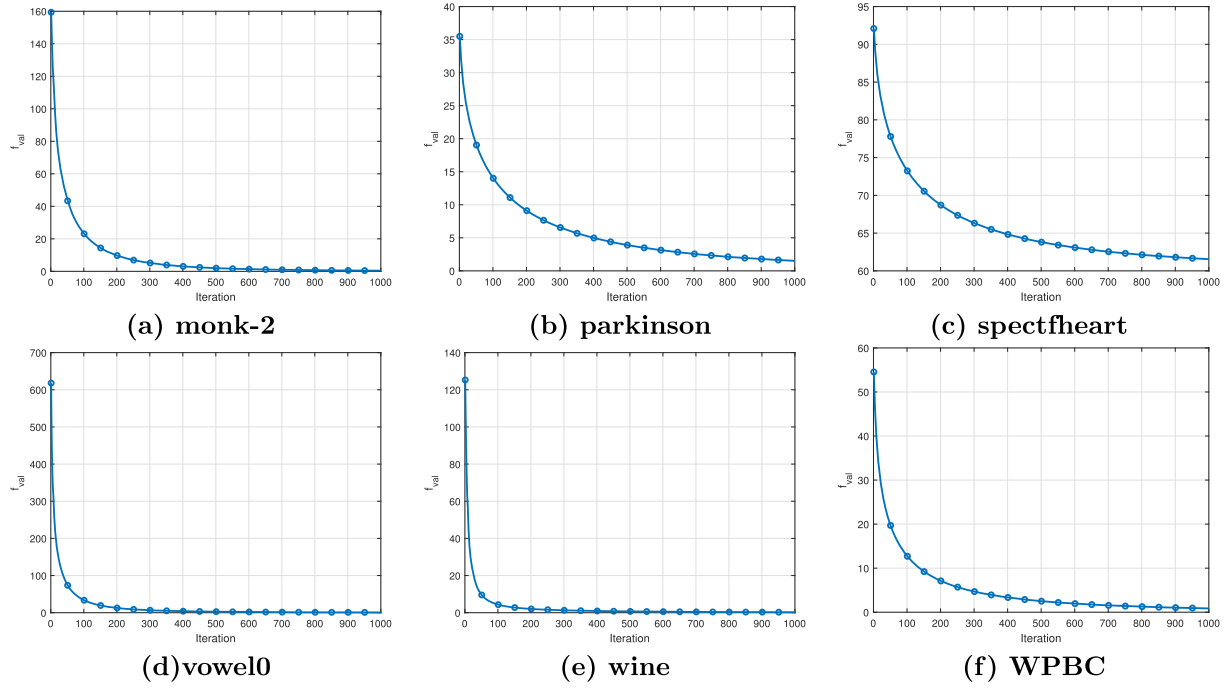


Fig. 7. Convergence analysis of the proposed RoBoSS-MvSVM on six benchmark datasets.

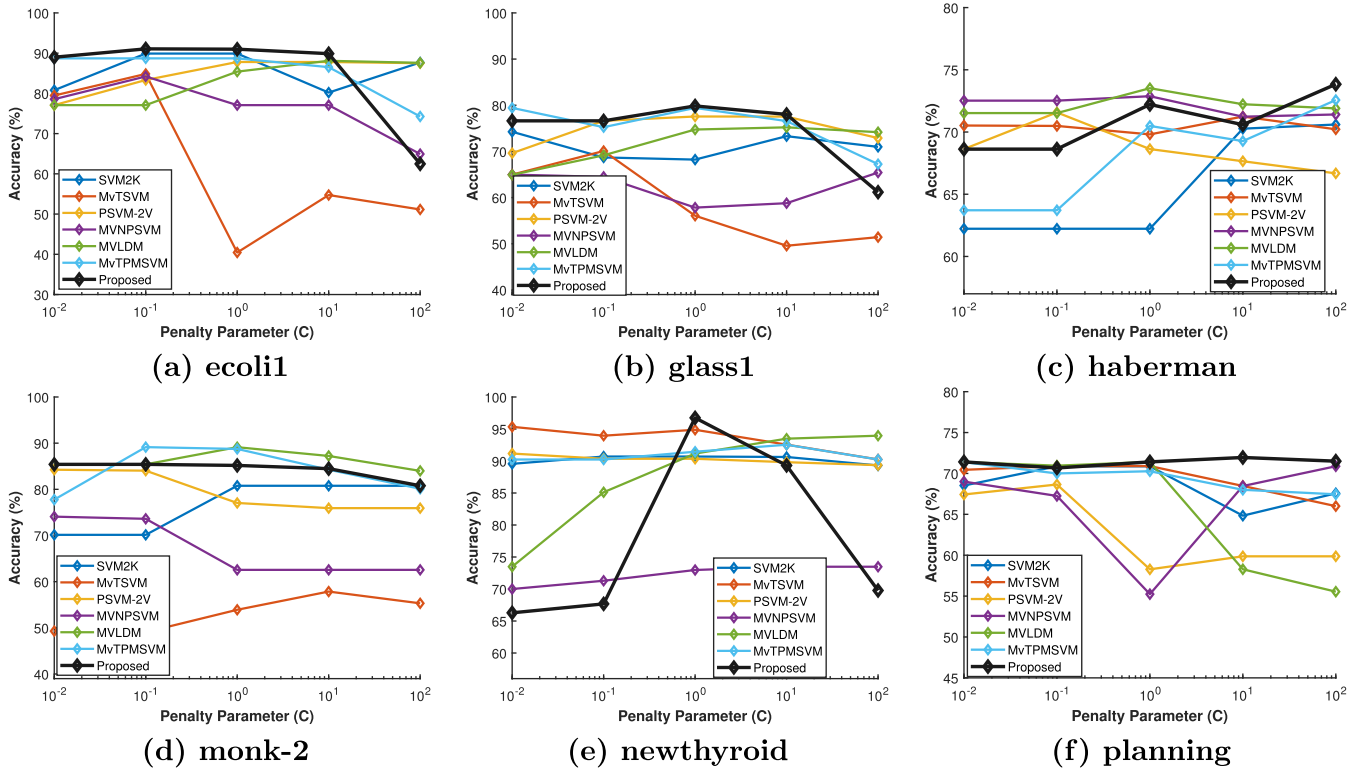


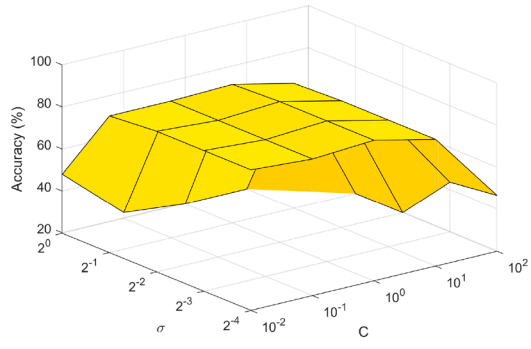
Fig. 8. Impact of varying C on the performance of proposed and baseline models.

proposed model consistently achieves superior accuracy across the majority of the datasets and noise levels, demonstrating strong resilience to label noise. Specifically, on the ecoli1 dataset, it maintains stable performance across all noise levels, achieving the highest average accuracy of 89.96%. Additionally, for the appendicitis, glass1, hepatitis, vote, and WPBC datasets, the proposed method attains the highest average accuracies of 83.76%, 74.35%, 82.10%, 92.18%, and 76.99%, respectively. These results highlight the effectiveness of the proposed approach in

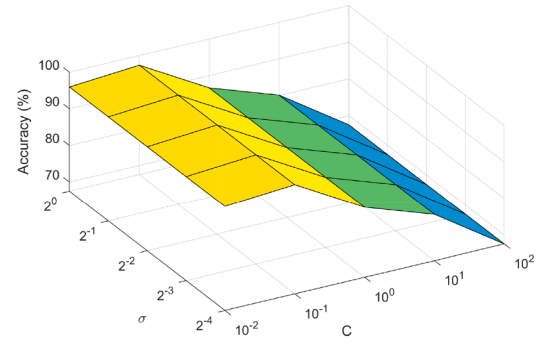
preserving high classification accuracy under noisy conditions, further demonstrating its robustness and practical applicability to real-world applications involving noisy datasets.

5.8. Experiments on UCI and KEEL datasets with added label noise

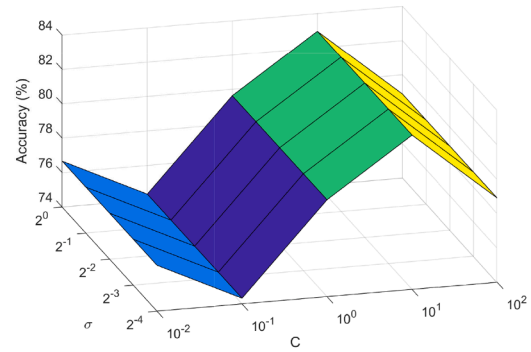
In addition to feature perturbations, we further assess the robustness of RoBoSS-MvSVM against label noise, which poses a significant



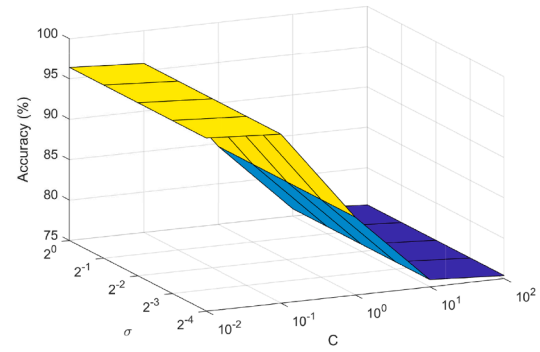
(a) appendicitis



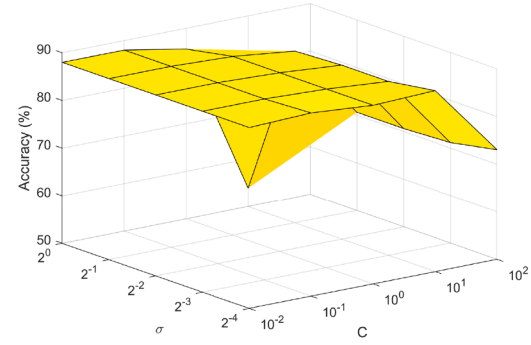
(b) breast cancer



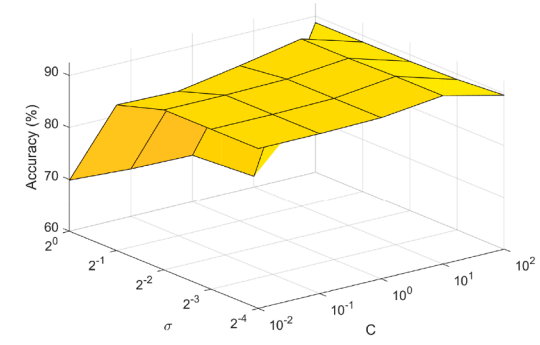
(c) heart



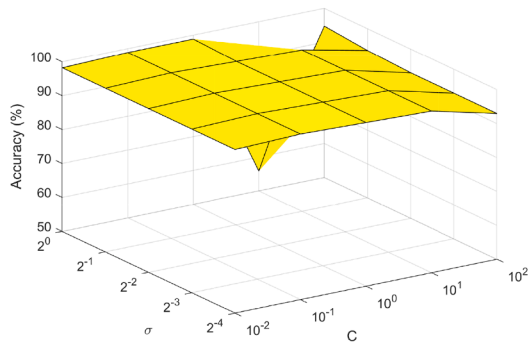
(d) parkinson



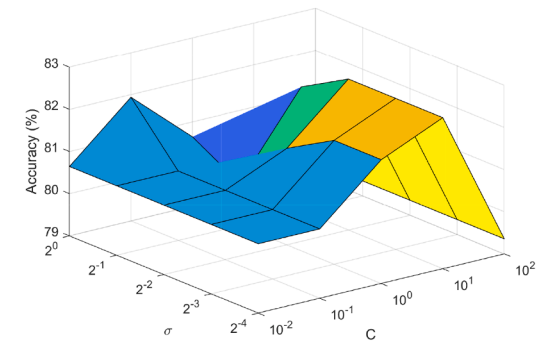
(e) sonar



(f) vote



(g) divorce



(h) hepatitis

Fig. 9. Impact of C and σ on the proposed RoBoSS-MvSVM.

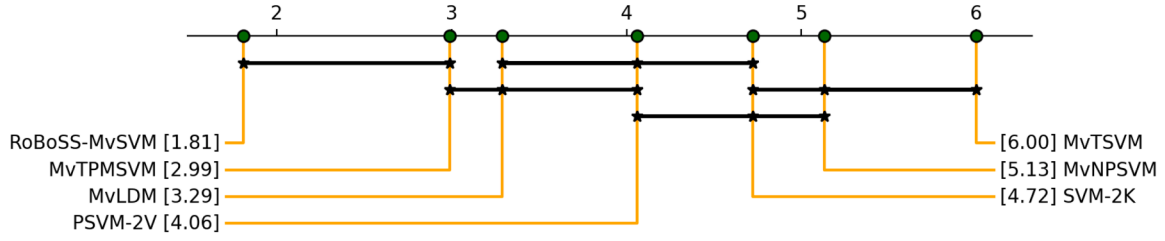


Fig. 10. Critical difference diagram on benchmark datasets.

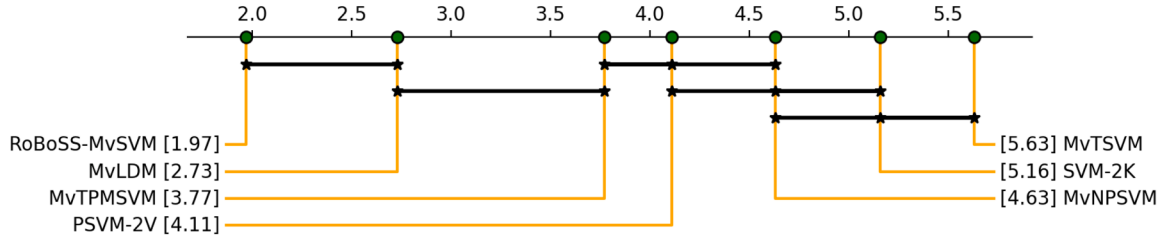


Fig. 11. Critical difference diagram on AWA datasets.

problem in real-world scenarios when training labels are intrinsically distorted. In our comparative analysis, we inject label noise at four different intensity levels (5%, 10%, 15%, and 20%) and evaluate performance on the ten benchmark datasets as presented in Table 12. The proposed method achieves the highest average accuracy across all noise levels on 8 out of 10 datasets. Moreover, for ecolil, hepatitis, newthyroid, and vote datasets, it consistently yields the greatest performance at every noise level compared to existing methods. This demonstrates its superior robustness against label noise and validates its effectiveness in handling corrupted supervision labels across diverse learning scenarios.

5.9. Ablation studies

To analyze the impact of individual components in the proposed RoBoSS-MvSVM, we perform ablation studies on five randomly selected datasets by carefully excluding essential components of the model, namely the consensus regularization, complementarity term, and the RoBoSS loss function. The influence of the consensus regularization term is removed by setting the parameter $D = 0$, while the effect of the RoBoSS loss function is eliminated by replacing it with its 0–1 loss approximation, setting $\lambda = 1$ and a sufficiently large value of a . All remaining parameters are fixed at their optimal values. The results of the experiment are presented in Table 13. It is observed from Table 13 that removing any component leads to a noticeable decline in classification accuracy for every dataset. Specifically, eliminating the RoBoSS loss results in a notable drop in performance, underscoring its crucial role in the robustness of the proposed method. These findings demonstrate that each component of RoBoSS-MvSVM makes a significant contribution to the overall performance, and their combined integration is essential to achieve superior and stable classification performance.

6. Statistical analysis

To evaluate the statistical significance of the proposed model's performance, we employ the Friedman test followed by the Nemenyi post-hoc test.

6.1. Friedman test

In the Friedman test (Demšar, 2006), classifiers are ranked on each dataset based on their performance, with the highest-performing model receiving rank 1, the second-best receiving rank 2, and so forth. In the

event of a tie, the classifiers share the mean ranks. The Friedman test null hypothesis posits that all classifiers achieve the same mean rank, indicating there is no statistically significant difference in their classification performance. For p classification models evaluated over n datasets, the Friedman statistic follows a chi-square distribution with $p - 1$ degrees of freedom and is calculated as follows:

$$\chi_F^2 = \frac{12n}{p(p+1)} \left(\sum_i R_i^2 - \frac{p(p+1)^2}{4} \right), \quad (34)$$

where R_i denotes the mean rank of i^{th} model. Subsequently, the F_F statistic is calculated, which follows an F -distribution with $p - 1$ and $(p - 1)(n - 1)$ degree of freedom:

$$F_F = \frac{(n - 1)\chi_F^2}{n(p - 1) - \chi_F^2}. \quad (35)$$

For UCI and KEEL datasets, with $p = 7$ and $n = 39$, the computed values of χ_F^2 and F_F are 101.29 and 29, respectively. By referring to the F -distribution table at 5% significance level (α), the critical value for $F_F(6, 228)$ is 2.14. Since the computed F_F value of 29 is substantially greater than this threshold, the null hypothesis is rejected. For the AWA datasets where $p = 7$ and $n = 45$, the calculated values of χ_F^2 and F_F are 94.96 and 23.87, respectively. From the F -distribution table at 5% level of significance (α), the critical value for $F_F(6, 264)$ is 2.14. As the obtained F_F value of 23.87 exceeds this critical threshold, the null hypothesis is rejected. This result indicates a clear and statistically significant difference among the models in both cases, highlighting the effectiveness of the proposed method in achieving consistently superior performance across all datasets.

6.2. Nemenyi test

Following the Friedman test, the Nemenyi post-hoc test (Demšar, 2006) is applied to perform pairwise comparisons among all classification models. Two models are considered to be statistically different if the gap between their average ranks is at least equal to the critical difference (CD). At 5% significance level, CD is calculated as

$$CD = q_\alpha \sqrt{\frac{p(p+1)}{6n}}$$

where p denotes the total number of classification models and n represents the total number of datasets. For the UCI and KEEL datasets, the CD is calculated to be 1.44, and for AWA datasets it is 1.34. To illustrate the differences in mean ranks, critical difference diagrams are presented

in Figs. 10 and 11 for the UCI/KEEL and AwA datasets, respectively. In these diagrams, any pair of classifiers whose rank difference does not exceed the CD is connected by a black line, indicating no statistically significant difference between them. From the figures, it is evident that the proposed method is statistically different from all the baseline models except MvLDM in both cases.

7. Conclusion

In this study, we propose a novel multi-view support vector machine with the RoBoSS loss function (RoBoSS-MvSVM) to address the challenges prevalent in MvL. The proposed method leverages the complementarity and consensus principles by incorporating a weighted scheme and a regularization term to enhance the utilization of multi-view representations and promote robust learning. The sparsity, smoothness, and bounded properties of the RoBoSS loss function enable efficient optimization and robust performance against noise and outliers. Furthermore, the classification-calibrated nature of the RoBoSS loss function and theoretical analysis using Rademacher complexity bounds ensure the robustness and generalization capabilities of the proposed RoBoSS-MvSVM. The optimization problem of the proposed approach is solved using the NAG method. The extensive experiments conducted on synthetic and 84 binary datasets further validate the effectiveness of RoBoSS-MvSVM. The proposed approach consistently outperforms existing state-of-the-art methods, achieving the highest average accuracy of 85.92% on the UCI/KEEL datasets and 73.85% on the AwA dataset, thereby highlighting its superior performance. The results of the hyperparameter sensitivity analysis, ablation studies, experiments with added Gaussian and label noise, and statistical significance tests further confirm the robustness and reliability of the proposed framework. In future work, the twin SVM framework can be extended to incorporate MvL with the RoBoSS loss, which would improve processing efficiency for large-scale datasets. Furthermore, the integration of granular ball methodology could provide additional improvements to both efficiency and scalability of the proposed method.

CRedit authorship contribution statement

Yash Arora: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis, Conceptualization; **S. K. Gupta:** Writing – review & editing, Visualization, Supervision, Resources, Project administration, Methodology, Investigation, Conceptualization; **M. Tanveer:** Writing – review & editing, Methodology, Conceptualization.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We sincerely thank the anonymous reviewers for their valuable comments and suggestions, which have greatly enhanced the quality and presentation of the paper. The first author is grateful to the [Council of Scientific and Industrial Research, India](#), for financial support, to carry out this work.

References

- Akhtar, M., Tanveer, M., & Arshad, M. (2025). RoBoSS: A robust, bounded, sparse, and smooth loss function for supervised learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(1), 149–160.
- Akhtar, M., Tanveer, M., & Arshad, M. (2026). RoBoTS: A robust bounded twin SVM based on RoBoSS loss function. *Pattern Recognition*, 179, 113653. <https://doi.org/10.1016/j.patcog.2026.113653>
- Akhtar, M., Tanveer, M., Arshad, M., & Alzheimer's Disease Neuroimaging Initiative (2024). Advancing supervised learning with the wave loss function: A robust and smooth approach. *Pattern Recognition*, 155, 110637.
- Bartlett, P. L., Jordan, M. L., & McAuliffe, J. D. (2006). Convexity, classification, and risk bounds. *Journal of the American Statistical Association*, 101(473), 138–156.
- Bartlett, P. L., & Mendelson, S. (2023). Rademacher and Gaussian complexities: Risk bounds and structural results. *Journal of Machine Learning Research*, 3, 463–482.
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20, 273–297.
- Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *The Journal of Machine Learning Research*, 7, 1–30.
- Derrac, J., Garcia, S., Sanchez, L., & Herrera, F. (2015). KEEL data-mining software tool: Data set repository, integration of algorithms and experimental analysis framework. *Journal of Multiple-Valued Logic and Soft Computing*, 17, 255–287.
- Dinuzzo, F., & Schölkopf, B. (2012). The representer theorem for Hilbert spaces: A necessary and sufficient condition. *Advances in Neural Information Processing Systems*, 25.
- Dua, D., & Graff, C. (2017). UCI machine learning repository, . Beijing China. <http://archive.ics.uci.edu/ml>
- Farquhar, J., Hardoon, D., Meng, H., Shawe Taylor, J., & Szedmak, S. (2005). Two view learning: SVM-2K, theory and practice. *Advances in Neural Information Processing Systems*, 18.
- Fu, H., Huang, F., Liu, X., Qiu, Y., & Zhang, W. (2022). MVGCN: Data integration through multi-view graph convolutional network for predicting links in biomedical bipartite networks. *Bioinformatics*, 38(2), 426–434.
- Houthuys, L., Langone, R., & Suykens, J. A. K. (2018). Multi-view least squares support vector machines classification. *Neurocomputing*, 282, 78–88.
- Hu, K., Xiao, Y., Zheng, W., Zhu, W., & Hsu, C.-H. (2025). Multiview large margin distribution machine. *IEEE Transactions on Neural Networks and Learning Systems*, 36(2), 2395–2409.
- Jayadeva, Khemchandani, R., & Chandra, S. (2007). Twin support vector machines for pattern classification. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 29(5), 905–910.
- Jing, X. Y., Liu, Q., Wu, F., Xu, B., Zhu, Y. P., & Chen, S. (2015). Web page classification based on uncorrelated semi-supervised intra-view and inter-view manifold discriminant feature extraction. In *Ijcai* (pp. 2255–2261).
- Kirkpatrick, S., Gelatt Jr, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680.
- Kolenda, T., Hansen, L. K., Larsen, J., & Winther, O. (2002). Independent component analysis for understanding multimedia content. In *Proceedings of the 12th IEEE workshop on neural networks for signal processing* (pp. 757–766). IEEE.
- Lampert, C. H., Nickisch, H., & Harmeling, S. (2013). Attribute-based classification for zero-shot visual object categorization. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 36(3), 453–465.
- Lou, C., & Xie, X. (2023). Multi-view intuitionistic fuzzy support vector machines with insensitive pinball loss for classification of noisy data. *Neurocomputing*, 549, 126458.
- Nesterov, Y. (1983). A method for unconstrained convex minimization problem with the rate of convergence $O(1/k^2)$. In *Doklady akademii nauk SSSR* (pp. 543–547). (vol. 269).
- Qian, N. (1999). On the momentum term in gradient descent learning algorithms. *Neural Networks*, 12(1), 145–151.
- Quadir, A., Akhtar, M., & Tanveer, M. (2025a). Enhancing multiview synergy: Robust learning by exploiting the wave loss function with consensus and complementarity principles. *Neural Networks*, 188, 107433.
- Quadir, A., Sajid, M., Akhtar, M., & Tanveer, M. (2025b). Twin restricted kernel machines for multiview classification. In *2025 international joint conference on neural networks (IJCNN)* (pp. 1–8). IEEE.
- Quadir, A., & Tanveer, M. (2024). Multiview learning with twin parametric margin SVM. *Neural Networks*, 180, 106598.
- Quadir, A., & Tanveer, M. (2025). Granular ball twin support vector machine with pinball loss function. *IEEE Transactions on Computational Social Systems*, 12(5), 3891–3900.
- Ruder, S. (2016). An overview of gradient descent optimization algorithms. [arXiv:1609.04747](https://arxiv.org/abs/1609.04747).
- Sun, S. (2011). Multi-view laplacian support vector machines. In *International conference on advanced data mining and applications* (pp. 209–222). Springer.
- Sun, S., Xie, X., & Dong, C. (2018). Multiview learning with generalized eigenvalue proximal support vector machines. *IEEE Transactions on Cybernetics*, 49(2), 688–697.
- Tang, J., Li, D., Tian, Y., & Liu, D. (2018a). Multi-view learning based on nonparallel support vector machine. *Knowledge-Based Systems*, 158, 94–108.
- Tang, J., Tian, Y., Zhang, P., & Liu, X. (2018b). Multiview privileged support vector machines. *IEEE Transactions on Neural Networks and Learning Systems*, 29(8), 3463–3477.
- Tang, J., Xu, W., Li, J., Tian, Y., & Xu, S. (2021). Multi-view learning methods with the LINEX loss for pattern classification. *Knowledge-Based Systems*, 228, 107285.
- Tang, J., Yi, Q., Fu, S., & Tian, Y. (2024). Incomplete multi-view learning: Review, analysis, and prospects. *Applied Soft Computing*, 153, 111278.
- Tanveer, M., Rajani, T., Rastogi, R., Shao, Y.-H., & Ganaie, M. A. (2024). Comprehensive review on twin support vector machines. *Annals of Operations Research*, 339(3), 1223–1268.
- Wang, H., Zhu, J., & Zhang, S. (2023). Safe screening rules for multi-view support vector machines. *Neural Networks*, 166, 326–343.

- Wang, W., & Zhou, Z. H. (2010). A new analysis of co-training. In *27th international conference on machine learning* (pp. 1135–1142).
- Wang, Y., Zhang, W., Wu, L., Lin, X., & Zhao, X. (2015). Unsupervised metric fusion over multiview data by graph random walk-based cross-view diffusion. *IEEE Transactions on Neural Networks and Learning Systems*, 28(1), 57–70.
- Xiao, Y., Pan, G., Liu, B., Zhao, L., Kong, X., & Hao, Z. (2024). Privileged multi-view one-class support vector machine. *Neurocomputing*, 572, 127186.
- Xie, X., Li, Y., & Sun, S. (2023). Deep multi-view multiclass twin support vector machines. *Information Fusion*, 91, 80–92.
- Xie, X., & Sun, S. (2015). Multi-view twin support vector machines. *Intelligent Data Analysis*, 19(4), 701–712.
- Yang, Y., Lan, C., Li, X., Luo, B., & Huan, J. (2014). Automatic social circle detection using multi-view clustering. In *Proceedings of the 23rd ACM international conference on conference on information and knowledge management* (pp. 1019–1028).
- Ye, Q., Huang, P., Zhang, Z., Zheng, Y., Fu, L., & Yang, W. (2021). Multiview learning with robust double-sided twin SVM. *IEEE Transactions on Cybernetics*, 52(12), 12745–12758.
- Zhao, N., & Bu, J. (2022). Robust multi-view subspace clustering based on consensus representation and orthogonal diversity. *Neural Networks*, 150, 102–111.
- Zheng, Y., Huang, H., Luo, Y., Qiu, Y., Wang, A., Zhou, G., & Zhao, Q. (2026). Multi-view subspace tensorization with attentive clustering embedding. *Neural Networks*, 196, 108344.
- Zheng, Y., Zhou, G., Huang, H., Luo, X., Huang, Z., & Zhao, Q. (2025). Unifying complete and incomplete multi-view clustering through an information-theoretic generative model. *Neural Networks*, 182, 106901.
- Zhu, J., Wang, H., Li, H., & Zhang, Q. (2022). Fast multi-view twin hypersphere support vector machine with consensus and complementary principles. *Applied Intelligence*, 52(11), 12684–12703.